

MVP - Machine Learning & Analytics

Nome: Max Brandão de Oliveira

Matrícula: 4052025002409

Dataset: Dados de sinistros de acidentes de trânsito em rodovias federais entre 2007 e 2026 [DATATRAN](#)

INTRODUÇÃO E MOTIVAÇÃO

A segurança no trânsito é um aspecto de extrema relevância mundial. De acordo com a Organização Mundial da Saúde (OMS), estima-se que, a cada ano, cerca de 1,25 milhões de pessoas morrem no trânsito e até 50 milhões sobrevivem com ferimentos e sequelas, resultando em custos pessoais, sociais e econômicos. Por exemplo, meu pai faleceu em um acidente de trânsito em 1989 quando o mesmo tinha 26 anos de idade e eu tinha apenas 1 ano.

Os acidentes de trânsito são uma das principais causas de morte entre jovens com idades entre 15 e 29 anos (OMS, 2018). Em 2020, para discutir estratégias para reduzir as mortes, a OMS organizou a terceira Conferência Ministerial Global sobre Segurança no Trânsito (OMS, 2020). No Brasil, de acordo com os dados divulgados pela Confederação Nacional do Transporte (CNT), o número de acidentes nas rodovias federais brasileiras aumentou em 1,6% em 2021 em relação a 2020, totalizando 64.452 casos. Além disso, houve um aumento de 2% no número de mortes, passando de 5.287 em 2020 para 5.391 em 2021, de acordo com os dados publicados pela Polícia Rodoviária Federal (PRF) (TRANSPORTE, CNT, 2021).

DESCRIÇÃO DO PROBLEMA

Esses acidentes oneram o sistema de saúde e a economia constantemente, bem como saturam a rede de emergência pública e privada. É preciso destacar que as sequelas permanentes decorrentes de acidentes de trânsito podem ter um impacto significativo na vida das pessoas, afetando sua capacidade de trabalho, lazer e convivência social. De acordo com informações divulgadas pelo Observatório Nacional de Segurança Viária (ONSV), uma organização dedicada a redução dos índices de acidentes de trânsito no Brasil, as internações decorrentes de acidentes rodoviários têm gerado um custo anual de R\$ 52 bilhões para o sistema público de saúde. Esse valor equivale a 2,8% do Produto Interno Bruto (PIB) do país e leva em consideração despesas com tratamentos médicos, indenizações, perda de produtividade decorrente de lesões ou mortes e gastos associados aos veículos.

Esses valores evidenciam o impacto econômico causado pelos acidentes de trânsito na sociedade. Desta forma, é fundamental investir em políticas públicas que promovam a segurança no trânsito e o suporte adaptado às vítimas de acidentes. Portanto, identificar os fatores que podem contribuir para acidentes que resultam em óbito é fundamental para o desenvolvimento de estratégias eficazes destinadas a reduzir drasticamente o número de mortes.

OBJETIVO

Este estudo tem como objetivo mapear os acidentes de trânsito com vítimas fatais no país entre 2007 e 2026 e **avaliar e identificar** possíveis fatores de risco para ocorrência de óbito nos sinistros nas rodovias federais. Em outras palavras, o presente trabalho busca de identificar fatores de risco relacionados às ocorrências de acidentes fatais nas rodovias brasileiras de 2007 a 2026 registrados pela Polícia Rodoviária Federal (PRF) através da aplicação de modelos de classificação pertencentes aos métodos de **aprendizado supervisionado e não supervisionado**.

HIPÓTESES DO PROBLEMA

No estudo de Moraes et al. (2014) os autores utilizaram a regressão logística para estudar a letalidade dos acidentes de trânsito na cidade de Palmas-TO, o objetivo principal foi prever a incidência de acidentes de trânsito de acordo com sua gravidade, sem vítimas, com vítimas ou com vítimas fatais, utilizando o Modelo Logit Ordenado. Assim, os autores constataram que há maior probabilidade de acidentes ocorrem aos domingos, sendo que os acidentes envolvendo bicicletas e motocicletas apresentam o maior número de vítimas fatais. Portanto, as hipóteses eluciadas consistem em:

1. **A chance de óbito em rodovias federais tem relação com o dia da semana;**
 2. **A probabilidade de ocorrência de óbito em sinistros está relacionada ao tipo de pista;**
 3. **A condição de visibilidade associada ao clima pode contribuir para ocorrência de óbito nos acidentes em rodovias federais;**
 4. **A região do país tem relação influencia na chance de óbito no sinistro de trânsito em rodovias federais;**
-

TIPO DE PROBLEMA

Diante do exposto, a estatística descritiva e inferencial pode ser útil na identificação de fatores relacionados à acidentes fatais, principalmente através de modelos complexos de **aprendizado supervisionado** empregado em modelos de **classificação** Segundo Agresti (2019), o modelo de regressão logística é o mais popular para dados binários, sendo que esse modelo é descrito como um Modelo Linear Generalizado (GLM) para um componente aleatório binomial com a função de ligação logit. Como exemplo disso, em seu Trabalho de Conclusão de Curso (TCC), Rosemberg (2018) aplicou a análise de regressão logística em seu estudo sobre a Incidência de Acidentes Fatais no Trânsito do Distrito Federal.

Este estudo apresenta resultados que podem orientar os órgãos competentes a adotarem medidas preventivas e, conseqüentemente, reduzir a quantidade de acidentes fatais nas rodovias federais. Deseja-se que os resultados possam contribuir com a gestão da segurança rodoviária, dado que o Código de Trânsito Brasileiro (CTB) assegura que o trânsito em condições seguras é um direito de todos e dever dos órgãos e entidades

componentes do Sistema Nacional de Trânsito (BRASIL, 1997), sendo uma importante atividade realizada pelos motoristas de vias e agentes de segurança pública.

Portanto, os resultados apresentados neste estudo podem ser valiosos para apoiar os esforços de gerenciamento de segurança rodoviária. Este é um problema de **classificação supervisionada** com o propósito de identificar o perfil de acidentes com vítimas fatais dentre as ocorrências de sinistros nas rodovias federais entre 2007 e 2026.

SELEÇÃO DE DADOS

Os dados retratam em cada linha um conjunto de atributos relacionados a cada sinistro ocorrido nas rodovias federais do Brasil. Os arquivos estão disponíveis no sítio <https://dados.gov.br/dados/conjuntos-dados/sinistros-de-transito-agrupados-por-ocorrencia>, que leva ao link <https://dados.gov.br/dados/conjuntos-dados/sinistros-de-transito-agrupados-por-ocorrencia>, onde estão localizados arquivos no formato .csv em arquivos zipados (.zip). Cada arquivo é intitulado `datatranXXXX.zip`, em que "XXXX" representa o ano referente aos dados. Há disponibilidade de 2007 até 2026, de modo que 2026 encontra-se incompleto.

Atributos do Dataset

Cada dataset possui **TRINTA** atributos originalmente declarados na seguinte forma:

- **ano** (int64): Identifica o ano do sinistro;
- **id** (float64): Um identificador único para cada registro no conjunto de dados, permitindo fácil referência e rastreamento de entradas individuais;
- **data_inversa** (str): A data em que o evento ocorreu, fornecendo um contexto temporal para os dados;
- **dia_semana** (str): O horário exato em que o evento ocorreu, permitindo uma análise temporal detalhada;
- **horario** (str): O horário exato em que o evento ocorreu, permitindo uma análise temporal detalhada;
- **uf** (str): Representa o estado onde o evento ocorreu, o que é essencial para a análise regional;
- **br** (int64): O número da rodovia federal associado ao evento, útil para identificar rotas específicas;
- **km** (str): O marcador quilométrico na rodovia onde o incidente ocorreu fornece informações precisas sobre a localização;
- **municipio** (str): O município onde o evento ocorreu, auxiliando na análise e na elaboração de relatórios locais;
- **causa_acidente** (str): Descreve a causa do acidente, o que é fundamental para compreender os fatores que contribuíram para ele;

- **tipo_acidente** (str): Classifica o tipo de acidente, ajudando a identificar tendências e cenários comuns;
- **classificacao_acidente** (str): Classifica a gravidade do acidente, o que é importante para avaliar o impacto e as necessidades de resposta;
- **fase_dia** (str): Indica a fase do dia (por exemplo, manhã, tarde, noite) em que o evento ocorreu, útil para análises baseadas no tempo;
- **sentido_via** (str): Descreve a direção do fluxo de tráfego na estrada no momento do evento, o que pode influenciar a dinâmica do acidente;
- **condicao_metereologica** (str): Descreve detalhadamente as condições meteorológicas no momento do evento, que podem ser um fator significativo na ocorrência de acidentes;
- **tipo_pista** (str): Especifica o tipo de superfície da estrada, por exemplo, asfalto, cascalho, onde o evento ocorreu, relevante para avaliações de segurança;
- **tracado_via** (str): Descreve o traçado da estrada por exemplo, reta, curva, o que pode influenciar as taxas de acidentes;
- **uso_solo** (str): Indica o tipo de uso do solo na área ao redor do evento, o que pode fornecer contexto para os padrões de tráfego;
- **pessoas** (int): Número total de pessoas envolvidas no evento é importante para entender a dimensão do incidente;
- **mortos** (int64): Número de fatalidades resultantes do evento é uma métrica crucial para avaliar a gravidade dos acidentes;
- **feridos_leves** (int64): Contabiliza o número de indivíduos com ferimentos leves, o que é útil para compreender o impacto nas vítimas;
- **feridos_graves** (int64): Número de feridos graves, que necessitam de atendimento de urgência e oferecem risco aos pacientes;
- **ilesos** (int64): Contabiliza o número de envolvidos que não sofreram lesões no sinistro;
- **ignorados** (int64): Indica o número de casos ignorados no sinistro;
- **feridos** (int64): Contabiliza o número de feridos envolvidos no sinistro;
- **veiculos** (int64): Indica a quantidade de veículos envolvidos no sinistro;
- **latitude** (float64): Indica no mapa a latitude do sinistro;
- **longitude** (float64): Indica no mapa a longitude do sinistro;
- **regional** (str): Indica a codificação de qual estado pertence a delegacia que atendeu ao sinistro;
- **delegacia** (str): Indica qual delegacia foi responsável por atender e registrar a ocorrência do sinistro, já que em cada estado existe diversas delegacias;

- **uop** (str): Se refere a qual unidade de uma delegacia foi responsável pelo atendimento do sinistro.

De posse dessas variáveis originais, teremos foco na presença de óbito no sinistro, bem como na condição de visibilidade no momento da ocorrência. Cada registro nos permitirá extrair outras informações, como mês, turno, visibilidade e outras.

Importação das Bibliotecas Necessárias

Esta seção consolida todas as importações de bibliotecas necessárias para a análise, visualização e pré-processamento dos dados, bem como o carregamento inicial dos datasets datatran de 2007 a 2026.

```
import pandas as pd, unicodedata, re
import numpy as np
import seaborn as sns
import gdown
import shutil
from tqdm import tqdm
!pip install prince
!pip install contextily --quiet
import prince, matplotlib.pyplot as plt, seaborn as sns, pandas as pd
from scipy.stats import chi2_contingency
import statsmodels.api as sm
import os, zipfile, tempfile
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.preprocessing import OneHotEncoder
from sklearn.decomposition import TruncatedSVD
import matplotlib.pyplot as plt, seaborn as sns, pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
from sklearn.decomposition import PCA
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, silhouette_samples
import umap
import requests
import pandas as pd
from io import StringIO
import csv
import geopandas as gpd
import matplotlib.pyplot as plt
from shapely.geometry import Point
import contextily as ctx
!pip install kmodes gower sklearn-extra
```

Leitura dos dados no formato .csv

Os dados estão salvos nos links disponibilizados no google através do sítio com link descrito na seção anterior. No código abaixo, a importação dos dados se dá diretamente pelo link público no google drive já mencionado anteriormente. Os .zip's são baixados, os .csv são extraídos, são observadas as variáveis presentes e, finalmente, todos os arquivos são concatenados em um único arquivo chamado de "data_07_26" que contém as informações de todos os anos. A leitura dos bancos de dados e a concatenação demanda um pouco mais de tempo e memória, mas não compromete a execução da análise.

```
# Pasta pública do Google Drive (sua URL)
drive_folder_url = "https://drive.google.com/drive/folders/1el64p7H7LRc-
wWDnwjldYmYbWurMbwbT"

# Parâmetros de leitura
encodings_to_try = ["utf-8", "latin1", "cp1252"]
sep = ";"          # ajuste se necessário
decimal = "."       # ajuste se necessário
years_to_keep = list(range(2007, 2027)) # 2007..2026 inclusive

# Pasta temporária para downloads/extrações
tmp_root = tempfile.mkdtemp(prefix="datatran_root_")
downloads_dir = os.path.join(tmp_root, "downloads")
os.makedirs(downloads_dir, exist_ok=True)

# 1) Baixa todo o conteúdo da pasta (gdown consegue baixar diretório público)
print("Baixando arquivos da pasta do Drive para:", downloads_dir)
# gdown.download_folder aceita URL ou folder id; quiet=False para ver progresso
gdown.download_folder(drive_folder_url, output=downloads_dir, quiet=False,
use_cookies=False)

# 2) Procura pelos ZIPs que combinam com datatranYYYY.zip e processa
def find_zip_for_years(base_dir, years):
    mapping = {}
    for root, _, files in os.walk(base_dir):
        for fn in files:
            name_lower = fn.lower()
            for y in years:
                target = f"datatran{y}.zip"
                if name_lower == target:
                    mapping[y] = os.path.join(root, fn)
    return mapping

zip_map = find_zip_for_years(downloads_dir, years_to_keep)
print(f"Zips encontrados para {len(zip_map)} anos:", sorted(zip_map.keys()))

# encodings_to_try, sep, decimal, years_to_keep, downloads_dir, zip_map

def read_csv_from_zip(zip_path, csv_name, encodings, sep=';', decimal='.'):
    with zipfile.ZipFile(zip_path, 'r') as z:
        candidates = [n for n in z.namelist() if os.path.basename(n).lower() ==
csv_name.lower()]
```

```

        if not candidates:
            raise FileNotFoundError(f"{csv_name} not found in {zip_path}")
        member = candidates[0]
        last_exc = None
        for enc in encodings:
            try:
                with z.open(member) as f:
                    df = pd.read_csv(f, sep=sep, decimal=decimal, encoding=enc,
low_memory=False)
                    return df
            except Exception as e:
                last_exc = e
        raise last_exc

dfs = []
missing_years = []
for y in years_to_keep:
    zip_path = zip_map.get(y)
    csv_name = f"datatran{y}.csv"
    if not zip_path:
        missing_years.append(y)
        print(f"ZIP não encontrado para {y}: esperado datatran{y}.zip")
        continue
    try:
        df = read_csv_from_zip(zip_path, csv_name, encodings_to_try, sep=sep,
decimal=decimal)
        df['__year'] = y
        dfs.append(df)
        print(f"Carregado {csv_name} de {os.path.basename(zip_path)}: shape {df.shape}")
    except Exception as e:
        print(f"Falha ao ler {csv_name} em {os.path.basename(zip_path)}: {e}")
        missing_years.append(y)
if not dfs:
    raise RuntimeError("Nenhum arquivo CSV foi carregado com sucesso.")

# Partindo de: dfs (lista de DataFrames lidos) e missing_years
# 4) Frequência de presença de colunas entre os arquivos
col_presence = {}
per_year_rowcount = {}
for df in dfs:
    # cada df deve ter coluna '__year' com o ano
    year = int(df['__year'].iloc[0]) if '__year' in df.columns else None
    per_year_rowcount[year] = len(df)
    for c in set(df.columns):
        col_presence[c] = col_presence.get(c, 0) + 1

# 5) Ordenar colunas por presença desc, nome asc; garantir '__year' primeiro
cols_sorted = sorted(col_presence.items(), key=lambda x: (-x[1], x[0]))
cols_ordered = [c for c, _ in cols_sorted]
if '__year' in cols_ordered:
    cols_ordered.remove('__year')
    cols_ordered = ['__year'] + cols_ordered

```

```

# 6) Escolher TARGET_COLS = 31
TARGET_COLS = 31
final_cols = cols_ordered.copy()

if len(final_cols) < TARGET_COLS:
    n_extra = TARGET_COLS - len(final_cols)
    final_cols += [f"extra_col_{i+1}" for i in range(n_extra)]
    print(f"Menos de {TARGET_COLS} colunas únicas encontradas. Adicionadas {n_extra} colunas placeholder.")
elif len(final_cols) > TARGET_COLS:
    # manter __year e as colunas mais frequentes
    if '__year' in final_cols:
        keep = ['__year'] + final_cols[1:TARGET_COLS]
    else:
        keep = final_cols[:TARGET_COLS]
    dropped = [c for c in final_cols if c not in keep]
    print(f"Mais de {TARGET_COLS} colunas únicas ({len(final_cols)}). Selecionando as {TARGET_COLS} mais frequentes.")
    print("Colunas removidas:", dropped)
    final_cols = keep

# Garante que 'dfs' existe e contém os 20 dataframes lidos
if 'dfs' not in globals() or not isinstance(dfs, list) or len(dfs) == 0:
    raise RuntimeError("Lista 'dfs' não encontrada ou vazia. Execute a leitura dos zips primeiro.")

# recompute per-year counts (segurança)
per_year_rowcount = {}
for df in dfs:
    if '__year' in df.columns:
        y = int(df['__year'].iloc[0])
    else:
        # se não, atribui None+index para contagem
        y = None
    per_year_rowcount.setdefault(y, 0)
    per_year_rowcount[y] += len(df) # soma caso haja múltiplos dfs com mesmo '__year'

# Se quiser garantir 20 arquivos, confirme:
print("Número de dataframes em 'dfs':", len(dfs))

# Se já definiu final_cols antes, usa; senão, define final_cols como união mais frequente (mantendo '__year' primeiro)
TARGET_COLS = 31
if 'final_cols' not in globals():
    col_presence = {}
    for df in dfs:
        for c in set(df.columns):
            col_presence[c] = col_presence.get(c, 0) + 1
    cols_sorted = sorted(col_presence.items(), key=lambda x: (-x[1], x[0]))
    cols_ordered = [c for c, _ in cols_sorted]
    if '__year' in cols_ordered:

```

```

cols_ordered.remove('__year')
cols_ordered = ['__year'] + cols_ordered
final_cols = cols_ordered
if len(final_cols) < TARGET_COLS:
    final_cols += [f"extra_col_{i+1}" for i in range(TARGET_COLS - len(final_cols))]
elif len(final_cols) > TARGET_COLS:
    if '__year' in final_cols:
        final_cols = ['__year'] + final_cols[1:TARGET_COLS]
    else:
        final_cols = final_cols[:TARGET_COLS]

# Reindexa e concatena
aligned_dfs = [df.reindex(columns=final_cols) for df in dfs]
data_07_26 = pd.concat(aligned_dfs, ignore_index=True, sort=False)

# Obtendo o mês da cada sinistro
s = data_07_26['data_inversa'].astype(str).str.strip()

def extract_month_series(s):
    # parte antes do espaço (remove hora)
    s0 = s.str.split().str[0].fillna("")
    # iniciais vazias
    mes = pd.Series(pd.NA, index=s.index, dtype="Int64")

    # DD/MM/YYYY ou DD/MM/YY -> captura o segundo grupo
    m1 = s0.str.match(r'^\s*(\d{1,2})/(\d{1,2})/(\d{2,4})\s*$', na=False)
    if m1.any():
        # extrai via regex groups
        meses = s0[m1].str.extract(r'^\s*\d{1,2}/(\d{1,2})/\d{2,4}\s*$')[0].astype(int)
        mes.loc[meses.index] = meses.values

    # YYYY-MM-DD -> captura o segundo componente após o primeiro '-'
    m2 = s0.str.match(r'^\s*(\d{4})-(\d{2})-(\d{2})\s*$', na=False)
    if m2.any():
        meses2 = s0[m2].str.extract(r'^\s*\d{4}-(\d{2})-\d{2}\s*$')[0].astype(int)
        mes.loc[meses2.index] = meses2.values

    # resultado em Int64 (suporta NA)
    return mes.astype('Int64')

data_07_26['mes_extr'] = extract_month_series(s)

```

Conferência e pré-processamento de dados

Após a unificação em um único data frame, obtivemos o objeto `data_07_26` com 2.218.342 linhas, que representam todos os sinistros ocorridos na rodovias federais de 2006 a 2026 com 31 variáveis, conforme foram apresentados na seção **"Atributos do dataset"**. As variáveis 'delegacia', 'latitude', 'longitude', 'regional', 'uop' foram coletadas apenas a partir de 2017. Antes disso, as variáveis contêm "NaN", indicando missing em tais respostas.

Em seguida, na etapa de pré-processamento, serão realizadas operações para correção de algumas inconsistências detectadas nas variáveis. Algumas variáveis relevantes para a pesquisa precisam receber um tratamento devido o preenchimento equivocado. Por exemplo, "fase_dia" tem duas categorias iguais, mas que o software considera diferente, como "Plena noite" e "Plena Noite". Para tanto, diversas correções serão necessárias, que serão realizadas nos códigos a seguir. O mesmo ocorre com tipo_pista e classificacao_acidente, que têm como respostas "Ignorado" e "(null)", que serão corrigidos como missing, para que não sejam contadas como categorias das variáveis. Além disso, a variável target para o estudo foi declarada como 'obito', que é uma variável dicotômica que indica se o sinistro apresentou óbito, ou seja, se 'mortos' > 0, então 'obito' == 'Sim', caso contrário, 'Não'. Essa variável foi acrescentada ao data_07_26 e mantida durante todo estudo, de modo que essa é considerada a variável resposta (target) do estudo, usada para fins de classificação.

Após a correção das inconsistências e a seleção das variáveis consideradas mais relevantes para a pesquisa, tivemos um arquivo com mais de 207Mb com 16 variáveis e com as mesmas 2.218.342 linhas referentes a cada sinistro. Esses dados serão usados para identificação de possíveis fatores de risco para ocorrência de óbito. Lembrando que o ano de 2026 está incompleto e apresenta respostas apenas até o mês de abril.

DIA DA SEMANA CORRETO

```
data_07_26['dia_semana'].unique()
data_07_26['dia_semana_correto'] = data_07_26['dia_semana'].map({'segunda-feira':
'Segunda', 'terça-feira': 'Terça', 'quarta-feira': 'Quarta', 'quinta-feira': 'Quinta', 'sexta-
feira': 'Sexta', 'sábado': 'Sábado', 'domingo': 'Domingo' })
print(data_07_26['dia_semana_correto'].unique()) # dia_semana_correto
```

27 UFs CORRETAS

```
data_07_26['uf'] = data_07_26['uf'].replace({'(null)': pd.NA})
```

CORREÇÃO FASE DO DIA

```
data_07_26['fase_dia'].unique()
data_07_26['fase_dia'].dropna().unique()
data_07_26['fase_dia_correto'] = data_07_26['fase_dia'].replace({'Plena Noite': 'Plena
noite', '(null)': pd.NA})
```

CORREÇÃO USO_SOLO

```
data_07_26['uso_solo_correto'] = data_07_26['uso_solo'].replace({
'Sim': 'Urbano', 'Não': 'Rural', '(null)': pd.NA })
```

CORREÇÃO TIPO DE PISTA

```
#print(data_07_26['tipo_pista'].unique())
data_07_26['tipo_pista_correto'] = data_07_26['tipo_pista'].replace({'(null)': pd.NA})
```

CORREÇÃO CLASSIFICAÇÃO DO ACIDENTE

```
data_07_26['classificacao_acidente'].unique()
data_07_26['classificacao_acidente_correto'] =
data_07_26['classificacao_acidente'].replace({'Ignorado': pd.NA, '(null)': pd.NA})
data_07_26['classificacao_acidente'].value_counts(dropna=False)
```

VISIBILIDADE (NOVA VARIÁVEL EM FUNÇÃO DA CONDIÇÃO METEOROLÓGICA)

```

data_07_26['condicao_metereologica'].unique()
norm = lambda s: " if pd.isna(s) else ".join(c for c in unicodedata.normalize('NFD',
str(s).strip().lower().replace('/', ' ')) if unicodedata.category(c) != 'Mn')
data_07_26['visibilidade'] = data_07_26['condicao_metereologica'].map(norm).replace({
'nevoeiro neblina':'Parcial', 'ceu claro':'Boa', 'nublado':'Boa', 'sol':'Boa',
'vento':'Boa',chuva':'Ruim',granizo':'Ruim',neve':'Ruim',garoa chuvisco':'Ruim',
'ignorada': pd.NA, '(null)': pd.NA, 'ignorado': pd.NA, "": pd.NA
})

### CORREÇÃO TIPO DE ACIDENTE
data_07_26['tipo_acidente_correto'] =
data_07_26['tipo_acidente'].replace({'Atropelamento de pessoa': 'Atropelamento de
Pedestre',
'Danos Eventuais': 'Danos eventuais',
'Queda de ocupante de veículo': 'Queda de motocicleta / bicicleta / veículo'})

### ÓBITO Presença (1)/ausência (0) de pelo menos um óbito no sinistro
data_07_26['obito'] = np.where(data_07_26['mortos'] > 0, 'Sim', 'Não')

## Renomeando o "ano"
data_07_26 = data_07_26.rename(columns={'__year': 'ano', 'mes_extr': 'mes'})

### BANCO DE DADOS FINAL COM AS VARIÁVEIS MAIS RELEVANTES
cols = ['ano','mes','uf','dia_semana_correto','fase_dia_correto','tipo_pista_correto',
'classificacao_acidente_correto','tipo_acidente_correto','visibilidade',
'uso_solo_correto',
'pessoas','obito','feridos','mortos','latitude','longitude']
## Banco de dados finalizado
df07_26 = data_07_26[cols].copy()

# salva sem o índice, com separador vírgula e encoding utf-8
df07_26.to_csv("df07_26.csv", index=False, encoding="utf-8")

print(df07_26.columns) # Variáveis finais
print(df07_26.shape)

# DOWNLOAD para inserção no github
# from google.colab import files
# files.download('df07_26.csv')

```

Análise de Dados

Na Análise de Dados Exploratória (EDA) sobre o datatran de 2007 a 2026, temos o interesse entender a distribuição, as relações e as características das variáveis, o que é crucial para o panorama do banco de dados.

Destaca-se que na etapa de pré-processamento de dados, foram selecionadas as variáveis consideradas mais relevantes para a pesquisa. Além disso, foram excluídas redundâncias semânticas e foram realizadas categorizações com o fim de trazer mais clareza e síntese à pesquisa.

Nas figuras a seguir teremos os gráficos mais simples para retratar as variáveis de forma mais simples. O primeiro mostra a quantidade total de óbitos por ano no país, onde observamos um queda de 2012 para 2018, e a partir daí, temos um leve crescimento até 2024, seguido por uma redução em 2025. Destaca-se que 2026 apresenta o menor valor pois tem suas medidas apenas até abril.

```
# soma de óbitos por ano (assume coluna 'mortos' e 'ano' já corretas)
# print(data_07_26.groupby('ano', dropna=True)['mortos'].sum().sort_index())

# usando mes_extr (Int64) e ano (Int64)
# ct = pd.crosstab(data_07_26['ano'], data_07_26['mes'], dropna=True)
# exibir tabela (anos como linhas, meses 1..12 como colunas)
# print(ct)

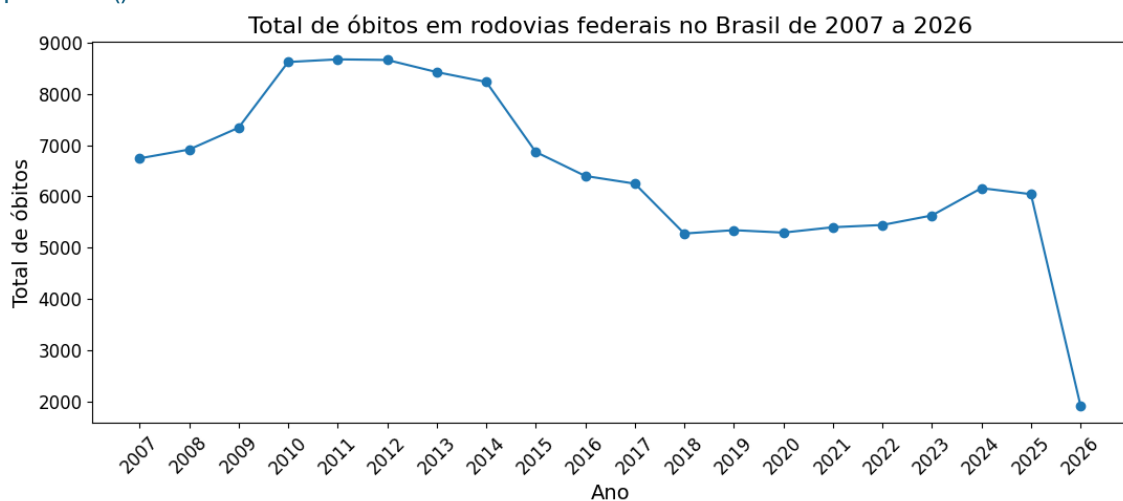
## GRÁFICOS SIMPLES

## Quantidade de óbitos por ano
plt.rcParams.update({'font.size': 12}) # opcional: define tamanho base global
fig, ax = plt.subplots(figsize=(11,5))

ax.plot(range(2007,2027),
        df07_26.groupby('ano')['mortos'].sum().reindex(range(2007,2027), fill_value=0).values,
        marker='o')
ax.set_xticks(list(range(2007,2027)))
ax.set_xlabel('Ano', fontsize=14)
ax.set_ylabel('Total de óbitos', fontsize=14)
ax.set_title('Total de óbitos em rodovias federais no Brasil de 2007 a 2026', fontsize=16)

# aumentar tamanho das labels dos ticks
ax.tick_params(axis='x', labelsize=12, rotation=45)
ax.tick_params(axis='y', labelsize=12)

plt.tight_layout()
plt.show()
```

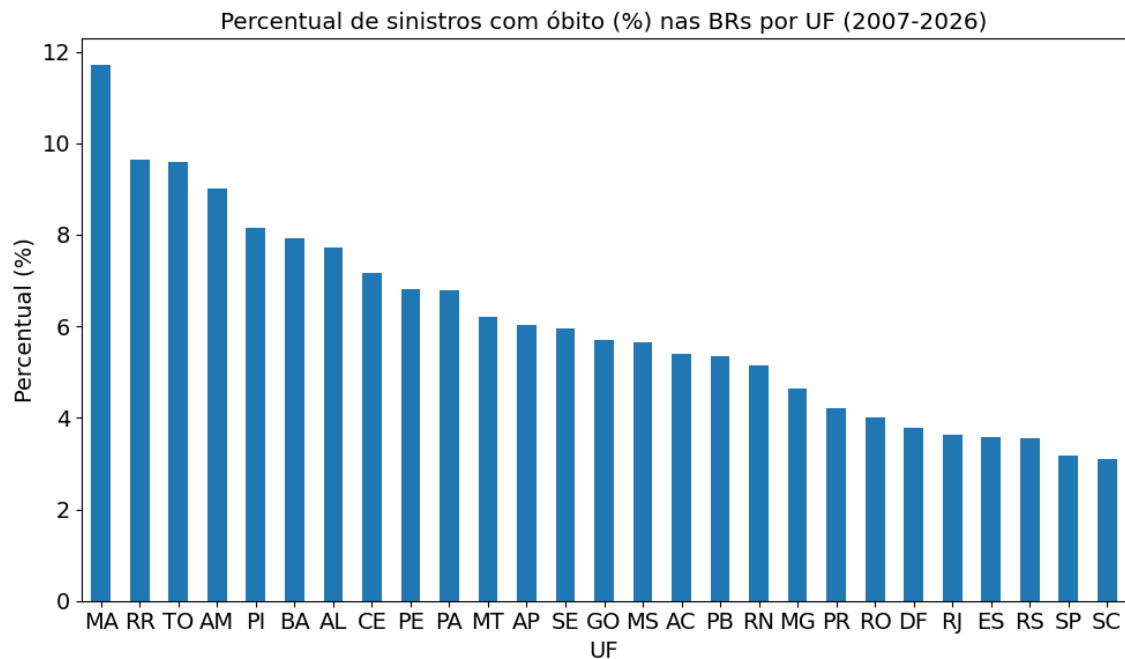


```
ct1 = pd.crosstab(df07_26['uf'], df07_26['obito'], normalize='index').round(4)*100
```

```
# Gráfico para a proporção
```

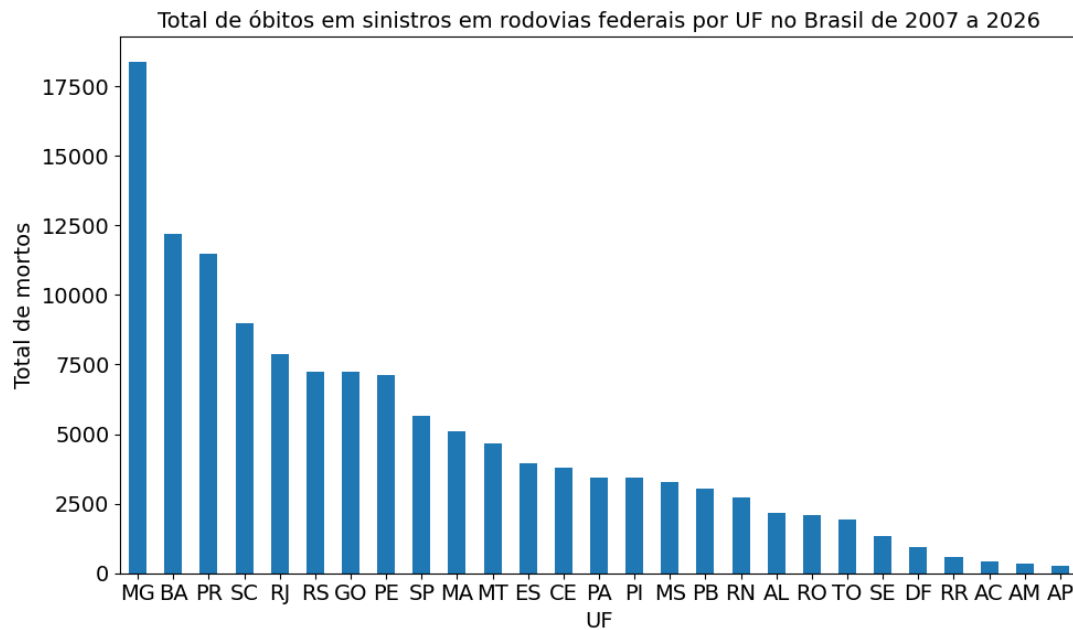
```
col = 'Sim' if 'Sim' in ct1.columns else ct1.columns[-1]
ct_sorted1 = ct1.sort_values(by=col, ascending=False)
```

```
ax = ct_sorted1[col].plot(kind='bar', figsize=(10,6), color='C0')
ax.set_xlabel('UF',fontsize=14)
ax.set_ylabel(f'Percentual (%)',fontsize=14)
ax.set_title(f'Percentual de sinistros com óbito (%) nas BRs por UF (2007-2026)')
# aumentar tamanho das labels dos ticks
ax.tick_params(axis='x', labelsize=14, rotation=0)
ax.tick_params(axis='y', labelsize=14)
plt.tight_layout()
plt.show()
```



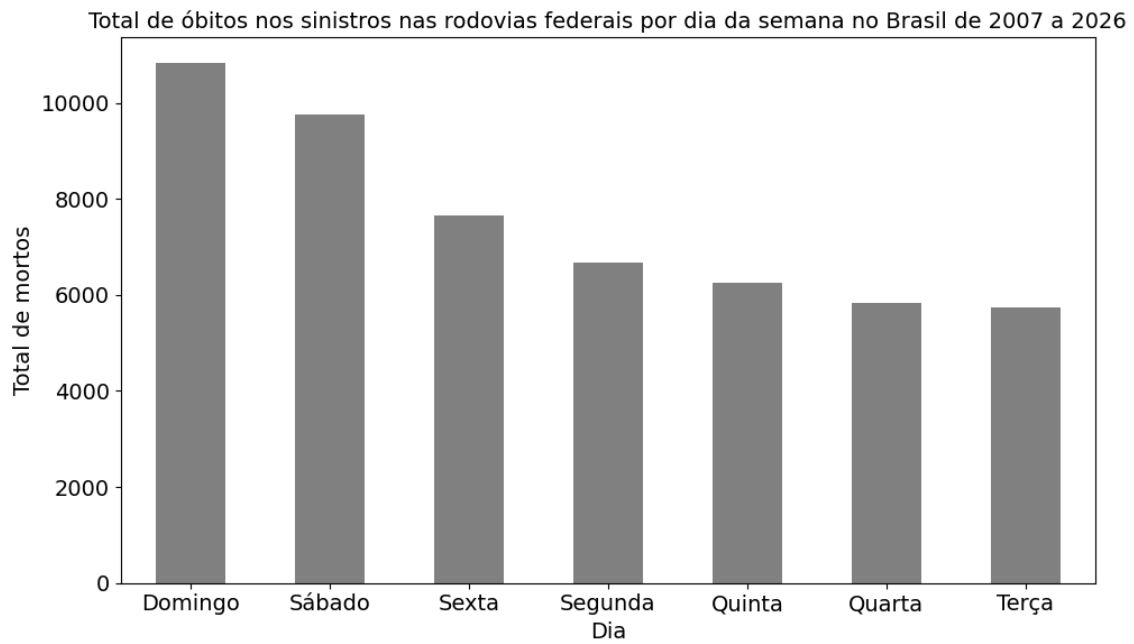
```
## Soma de óbitos por UF
fig, ax = plt.subplots(figsize=(10,6))
(df07_26.groupby('uf', sort=False)['mortos']
 .sum()
 .sort_values(ascending=False)
 .plot(kind='bar', ax=ax))
ax.set_xlabel('UF',fontsize=14); ax.set_ylabel('Total de mortos',fontsize=14);
ax.set_title('Total de óbitos em sinistros em rodovias federais por UF no Brasil de 2007 a 2026',fontsize=14)

# aumentar tamanho das labels dos ticks
ax.tick_params(axis='x', labelsize=14, rotation=0)
ax.tick_params(axis='y', labelsize=14)
plt.tight_layout()
plt.show()
```



```
ct2 = pd.crosstab(df07_26['dia_semana_correto'], df07_26['obito'],
normalize='index').round(4)*100
print(ct2.sort_values(by=('Sim' if 'Sim' in ct2.columns else ct2.columns[-1]),
ascending=False))

## POR DIA DA SEMANA
fig, ax = plt.subplots(figsize=(10,6))
(df07_26.groupby('dia_semana_correto', sort=False)['mortos']
.sum()
.sort_values(ascending=False)
.plot(kind='bar', ax=ax, color='gray'))
ax.set_xlabel('Dia',fontsize=14); ax.set_ylabel('Total de mortos',fontsize=14);
ax.set_title('Total de óbitos nos sinistros nas rodovias federais por dia da semana no Brasil
de 2007 a 2026',fontsize=14)
# aumentar tamanho das labels dos ticks
ax.tick_params(axis='x', labelsize=14, rotation=0)
ax.tick_params(axis='y', labelsize=14)
plt.tight_layout()
plt.show()
```



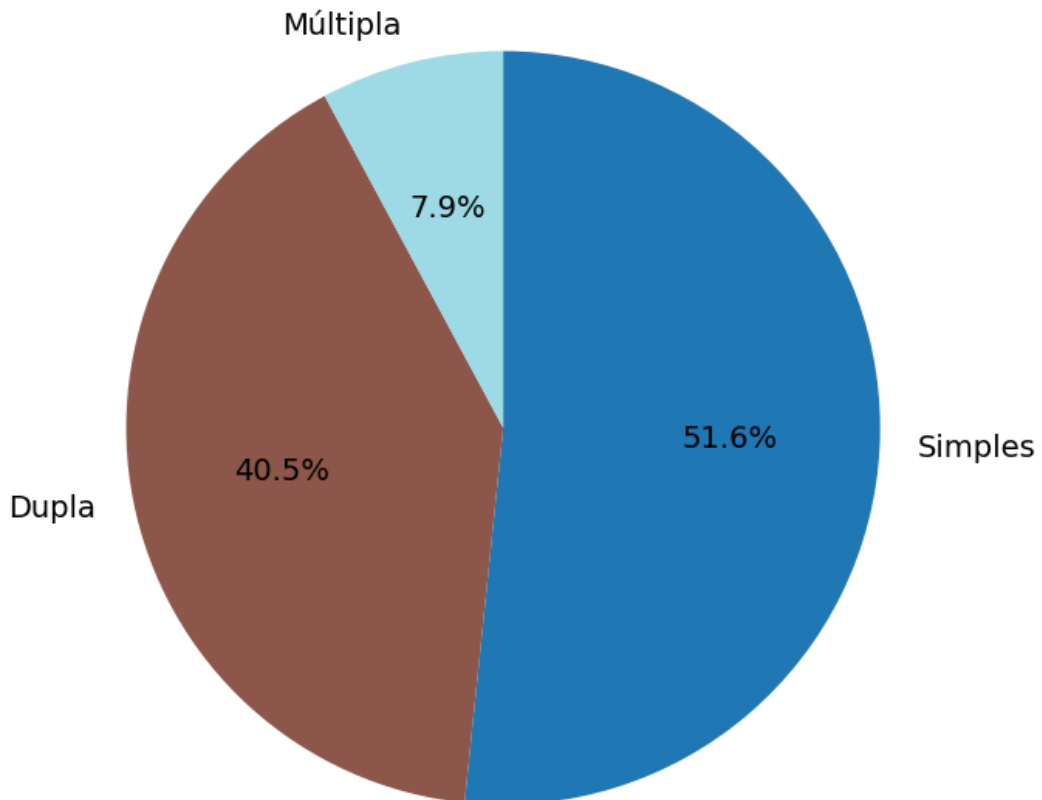
```
plt.rcParams.update({'font.size': 12}) # base global
```

```
fig, ax = plt.subplots(figsize=(7,7))
data = df07_26['tipo_pista_correto'].dropna().value_counts(ascending=False)
data.plot.pie(
    ax=ax,
    autopct='%1.1f%%',
    startangle=90,
    cmap='tab20',
    counterclock=False,
    textprops={'fontsize': 14} # aumenta os percentuais e labels exibidos no gráfico
)
```

```
ax.set_ylabel("")
ax.set_title('Proporção de sinistros por tipo de pista em BRs (2007-2026)', fontsize=15,
weight='bold')
# se quiser legendar ao invés de mostrar labels nos wedges:
# ax.legend(labels=data.index, loc='center left', bbox_to_anchor=(1, 0.5), fontsize=14)

# ajustar ticks/labels do eixo (se houver)
ax.tick_params(axis='x', labelsize=14)
ax.axis('equal')
plt.tight_layout()
plt.show()
```

Proporção de sinistros por tipo de pista em BRs (2007-2026)



```
## DIAGRAMA DE PARETO PARA OS TIPOS DE ACIDENTES
df = (df07_26.groupby('tipo_acidente_correto', dropna=False)['mortos']
      .sum().reset_index().sort_values('mortos', ascending=False).head(11))
df['cum_pct'] = df['mortos'].cumsum() / df['mortos'].sum() * 100

import matplotlib.pyplot as plt

plt.rcParams.update({'font.size': 12})

x = range(len(df))
fig, ax = plt.subplots(figsize=(12,6))

ax.bar(x, df['mortos'], color='C0')
ax.set_xticks(x)
ax.set_xticklabels(df['tipo_acidente_correto'], rotation=45, ha='right', fontsize=12)
ax.set_ylabel('Mortos', fontsize=14)
ax.tick_params(axis='y', labelsize=12)

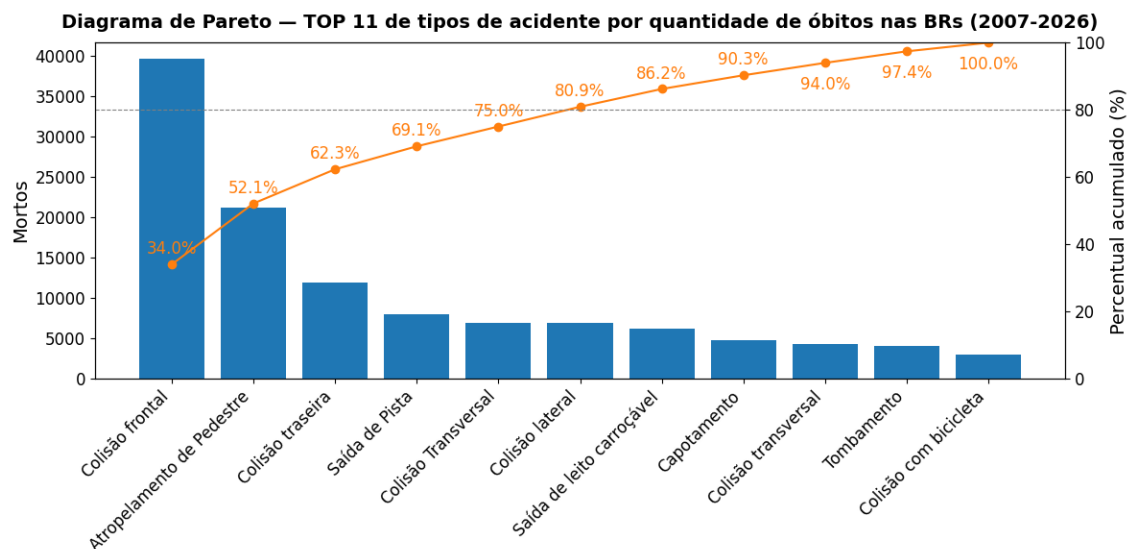
ax2 = ax.twinx()
ax2.plot(x, df['cum_pct'], color='C1', marker='o', ms=6)
ax2.axhline(80, color='gray', linestyle='--', linewidth=0.8)
ax2.set_ylim(0,100)
ax2.set_ylabel('Percentual acumulado (%)', fontsize=14)
ax2.tick_params(axis='y', labelsize=12)
```

```

# ajustar posição dos rótulos percentuais para evitar sobreposição com título
for xi, v in zip(x, df['cum_pct']):
    if v > 92:
        va = 'top'
        y = v - 4 # coloca abaixo do ponto para valores muito altos
    else:
        va = 'bottom'
        y = v + 2 # coloca acima do ponto
    ax2.text(xi, y, f'{v:.1f}%', ha='center', va=va, color='C1', fontsize=12, clip_on=True)

# Define título e margens (aumenta top para não sobrepor)
plt.title('Diagrama de Pareto — TOP 11 de tipos de acidente por quantidade de óbitos nas BRs (2007-2026)',
        fontsize=14, weight='bold', y=1.02)
plt.subplots_adjust(bottom=0.30, top=0.88) # reduz risco de sobreposição
plt.tight_layout()
plt.show()

```



```
vars = ['ano','mes','uf','dia_semana_correto','fase_dia_correto','visibilidade','obito']
```

```

# cálculo da matriz de p-valores (igual ao seu)
pmat = pd.DataFrame(np.nan, index=vars, columns=vars)
for i, a in enumerate(vars):
    for j, b in enumerate(vars):
        if i>=j:
            continue
        sub = df07_26[[a,b]].dropna()
        if sub.empty:
            p = np.nan
        else:
            tbl = pd.crosstab(sub[a], sub[b])
            if tbl.size==0 or tbl.shape[0]<2 or tbl.shape[1]<2:
                p = np.nan
            else:
                _, p, _, _ = chi2_contingency(tbl, correction=False)

```

```

    pmat.at[a,b] = p
    pmat.at[b,a] = p

# Bonferroni
n_tests = np.triu(~pmat.isna(), 1).sum().sum()
alpha = 0.05
alpha_bonf = alpha / max(1, n_tests)

# rótulos legíveis
label_map = {
    'ano': 'Ano',
    'mes': 'Mês',
    'uf': 'UF',
    'dia_semana_correto': 'Dia da semana',
    'fase_dia_correto': 'Fase do dia',
    'visibilidade': 'Visibilidade',
    'obito': 'Óbito'
}
labels = [label_map[v] for v in vars]

# preparar anotação com "< 0,001"
annot = pmat.copy().astype(object)
for i in annot.index:
    for j in annot.columns:
        v = annot.at[i,j]
        if pd.isna(v):
            annot.at[i,j] = '-'
        else:
            if v < 0.001:
                annot.at[i,j] = "< 0,001"
            else:
                annot.at[i,j] = f"{v:.3f}"

mask = np.eye(len(vars), dtype=bool)

plt.rcParams.update({'font.size': 12}) # base global

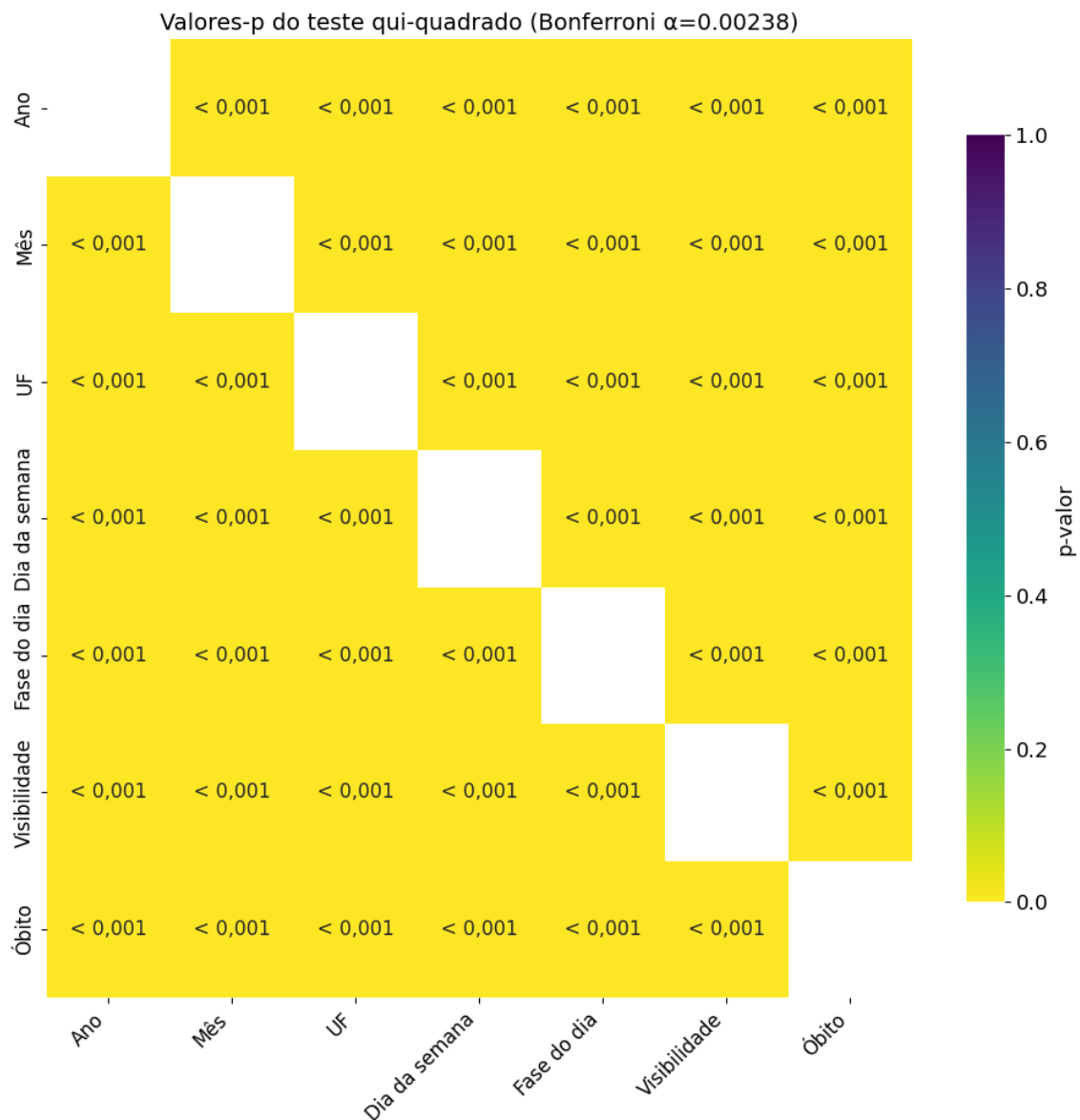
fig, ax = plt.subplots(figsize=(10,10))
sns.heatmap(pmat.astype(float), annot=annot, fmt="", cmap='viridis_r',
            cbar_kws={'label': 'p-valor', 'shrink': 0.8},
            vmin=0, vmax=1, mask=mask,
            xticklabels=labels, yticklabels=labels,
            annot_kws={'fontsize': 12}, ax=ax)

# ticks fontsize
ax.set_xticklabels(labels, rotation=45, ha='right', fontsize=12)
ax.set_yticklabels(labels, fontsize=12)

# cbar label fontsize
cbar = ax.collections[0].colorbar
cbar.set_label('p-valor', fontsize=13)
cbar.ax.tick_params(labelsize=13)

```

```
plt.title(f'Valores-p do teste qui-quadrado (Bonferroni  $\alpha$ = $\alpha_{\text{bonf}}$ :.3g)', fontsize=14)
plt.tight_layout()
plt.show()
```



Mapa dos sinistros no país de 2007 a 2026

Para finalizar, vemos o mapa. O primeiro apresenta apenas os sinistros, independente se tivesse ocorrido ou não. Já o segundo, foi realizada uma discriminação por cor, aonde fica visível a predominância de óbitos no norte nordeste em relação ao sul. Tudo isso se deve ao fato de que as condições estruturais nas regiões norte e nordeste ainda serem precárias e isso se reflete na ocorrência de acidentes fatais.

```
df = df07_26.dropna(subset=['latitude','longitude']).copy()
```

```
gdf_pts = gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df['longitude'],
df['latitude']), crs="EPSG:4326")
```

```
countries_url = "https://raw.githubusercontent.com/datasets/geo-
countries/master/data/countries.geojson"
```

```

countries = gpd.read_file(countries_url)
if 'ISO_A3' in countries.columns:
    brazil = countries[countries['ISO_A3'] == 'BRA']
elif 'ADMIN' in countries.columns:
    brazil = countries[countries['ADMIN'].str.contains('Brazil', case=False, na=False)]
elif 'NAME' in countries.columns:
    brazil = countries[countries['NAME'].str.contains('Brazil', case=False, na=False)]
else:
    mask = countries.apply(lambda row: row.astype(str).str.contains('Brazil', case=False,
na=False).any(), axis=1)
    brazil = countries[mask]

brazil = brazil.to_crs(epsg=3857)
gdf_pts = gdf_pts.to_crs(epsg=3857)

brazil_union = brazil.geometry.union_all()
gdf_pts_bra = gdf_pts[gdf_pts.geometry.intersects(brazil_union)].copy()

gdf_pts_bra['obito'] = gdf_pts_bra['obito'].astype(str).str.strip().str.capitalize() # fica 'Sim'
ou 'Não' se for esse padrão

pts_obito = gdf_pts_bra[gdf_pts_bra['obito'] == 'Sim']
pts_no_obito = gdf_pts_bra[gdf_pts_bra['obito'] == 'Não']

fig, ax = plt.subplots(figsize=(10,10))
brazil.plot(ax=ax, color="#f7f7f7", edgecolor="#444444", linewidth=0.5)

if len(pts_no_obito):
    pts_no_obito.plot(ax=ax, markersize=6, color="#1f77b4", alpha=0.7, marker='o',
label='Sem óbito')
if len(pts_obito):
    pts_obito.plot(ax=ax, markersize=6, color="#d62728", alpha=0.1, marker='o', label='Com
óbito')

import contextily as ctx
ctx.add_basemap(ax, source=ctx.providers.CartoDB.PositronNoLabels) # Sem nomes
#ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)

minx, miny, maxx, maxy = brazil.total_bounds
dx = (maxx - minx) * 0.03
dy = (maxy - miny) * 0.03
ax.set_xlim(minx - dx, maxx + dx)
ax.set_ylim(miny - dy, maxy + dy)

ax.set_title("Sinistros por ocorrência de óbito nas BRs do Brasil (2007 - 2026)")
ax.set_axis_off()
ax.legend(loc='lower right')

plt.tight_layout()
plt.show()

```

Sinistros por ocorrência de óbito nas BRs do Brasil (2007 - 2026)



Aprendizado não supervisionado

De posse dos resultados observados, inclusive pelo teste qui-quadrado, que detectou associação entre as principais variáveis do estudo, apesar de não ser o principal objetivo do estudo, aplicaremos metodologias de aprendizado não supervisionado entre as variáveis categóricas mencionadas. A primeira metodologia aplicada foi a Análise de Correspondência Multivariada (ACM), que nos permite detalhar melhor a relação entre variáveis e suas respectivas categorias. Para tanto, foram usadas as variáveis 'ano', 'mes', 'uf', 'dia_semana_correto', 'fase_dia_correto', 'tipo_pista_correto', 'visibilidade', 'uso_solo_correto' e 'obito'.

De acordo com a apresentação do biplot, observamos o que já havia sido constatado, que foi a forte associação entre tipo de pista simples, ocorrência de óbito, visibilidade parcial e com as regiões norte e nordeste. Essas convergências, apesar de serem repetitivas, são importantes para corroborar com a inserção de tais variáveis nos modelos supervisionados. Infelizmente, pela nuvem de pontos, não possível identificar nenhum agrupamento específico pela ACM.

Análise de Correspondência Multivariada

```
from sklearn.manifold import MDS
import matplotlib.pyplot as plt

import prince
import pandas as pd

# Selecionar colunas
cols = ['ano','mes','uf','dia_semana_correto','fase_dia_correto',
        'tipo_pista_correto',
        'visibilidade','uso_solo_correto','obito']

# preparar df_mca como antes (categorical, sem NA)
df_mca = df07_26[cols].dropna().copy()
for c in df_mca.columns:
    df_mca[c] = df_mca[c].astype('category')

mca = prince.MCA(
    n_components=5,
    n_iter=100,
    copy=True,
    check_input=True,
    engine='sklearn', # use 'sklearn', 'scipy' ou 'fbpca'
    random_state=42
)

mca = mca.fit(df_mca)
row_coords = mca.transform(df_mca)
col_coords = mca.column_coordinates(df_mca)

print(mca.eigenvalues_)
# Calculando a inércia explicada manualmente, pois explained_inertia_ está gerando erro
print(mca.eigenvalues_ / mca.eigenvalues_.sum())

column_labels = col_coords.index.astype(str).tolist()

for i, txt in enumerate(column_labels):
    x = col_coords.iloc[i, 0] # componente 1
    y = col_coords.iloc[i, 1] # componente 2
    ax.annotate(txt,
                xy=(x, y),
                xytext=(x + 0.03, y + 0.03),
                fontsize=10,
                bbox=dict(boxstyle='round,pad=0.2', fc='white', alpha=0.6),
                arrowprops=dict(arrowstyle='-', alpha=0.3))

# maior figura
fig, ax = plt.subplots(figsize=(10,10))

# Aumentar tamanho dos pontos de observações
```

```

ax.scatter(row_coords[0], row_coords[1], s=40, alpha=0.3, color='gray',
label='Observações', zorder=1)

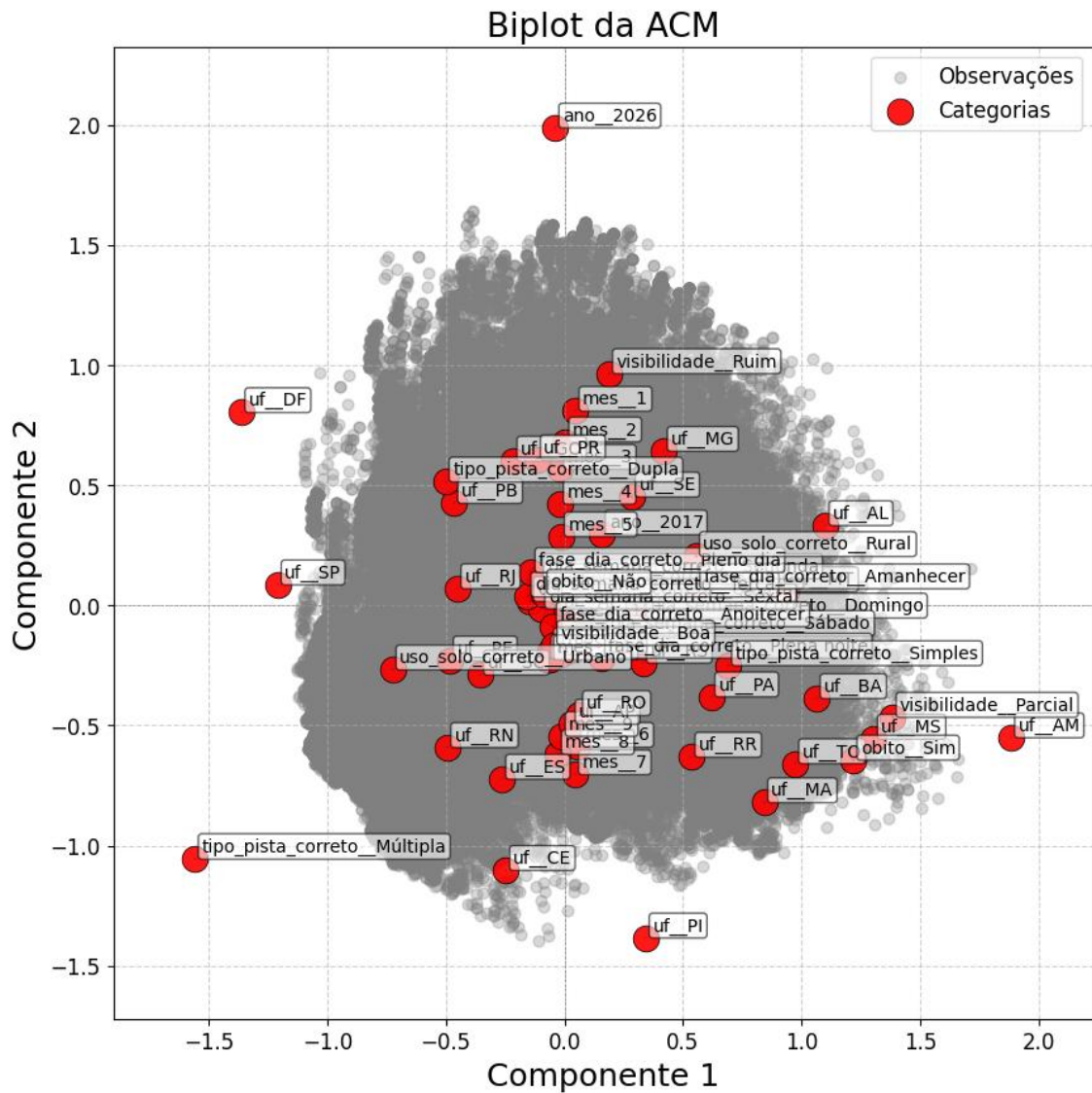
# Aumentar e destacar pontos de categorias
ax.scatter(col_coords[0], col_coords[1], s=220, color='red', alpha=0.9,
edgecolors='k', linewidths=0.6, label='Categorias', zorder=2)

# Anotações com deslocamento e bbox para legibilidade
for i, txt in enumerate(column_labels):
    ax.annotate(txt,
        xy=(col_coords[0].iloc[i], col_coords[1].iloc[i]),
        xytext=(col_coords[0].iloc[i] + 0.03, col_coords[1].iloc[i] + 0.03),
        fontsize=10,
        bbox=dict(boxstyle='round,pad=0.2', fc='white', alpha=0.6),
        arrowprops=dict(arrowstyle='-', alpha=0.3))

# Aumentar limites/margem se pontos muito colados ao eixo
xpad = (col_coords[0].max() - col_coords[0].min()) * 0.10
ypad = (col_coords[1].max() - col_coords[1].min()) * 0.10
ax.set_xlim(col_coords[0].min()-xpad, col_coords[0].max()+xpad)
ax.set_ylim(col_coords[1].min()-ypad, col_coords[1].max()+ypad)

# Restante do layout
plt.title('Biplot de MCA (Componentes 1 e 2)', fontsize=19)
plt.xlabel('Componente 1', fontsize=18)
plt.ylabel('Componente 2', fontsize=18)
plt.grid(True, linestyle='--', alpha=0.6)
ax.axhline(0, color='gray', linestyle='--', linewidth=0.5)
ax.axvline(0, color='gray', linestyle='--', linewidth=0.5)
ax.legend()

```



Os próximos métodos de aprendizado não supervisionado são o PAM (Partitioning Around Medoids) com a métrica de Gower e o K-Modes. Usando o método PAM com o método silhueta para selecionar o número de grupos (entre 2 e 10 grupos), obtivemos 2 grupos como o melhor agrupamento neste método não hierárquico de agrupamento, como mostra o primeiro gráfico. O segundo gráfico exibe o agrupamento, aonde fica evidente que há dois grupos bem definidos, apesar de ter uma interseção entre eles. Para tanto, usamos o método de Análise de Cesta de Compras (Market Basket Analysis), que serve para identificar produtos ou serviços costumam ser comprados juntos.

Usamos o método para os dois grupos, que forneceu o seguinte resultado: A não ocorrência de óbito está associada ao pleno dia na zona rural no cluster 1, já no cluster 1 tem muitas regras óbito: Sim aparece mais nos resultados brutos para plena noite, pista simples e região nordeste.

Destaca-se que, pela alta demanda computacional, os resultados foram disponibilizados sem a necessidade de execução do código, pois a demanda é em torno de 20 minuto para alguns blocos de execução. Essa estratégia se fez necessária para uma avaliação objetiva sem levar prejuízo de tempo ao docente.

K-Medoids (k=2) e Silhueta (k = 2, 3, ..., 10)

```
!pip install pyclustering
import random
from pyclustering.cluster.kmedoids import kmedoids
from sklearn.manifold import MDS
from sklearn.metrics import silhouette_score
# Preparando os dados
cols = ['ano','mes','uf','dia_semana_correto','fase_dia_correto',
        'tipo_pista_correto','visibilidade','uso_solo_correto','obito']
df_pam = df07_26[cols].dropna().copy()
for c in df_pam.columns:
    df_pam[c] = df_pam[c].astype('category')

# função para matriz de distância de Gower
def gower_distance_matrix(df):
    n = df.shape[0]
    D = np.zeros((n, n), dtype=float)
    for i in range(n):
        xi = df.iloc[i]
        for j in range(i+1, n):
            xj = df.iloc[j]
            dist = 0.0
            valid = 0
            for col in df.columns:
                vi, vj = xi[col], xj[col]
                if pd.api.types.is_categorical_dtype(df[col]) or df[col].dtype == object:
                    d = 0.0 if vi == vj else 1.0
                else:
                    col_min = df[col].min()
                    col_max = df[col].max()
                    rng = col_max - col_min
                    d = 0.0 if rng == 0 else abs(vi - vj) / rng
                dist += d
                valid += 1
            D[i, j] = D[j, i] = dist / valid if valid>0 else 0.0
    return D

# calcular matriz de Gower (pode demorar se df for muito grande)
n_rows = df_pam.shape[0]
if n_rows > 2000:
    # opcional: amostrar para acelerar
    sample_idx = np.random.choice(n_rows, 2000, replace=False)
    df_use = df_pam.iloc[sample_idx].reset_index(drop=True)
else:
    df_use = df_pam.reset_index(drop=True)

D = gower_distance_matrix(df_use)
# função auxiliar para rodar kmedoids do pyclustering com matriz de distância
def run_kmedoids_precomputed(D, k, rng_seed=42):
    n = D.shape[0]
    random.seed(rng_seed + k)
```

```

initial_medoids = random.sample(range(n), k)
km = kmedoids(D.tolist(), initial_medoids, data_type='distance_matrix')
km.process()
clusters = km.get_clusters()
labels = np.empty(n, dtype=int)
for cidx, cluster in enumerate(clusters):
    labels[cluster] = cidx
return labels, km.get_medoids()

# testar KMedoids (PAM-like) para k=2..10 com distância pré-computada
ks = list(range(2, 11))
sil_scores = []
models = {}
for k in ks:
    labels, medoids = run_kmedoids_precomputed(D, k, rng_seed=42)
    score = silhouette_score(D, labels, metric='precomputed')
    sil_scores.append(score)
    models[k] = (labels, medoids)

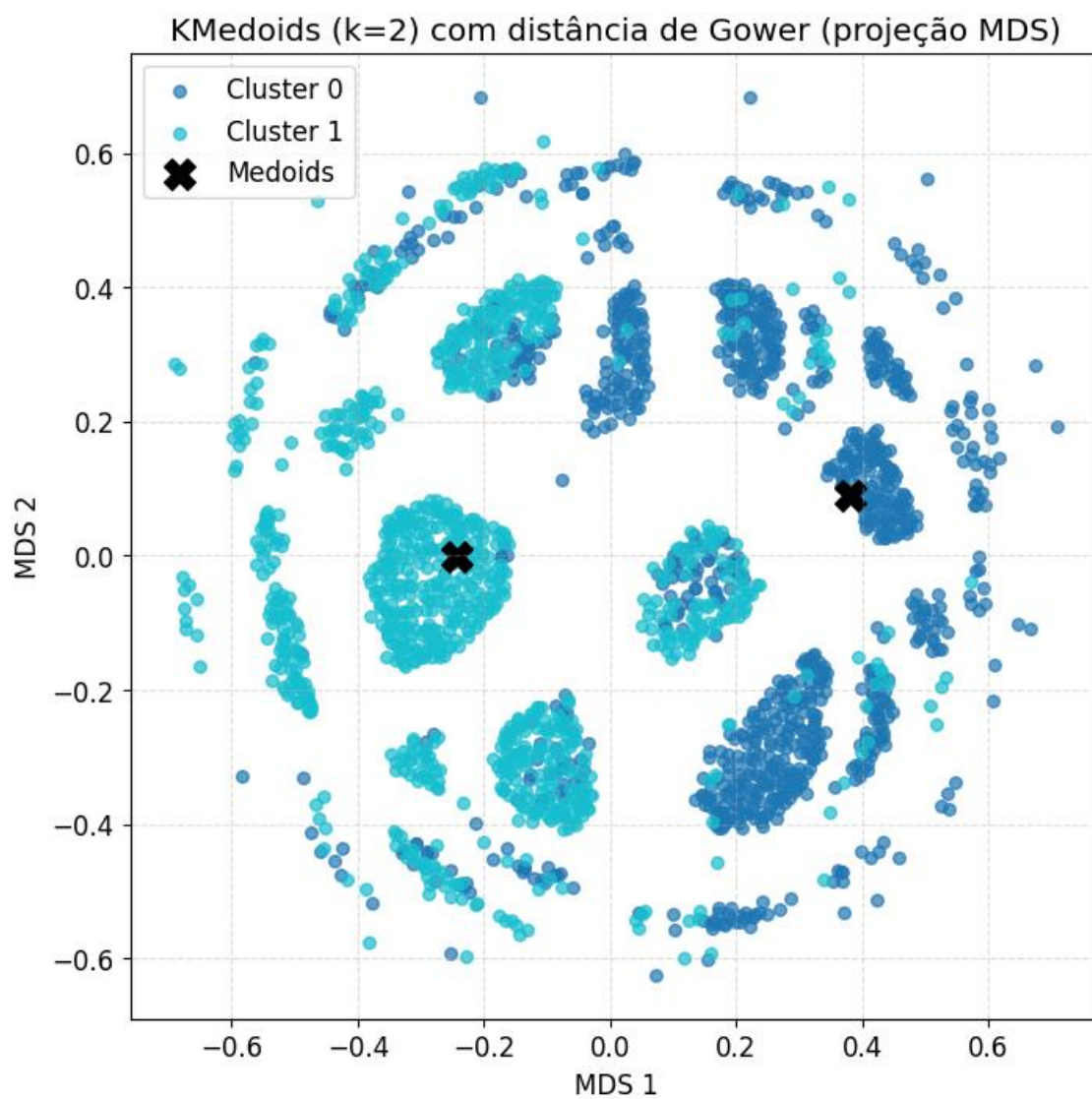
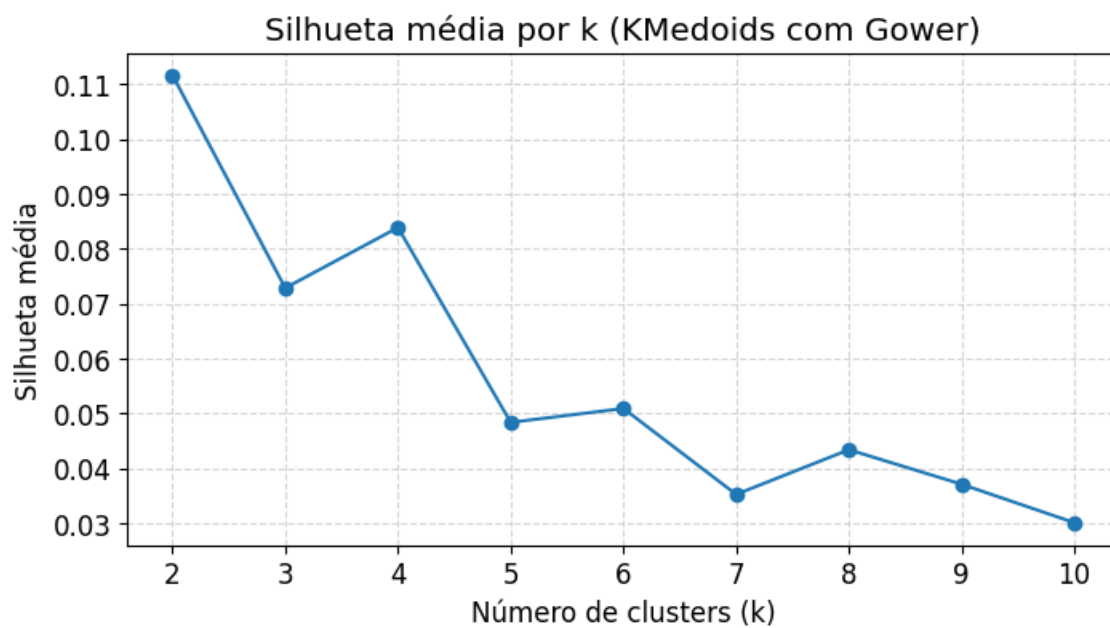
# plot silhueta por k
plt.figure(figsize=(8,4))
plt.plot(ks, sil_scores, marker='o')
plt.xticks(ks)
plt.xlabel('Número de clusters (k)')
plt.ylabel('Silhueta média')
plt.title('Silhueta média por k (KMedoids com Gower)')
plt.grid(True, ls='--', alpha=0.5)
plt.show()

# melhor k e plot MDS 2D para visualização
best_k = ks[int(np.argmax(sil_scores))]
best_labels, best_medoids = models[best_k]
print(f"Melhor k por silhueta: {best_k} (silhueta = {max(sil_scores):.4f})")

mds = MDS(n_components=2, dissimilarity='precomputed', random_state=42)
X2 = mds.fit_transform(D)

plt.figure(figsize=(8,8))
palette = plt.cm.get_cmap('tab10', best_k)
for cluster in range(best_k):
    idx = best_labels == cluster
    plt.scatter(X2[idx,0], X2[idx,1], s=30, alpha=0.7, color=palette(cluster), label=f'Cluster {cluster}')
# Marcando medoids
plt.scatter(X2[best_medoids,0], X2[best_medoids,1], s=200, c='black', marker='X',
label='Medoids')
plt.xlabel('MDS 1')
plt.ylabel('MDS 2')
plt.title(f'KMedoids (k={best_k}) com distância de Gower (projeção MDS)')
plt.legend()
plt.grid(True, ls='--', alpha=0.4)
plt.show()

```



Análise de Cesta de Compras (Market Basket Analysis)

```
!pip install mlxtend
from mlxtend.frequent_patterns import apriori, association_rules

## Usando Análise de Cesta de Compras (Market Basket Analysis) de forma geral
for c in df_use.columns:
    df_use[c] = df_use[c].astype('category')

df_items = pd.get_dummies(df_use.apply(lambda col: col.astype(str)))

def mine_rules_for_cluster(cluster_label, df_items, labels, min_support=0.01,
min_threshold=0.6, top_n=20):
    # selecionar transações do cluster
    idx = (labels == cluster_label)
    X = df_items.loc[idx]
    if X.shape[0] == 0:
        print(f"Cluster {cluster_label}: vazio")
        return None
    # frequent itemsets
    freq = apriori(X, min_support=min_support, use_colnames=True)
    if freq.empty:
        print(f"Cluster {cluster_label}: nenhum item frequente com suporte >=
{min_support}")
        return None
    # gerar regras
    rules = association_rules(freq, metric="confidence", min_threshold=min_threshold)
    if rules.empty:
        print(f"Cluster {cluster_label}: nenhuma regra com confidence >= {min_threshold}")
        return None
    # ordenar por lift (ou confidence) e mostrar top N
    rules = rules.sort_values(by=['lift','confidence'],
ascending=False).reset_index(drop=True)
    # formatar Antecedents/Consequents legíveis
    def fmt_items(itemset):
        return ', '.join(sorted(list(itemset)))
    rules['antecedents_str'] = rules['antecedents'].apply(fmt_items)
    rules['consequents_str'] = rules['consequents'].apply(fmt_items)
    display_cols = ['antecedents_str','consequents_str','support','confidence','lift']
    print(f"Cluster {cluster_label}: n_transações = {X.shape[0]}, n_itemsets = {len(freq)},
n_regras = {len(rules)}")
    return rules[display_cols].head(top_n)

unique_clusters = sorted(pd.Series(best_labels).unique())
results = {}
for cl in unique_clusters:
    res = mine_rules_for_cluster(cl, df_items, best_labels, min_support=0.02,
min_threshold=0.6, top_n=30)
    results[cl] = res
    print(res)
    print("\n" + "-"*80 + "\n")
```

```
## K-Modes 1
```

```
## Usando Análise de Cesta de Compras (Market Basket Analysis) fixando óbito
```

```
from mlxtend.frequent_patterns import apriori, association_rules
```

```
import pandas as pd
```

```
min_support = 0.02
```

```
min_confidence = 0.5
```

```
top_n = 20
```

```
labels = pd.Series(best_labels, index=df_items.index)
```

```
def top_obito_rules_for_cluster(cluster_label):
```

```
    idx = labels == cluster_label
```

```
    X = df_items.loc[idx]
```

```
    if X.shape[0] == 0:
```

```
        print(f"Cluster {cluster_label}: vazio")
```

```
        return None
```

```
    freq = apriori(X, min_support=min_support, use_colnames=True)
```

```
    if freq.empty:
```

```
        print(f"Cluster {cluster_label}: nenhum itemset frequente
```

```
(min_support={min_support})")
```

```
        return None
```

```
    rules = association_rules(freq, metric="confidence", min_threshold=min_confidence)
```

```
    if rules.empty:
```

```
        print(f"Cluster {cluster_label}: nenhuma regra (min_confidence={min_confidence})")
```

```
        return None
```

```
    # transformar conjuntos em strings legíveis
```

```
    rules['antecedents_str'] = rules['antecedents'].apply(lambda s: ', '.join(sorted(list(s))))
```

```
    rules['consequents_str'] = rules['consequents'].apply(lambda s: ', '.join(sorted(list(s))))
```

```
    # filtrar regras cujo conseqüente contenha obito_Não ou obito_Sim
```

```
    mask_obito = rules['consequents_str'].str.contains('obito_Não') |
```

```
rules['consequents_str'].str.contains('obito_Sim')
```

```
    rules_obito = rules[mask_obito].copy()
```

```
    if rules_obito.empty:
```

```
        print(f"Cluster {cluster_label}: sem regras com 'obito' nos conseqüentes")
```

```
        return None
```

```
    # ordenar e retornar top_n por lift e por confidence
```

```
    top_by_lift = rules_obito.sort_values(by='lift', ascending=False).head(top_n)
```

```
    top_by_conf = rules_obito.sort_values(by='confidence', ascending=False).head(top_n)
```

```
    print(f"\n=== Cluster {cluster_label} — top {top_n} regras com 'obito' por LIFT ===")
```

```
    display_cols = ['antecedents_str', 'consequents_str', 'support', 'confidence', 'lift']
```

```
    print(top_by_lift[display_cols].to_string(index=False))
```

```
    print(f"\n=== Cluster {cluster_label} — top {top_n} regras com 'obito' por CONFIDENCE
```

```
===")
```

```
    print(top_by_conf[display_cols].to_string(index=False))
```

```
    return {'by_lift': top_by_lift[display_cols], 'by_confidence': top_by_conf[display_cols]}
```

```
    unique_clusters = sorted(labels.unique())
```

```
all_results = {}
```

```
for cl in unique_clusters:
```

```

res = top_obito_rules_for_cluster(cl)
all_results[cl] = res

## K-Modes 2
!pip install scikit-learn kmodes

from sklearn.metrics import pairwise_distances, silhouette_score
from kmodes.kmodes import KModes
import numpy as np
import matplotlib.pyplot as plt

# preparar dados categóricos (substitua df07_26/cols conforme necessário)
cols = ['ano','mes','uf','dia_semana_correto','fase_dia_correto',
        'tipo_pista_correto','visibilidade','uso_solo_correto','obito']
df_km = df07_26[cols].dropna().copy()
for c in df_km.columns:
    df_km[c] = df_km[c].astype('category')

# converter categorias para códigos inteiros (apenas para calcular distâncias; não é
transformação numérica com sentido ordinal)
X_codes = np.vstack([df_km[col].cat.codes.values for col in df_km.columns]).T

# matriz de dissimilaridade Hamming (proporção de atributos diferentes)
D = pairwise_distances(X_codes, metric='hamming') # valores entre 0 e 1

ks = list(range(2, 11))
sil_scores = []
models = {}
for k in ks:
    km = KModes(n_clusters=k, init='Huang', n_init=5, verbose=0, random_state=42)
    labels = km.fit_predict(df_km.values) # kmodes aceita array de objetos/categorias
    # silhueta usando matriz de distâncias pré-computada
    score = silhouette_score(D, labels, metric='precomputed')
    sil_scores.append(score)
    models[k] = {'model': km, 'labels': labels}

# plot silhueta por k
plt.figure(figsize=(8,4))
plt.plot(ks, sil_scores, marker='o')
plt.xticks(ks)
plt.xlabel('Número de clusters (k)')
plt.ylabel('Silhueta média (Hamming)')
plt.title('Silhueta média por k — K-modes')
plt.grid(True, ls='--', alpha=0.5)
plt.show()

# resultado melhor k
best_k = ks[int(np.argmax(sil_scores))]
print(f"Melhor k por silhueta: {best_k} (silhueta = {max(sil_scores):.4f})")

# resumo tamanhos por cluster do melhor modelo
best_labels = models[best_k]['labels']

```

```

print(pd.Series(best_labels).value_counts().sort_index())

# (opcional) anexar rótulos ao df_km
df_km_clustered = df_km.reset_index(drop=True).copy()
df_km_clustered['cluster'] = best_labels

# exibir primeiras linhas com cluster
print(df_km_clustered.head())

import pandas as pd
import numpy as np
from sklearn.metrics import pairwise_distances, silhouette_score
from kmodes.kmodes import KModes

cols = ['ano','mes','uf','dia_semana_correto','fase_dia_correto',
        'tipo_pista_correto','visibilidade','uso_solo_correto','obito']
df_km = df07_26[cols].dropna().reset_index(drop=True).copy()

# garantir pandas e tipos string (kmodes trabalha bem com objetos)
for c in df_km.columns:
    df_km[c] = df_km[c].astype(str)

# códigos inteiros (consistentes com a ordem das linhas)
X_codes = np.vstack([df_km[col].astype('category').cat.codes.values for col in
df_km.columns]).T

# matriz Hamming (0..1), shape (n,n)
D = pairwise_distances(X_codes, metric='hamming')

ks = list(range(2, 11))
sil_scores = []
models = {}
for k in ks:
    km = KModes(n_clusters=k, init='Huang', n_init=5, verbose=0, random_state=42)
    labels = km.fit_predict(df_km.values)    # ou km.fit_predict(df_km.to_numpy())
    if len(labels) != D.shape[0]:
        raise ValueError("labels length != D.shape[0]")
    score = silhouette_score(D, labels, metric='precomputed')
    sil_scores.append(score)
    models[k] = {'model': km, 'labels': labels}

best_k = ks[int(np.argmax(sil_scores))]
print("Melhor k:", best_k, "silhueta:", max(sil_scores))

```

K-Modes com 10k linhas

Ao efetuar o k-modes com os dados originais, havia estouro da memória. Portanto, separamos uma amostra de 10k sinistros para aplicação. O sorteio usou a proporção de óbitos para manutenção da mesma proporção da amostra. Foi usado o método da silhueta para selecionar a melhor quantidade de grupos, que no caso novamente foram 2 grupos. Dos 10.000 sinistros, foram usados 80% para treino e 20% para validação, conforme mostram os resultados a seguir. Percebemos, assim como nos demais casos testados, a

proporção de óbitos foi maior no primeiro grupo, apesar de também haver uma baixa frequência de óbito no segundo grupo. Esse método, mesmo apresentando uma drástica redução, apresentou em tempo de execução de aproximadamente 45 minutos. Mas os resultados foram expressos na imagem a seguir.

```
!pip install kmodes --quiet
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from kmodes.kmodes import KModes
from sklearn.metrics import silhouette_score
from sklearn.model_selection import train_test_split

# parâmetros
TARGET_SAMPLE = 10000    # tamanho desejado da amostra estratificada
TRAIN_FRAC = 0.8         # fração para treino (ex.: 0.8)
KS = list(range(2, 11))  # valores de k a testar
RANDOM_STATE = 42
COLS = ['ano','mes','uf','dia_semana_correto','fase_dia_correto',
        'tipo_pista_correto','visibilidade','uso_solo_correto','obito']

# verificar se df07_26 existe
assert 'df07_26' in globals(), "df07_26 não está definido. Carregue seu DataFrame antes."

# preparar dataframe e converter para category
df = df07_26[COLS].dropna().copy()
for c in df.columns:
    df[c] = df[c].astype('category')

# amostragem estratificada (sem reposição) até TARGET_SAMPLE
total_n = len(df)
if total_n <= TARGET_SAMPLE:
    sampled = df.reset_index(drop=True)
    print(f"total de observações ({total_n}) <= TARGET_SAMPLE; usando todo o dataset.")
else:
    counts = df['obito'].value_counts()
    props = counts / counts.sum()
    # números inteiros por classe
    n_per_class = (props * TARGET_SAMPLE).astype(int)
    # distribuir o resto entre as classes mais representadas
    remainder = TARGET_SAMPLE - n_per_class.sum()
    if remainder > 0:
        order = props.sort_values(ascending=False).index
        for i in range(remainder):
            n_per_class.loc[order[i % len(order)]] += 1
    # não pedir mais do que existe em cada classe
    n_per_class = np.minimum(n_per_class, counts)
    pieces = []
    for label, n in n_per_class.items():
        if n > 0:
            pieces.append(df[df['obito'] == label].sample(n=int(n),
random_state=RANDOM_STATE))
```

```

sampled = pd.concat(pieces).reset_index(drop=True)

print("distribuição da amostra por 'obito':")
print(sampled['obito'].value_counts())

# separação treino/validação estratificada
train_df, val_df = train_test_split(
    sampled,
    train_size=TRAIN_FRAC,
    stratify=sampled['obito'],
    random_state=RANDOM_STATE
)
train_df = train_df.reset_index(drop=True)
val_df = val_df.reset_index(drop=True)
print(f"tamanhos: treino={len(train_df)}, validação={len(val_df)}")

# garantir categorias consistentes entre conjuntos (usar categorias da amostra completa)
for c in sampled.columns:
    sampled[c] = sampled[c].astype('category')
    train_df[c] = train_df[c].astype('category').cat.set_categories(sampled[c].cat.categories)
    val_df[c] = val_df[c].astype('category').cat.set_categories(sampled[c].cat.categories)

# converter para códigos inteiros para distância Hamming
X_train_codes = np.vstack([train_df[col].cat.codes.values for col in train_df.columns]).T
X_val_codes = np.vstack([val_df[col].cat.codes.values for col in val_df.columns]).T

# loop kmodes + cálculo de silhueta (treino + validação)
sil_train = []
sil_val = []
models = {}

for k in KS:
    km = KModes(n_clusters=k, init='Huang', n_init=3, verbose=0,
random_state=RANDOM_STATE)
    labels_train = km.fit_predict(train_df.values)    # treinar no conjunto de treino
    score_train = silhouette_score(X_train_codes, labels_train, metric='hamming')
    sil_train.append(score_train)

    labels_val = km.predict(val_df.values)           # prever na validação
    score_val = silhouette_score(X_val_codes, labels_val, metric='hamming')
    sil_val.append(score_val)

    models[k] = {'model': km, 'labels_train': labels_train, 'labels_val': labels_val}
    print(f"k={k}: silhueta treino={score_train:.4f}, silhueta validação={score_val:.4f}")

# selecionar melhor k (pela silhueta no treino por padrão)
best_k = KS[int(np.argmax(sil_train))]
best_score_train = max(sil_train)
best_score_val = sil_val[KS.index(best_k)]
print(f"\nmelhor k (pela silhueta no treino): {best_k} (treino={best_score_train:.4f},
validação={best_score_val:.4f})")

```

```

# resumo dos clusters do melhor modelo
best = models[best_k]
print("\ncontagem por cluster (treino):")
print(pd.Series(best['labels_train']).value_counts().sort_index())
print("\ncontagem por cluster (validação):")
print(pd.Series(best['labels_val']).value_counts().sort_index())

# anexar rótulos e mostrar exemplo
train_clustered = train_df.copy()
train_clustered['cluster'] = best['labels_train']
val_clustered = val_df.copy()
val_clustered['cluster'] = best['labels_val']

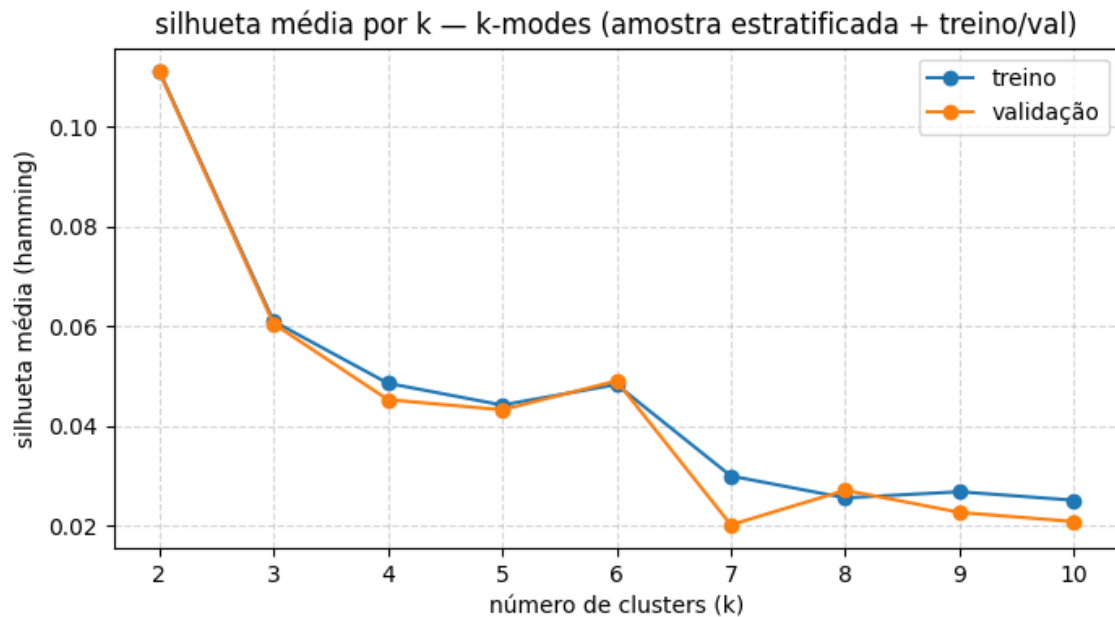
print("\nexemplo (validação, primeiras 10 linhas):")
print(val_clustered.head(10))

# salvar resultados opcionalmente (descomente se quiser)
# train_clustered.to_csv('train_clustered.csv', index=False)
# val_clustered.to_csv('val_clustered.csv', index=False)

# plotar silhuetas
plt.figure(figsize=(8,4))
plt.plot(KS, sil_train, marker='o', label='treino')
plt.plot(KS, sil_val, marker='o', label='validação')
plt.xticks(KS)
plt.xlabel('número de clusters (k)')
plt.ylabel('silhueta média (hamming)')
plt.title('silhueta média por k — k-modes (amostra estratificada + treino/val)')
plt.grid(True, ls='--', alpha=0.5)
plt.legend()
plt.show()

# aplicar o melhor modelo ao dataset completo (opcional)
# atenção: as categorias do dataset completo devem ser alinhadas com `sampled` antes
# de prever
# km_best = models[best_k]['model']
# df_all = df.copy()
# for c in df_all.columns:
#     df_all[c] = df_all[c].astype('category').cat.set_categories(sampled[c].cat.categories)
# labels_all = km_best.predict(df_all.values)
# df_all['cluster'] = labels_all
# print(df_all.head())

```



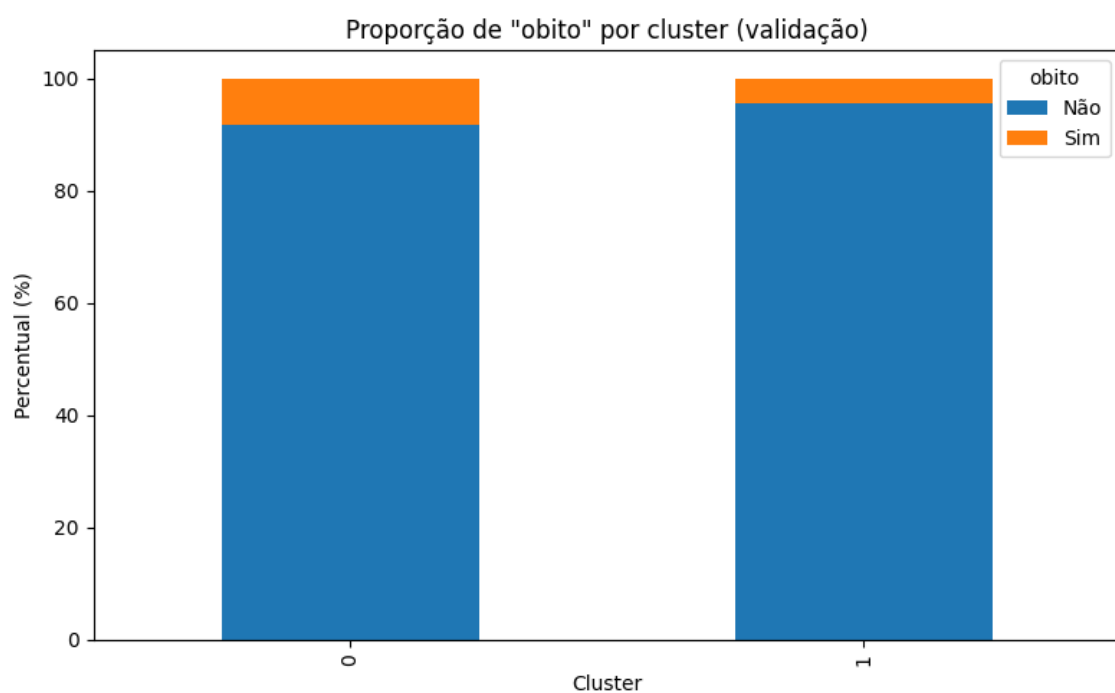
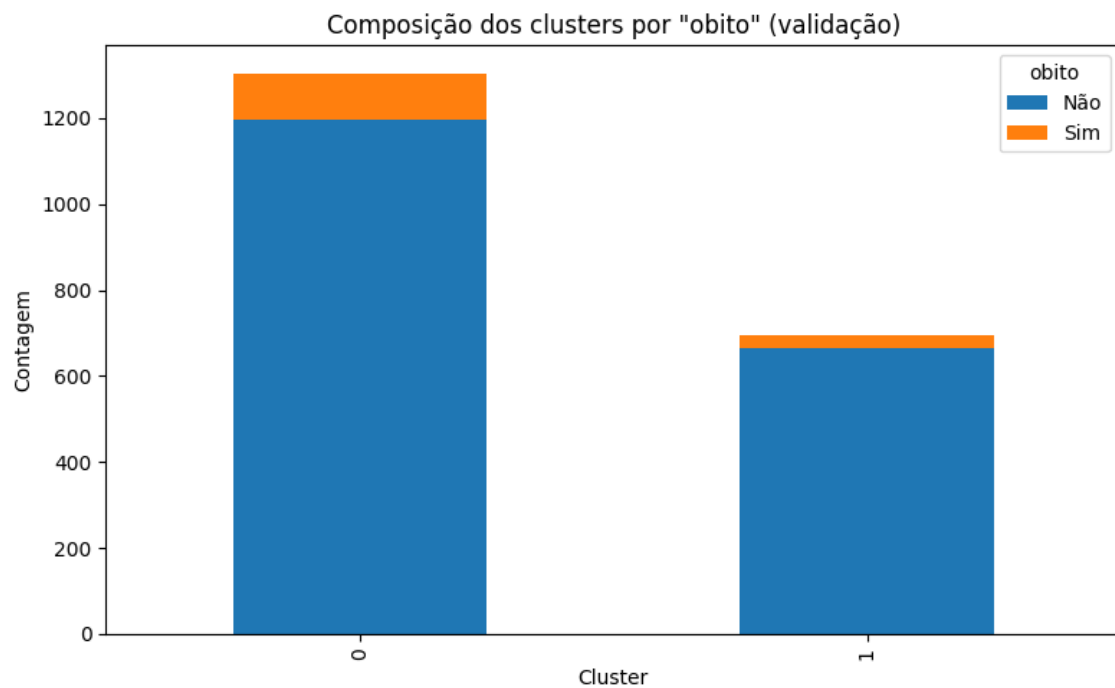
```
# Usando o agrupamento 2-Modes
df_plot = val_clustered.copy()

# agregação: contagem por cluster x obito
ct = pd.crosstab(df_plot['cluster'], df_plot['obito']).sort_index()

# ordem das colunas (opcional) e cores para as classes de 'obito'
cols_obito = ct.columns.tolist()
colors = ['#1f77b4', '#ff7f0e'] # dois tons, troque se quiser

# plot empilhado
ax = ct.plot(kind='bar', stacked=True, color=colors, figsize=(8,5))
ax.set_xlabel('Cluster')
ax.set_ylabel('Contagem')
ax.set_title('Composição dos clusters por "obito" (validação)')
ax.legend(title='obito')
plt.tight_layout()
plt.show()

# gráfico percentual por cluster
ct_pct = ct.div(ct.sum(axis=1), axis=0) * 100
ax = ct_pct.plot(kind='bar', stacked=True, color=colors, figsize=(8,5))
ax.set_xlabel('Cluster')
ax.set_ylabel('Percentual (%)')
ax.set_title('Proporção de "obito" por cluster (validação)')
ax.legend(title='obito', loc='upper right')
plt.tight_layout()
plt.show()
```



K-Modes para k = 2

O método k-Modes para a construção do subespaço bidimensional para discriminação dos pontos evidencia uma diferença entre os grupos, de modo que aqueles com óbito estão na periferia da região.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from kmodes.kmodes import KModes
```

```

from sklearn.metrics import pairwise_distances, silhouette_score
from sklearn.manifold import MDS
from sklearn.model_selection import train_test_split

# parâmetros
TARGET_SAMPLE = 4000
TRAIN_FRAC = 0.8
KS = list(range(2, 11))
RANDOM_STATE = 42
COLS = ['ano','mes','uf','dia_semana_correto','fase_dia_correto',
        'tipo_pista_correto','visibilidade','uso_solo_correto','obito']

# checar df07_26
assert 'df07_26' in globals(), "df07_26 não está definido. Carregue seu DataFrame antes."

# preparar dataframe e converter colunas para category
df = df07_26[COLS].dropna().copy()
for c in df.columns:
    df[c] = df[c].astype('category')

# amostragem estratificada sem reposição até TARGET_SAMPLE
total_n = len(df)
if total_n <= TARGET_SAMPLE:
    sampled = df.reset_index(drop=True)
else:
    counts = df['obito'].value_counts()
    props = counts / counts.sum()
    n_per_class = (props * TARGET_SAMPLE).astype(int)
    remainder = TARGET_SAMPLE - n_per_class.sum()
    if remainder > 0:
        order = props.sort_values(ascending=False).index
        for i in range(remainder):
            n_per_class.loc[order[i % len(order)]] += 1
    n_per_class = np.minimum(n_per_class, counts)
    pieces = []
    for label, n in n_per_class.items():
        if n > 0:
            pieces.append(df[df['obito'] == label].sample(n=int(n),
random_state=RANDOM_STATE))
    sampled = pd.concat(pieces).reset_index(drop=True)

# mostrar distribuição da amostra
print("distribuição da amostra por 'obito':")
print(sampled['obito'].value_counts())

# separar treino/validação estratificada
train_df, val_df = train_test_split(sampled, train_size=TRAIN_FRAC,
stratify=sampled['obito'], random_state=RANDOM_STATE)
train_df = train_df.reset_index(drop=True)
val_df = val_df.reset_index(drop=True)
print(f"tamanhos: treino={len(train_df)}, validação={len(val_df)}")

```

```

# alinhar categorias entre conjuntos (usar categorias da amostra completa)
for c in sampled.columns:
    sampled[c] = sampled[c].astype('category')
    train_df[c] = train_df[c].astype('category').cat.set_categories(sampled[c].cat.categories)
    val_df[c] = val_df[c].astype('category').cat.set_categories(sampled[c].cat.categories)

# converter para códigos inteiros (necessário para distância Hamming)
X_sample_codes = np.vstack([sampled[col].cat.codes.values for col in
                             sampled.columns]).T

# calcular matriz de dissimilaridade Hamming para a amostra (usada no MDS)
D = pairwise_distances(X_sample_codes, metric='hamming')

# projetar com MDS (a partir da matriz de dissimilaridade)
mds = MDS(n_components=2, dissimilarity='precomputed',
          random_state=RANDOM_STATE, n_init=4, max_iter=300)
X_mds = mds.fit_transform(D)

# escolher k (usar silhueta no treino para escolher automaticamente)
# aqui vamos treinar modelos no train_df e avaliar silhueta no treino para selecionar
melhor k
X_train_codes = np.vstack([train_df[col].cat.codes.values for col in train_df.columns]).T
sil_train = []
models = {}
for k in KS:
    km = KModes(n_clusters=k, init='Huang', n_init=3, verbose=0,
                random_state=RANDOM_STATE)
    labels_train = km.fit_predict(train_df.values)
    score_train = silhouette_score(X_train_codes, labels_train, metric='hamming')
    sil_train.append(score_train)
    models[k] = km
    print(f"k={k}: silhueta treino={score_train:.4f}")

best_k = KS[int(np.argmax(sil_train))]
print(f"\nmelhor k (pela silhueta no treino): {best_k}")

# treinar o melhor modelo na amostra completa (sampled) para plotar clusters/medoids
km_best = KModes(n_clusters=best_k, init='Huang', n_init=5, verbose=0,
                 random_state=RANDOM_STATE)
labels_sample = km_best.fit_predict(sampled.values)

# calcular medoids aproximados: para cada cluster, escolher o ponto com menor
distância média ao restante do cluster
medoid_idx = []
for cluster in range(best_k):
    idx = np.where(labels_sample == cluster)[0]
    if len(idx) == 0:
        medoid_idx.append(None)
        continue
    subD = D[np.ix_(idx, idx)]
    mean_dist = subD.mean(axis=1)
    medoid_local = idx[np.argmin(mean_dist)]

```

```

medoid_idx.append(medoid_local)

# preparar DataFrame para plot
plot_df = pd.DataFrame({
    'MDS1': X_mds[:, 0],
    'MDS2': X_mds[:, 1],
    'cluster': labels_sample,
    'obito': sampled['obito'].astype(str).values
})

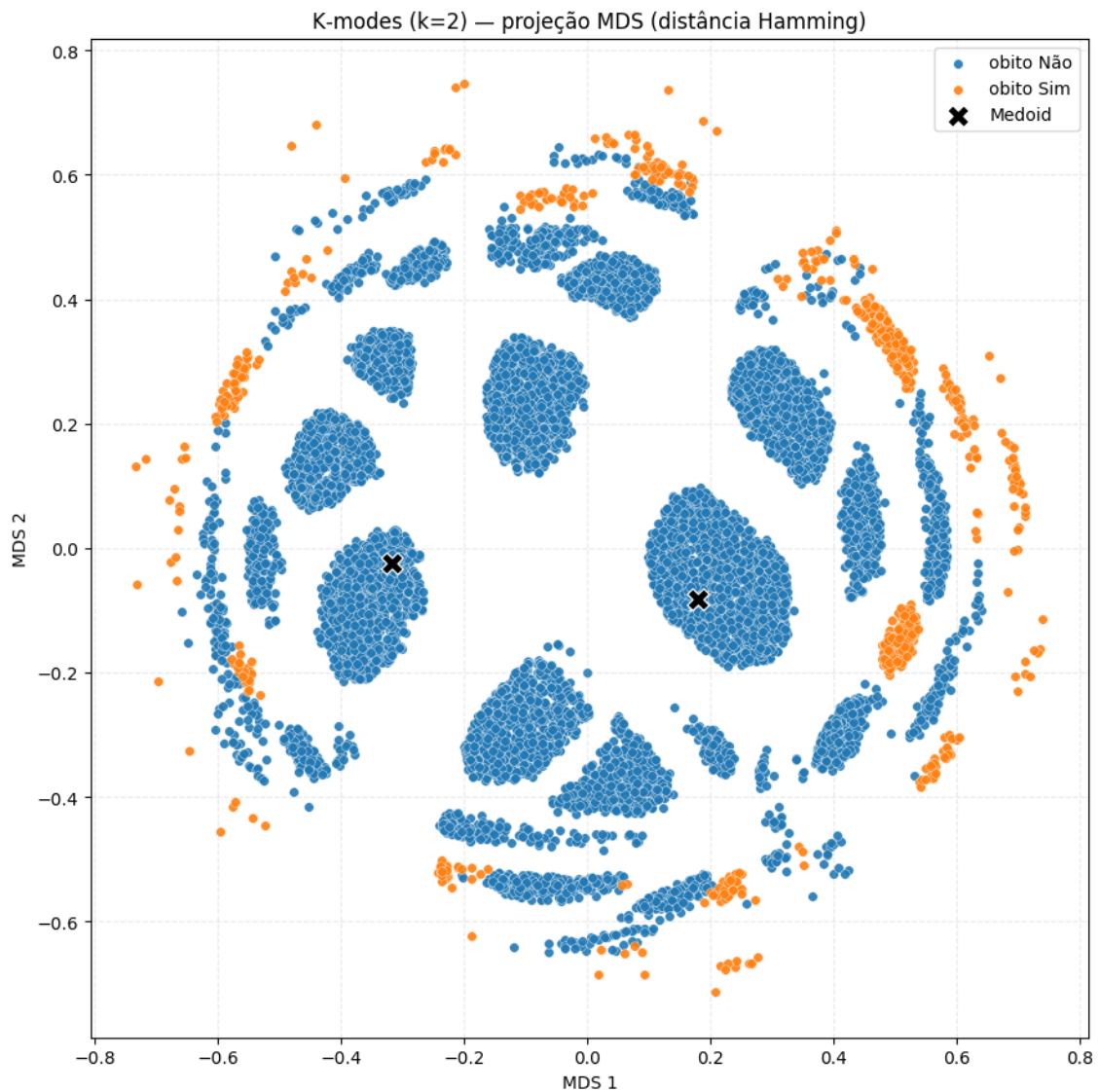
# paleta para 'obito' (duas cores)
unique_obito = sorted(plot_df['obito'].unique())
palette = sns.color_palette("tab10", n_colors=len(unique_obito))
color_map = {ob: palette[i] for i, ob in enumerate(unique_obito)}

# plot: pontos coloridos por 'obito', marcador por cluster opcional (a cor principal é obito)
plt.figure(figsize=(9,9))
for ob in unique_obito:
    sub = plot_df[plot_df['obito'] == ob]
    plt.scatter(sub['MDS1'], sub['MDS2'],
                c=[color_map[ob]],
                s=30, alpha=0.9, label=f"obito {ob}", edgecolors='w', linewidths=0.3)

# desenhar contorno leve por cluster (opcional): centroides dos modos não estão no
# espaço MDS, então marcamos medoids
for ci, midx in enumerate(medoid_idx):
    if midx is None:
        continue
    x_med, y_med = X_mds[midx, 0], X_mds[midx, 1]
    plt.scatter(x_med, y_med, marker='X', s=160, c='k', edgecolors='w', linewidths=0.9,
                label='Medoid' if ci==0 else "")

# ajustes estéticos
plt.xlabel('MDS 1')
plt.ylabel('MDS 2')
plt.title(f'K-modes (k={best_k}) — projeção MDS (distância Hamming)')
plt.legend(frameon=True)
plt.grid(True, linestyle='--', alpha=0.25)
plt.tight_layout()
plt.show()

```



Aprendizado supervisionado

Por se tratar de um método problema de classificação, utilizaremos métodos apropriados para a classificação em uma perspectiva de aprendizado supervisionado, serão elas:

- Regressão Logística (RL);
- Regressão Logística de Firth (RLF);
- Redes Neurais (RN);
- Redes Neurais Profundas (RNP);
- Árvores de Classificação (AC);
- XGboost (X);
- SVM Linear;
- SVM Radial;

- SVM Polinomial.

A RLF é um modelo que trabalha com eventos raros ($p < 0.05$). Veja que a proporção de acidentes com óbitos nos dados está próximo ao limiar proposto no estudo, por isso que foi decidido implementar tal ajuste, inclusive para comparar com a RL tradicional.

De posse dos resultados observados, inclusive pelo teste qui-quadrado e pelas análises não supervisionados, que detectaram associação entre as principais variáveis do estudo, o que justifica o uso das características nos modelos. Para tanto, foram usadas as variáveis: 'uf', 'dia_semana_correto', 'fase_dia_correto', 'tipo_pista_correto', 'visibilidade', 'uso_solo_correto' e 'obito' como a variável target.

Holdout

```
import warnings
warnings.filterwarnings("ignore")

# bibliotecas básicas
import seaborn as sns

# pré-processamento e métricas
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.metrics import (roc_curve, auc, precision_recall_curve,
                             average_precision_score,
                             accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix)

# modelos
from sklearn.linear_model import LogisticRegression
# tentar importar Firth (se disponível); se não, pular
try:
    from logfirth import firth_logistic # pacote externo (opcional)
    HAS_FIRTH = True
except Exception:
    HAS_FIRTH = False

from sklearn.neural_network import MLPClassifier
from sklearn.tree import DecisionTreeClassifier
import xgboost as xgb
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

# -----
# 1) Preparar dados
# -----

# verificar df07_26
assert 'df07_26' in globals(), "df07_26 não está definido no ambiente."
```

```

# selecionar colunas
FEATURE_COLS = ['dia_semana_correto','fase_dia_correto',
                 'tipo_pista_correto','visibilidade','uso_solo_correto']
TARGET_COL = 'obito'

df = df07_26[['*FEATURE_COLS, TARGET_COL]].dropna().copy()

# converter target binário para 0/1 (assume que 'Sim' ou '1' indica óbito)
df[TARGET_COL] = df[TARGET_COL].astype(str)
positive_labels = ['sim','Sim','SIM','1','True','true','TRUE','S','s','yes','Yes','YES']
df['y'] = df[TARGET_COL].apply(lambda x: 1 if x in positive_labels else 0)

# tratar features como categóricas (forçar dtype)
for c in FEATURE_COLS:
    df[c] = df[c].astype('category')

X = df[FEATURE_COLS]
y = df['y'].values

# dividir treino/ teste (estratificado)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# pipeline de pré-processamento: OneHot para categóricas
cat_transformer = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

preprocessor_for_linear = ColumnTransformer([
    ('cat', cat_transformer, FEATURE_COLS)
], remainder='drop')

# pré-processador para modelos que precisam de escala (SVM / NN)
preprocessor_scaled = ColumnTransformer([
    ('cat', cat_transformer, FEATURE_COLS)
], remainder='drop')

# transformar X_train/X_test em arrays one-hot (reutilizável)
X_train_ohe = preprocessor_for_linear.fit_transform(X_train)
X_test_ohe = preprocessor_for_linear.transform(X_test)

# guardar feature names de OHE (opcional)
try:
    ohe_cols = preprocessor_for_linear.named_transformers_['cat'].get_feature_names_out(FEATURE_COLS)
except Exception:
    ohe_cols = None

# -----
# 2) Definir modelos
# -----

models = {}

```

Os dados sem observações faltantes foram separados em treinamento (80%) e teste (20%) para avaliação dos modelos, totalizando 522.504 sinistros com as observações completas e mantendo a mesma proporção de acordo com a variável "obito".

Regressão Logística (RL)

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer

# Definindo o modelo
logreg = Pipeline(steps=[
    ('clf', LogisticRegression(max_iter=1000, solver='liblinear'))
])

models['Logistic'] = logreg

model_logistic = models.get('Logistic', None)

# AJUSTE
if model_logistic is None:
    # criar/treinar caso não exista
    model_logistic = Pipeline(steps=[
        ('clf', LogisticRegression(max_iter=1000, solver='liblinear'))
    ])
    model_logistic.fit(X_train_ohe, y_train)
    models['Logistic'] = model_logistic

elif not hasattr(model_logistic, 'named_steps') or \
    'clf' not in model_logistic.named_steps or \
    not hasattr(model_logistic.named_steps['clf'], "coef_"):
    model_logistic.fit(X_train_ohe, y_train)

pipe = models['Logistic']

try:
    proba = pipe.predict_proba(X_test_ohe)[:, 1]
except Exception:
    try:
        df_dec = pipe.decision_function(X_test_ohe)
        proba = 1 / (1 + np.exp(-df_dec))
    except Exception:
        proba = pipe.predict(X_test_ohe).astype(float)

yhat = (proba >= 0.5).astype(int)

accuracy = accuracy_score(y_test, yhat)
precision = precision_score(y_test, yhat, zero_division=0)
sensitivity = recall_score(y_test, yhat, zero_division=0)
f1 = f1_score(y_test, yhat, zero_division=0)
```

```

tn, fp, fn, tp = confusion_matrix(y_test, yhat).ravel()
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
fpr, tpr, _ = roc_curve(y_test, proba)
auc_roc = auc(fpr, tpr)
auprc = average_precision_score(y_test, proba)

```

```

metrics = {
    'accuracy': accuracy,
    'precision': precision,
    'sensitivity (recall)': sensitivity,
    'specificity': specificity,
    'f1': f1,
    'auc_roc': auc_roc,
    'auprc': auprc
}

```

```

print({k: round(v, 4) for k, v in metrics.items()})

```

Rede Neural (RN)

```

from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_curve, auc, average_precision_score, confusion_matrix)

# Treinar o modelo NN_shallow se ainda não estiver treinado
if 'NN_shallow' not in models or models['NN_shallow'] is None:
    models['NN_shallow'] = Pipeline([
        ('clf', MLPClassifier(hidden_layer_sizes=(64,), activation='relu',
                              solver='adam', max_iter=300, random_state=42))
    ])

pipe = models['NN_shallow']

# Verificação do ajuste
need_fit = True
try:
    # tenta predict_proba com um exemplo (usa X_train_ohe disponível)
    if hasattr(pipe, "predict_proba"):
        pipe.predict_proba(X_train_ohe.iloc[:1]) if hasattr(X_train_ohe, "iloc") else
X_train_ohe[:1])
    else:
        pipe.predict(X_train_ohe.iloc[:1] if hasattr(X_train_ohe, "iloc") else X_train_ohe[:1])
        need_fit = False
except Exception:
    need_fit = True

if need_fit:
    pipe.fit(X_train_ohe, y_train)

# obter probabilidades (ou fallback)

```

```
try:
    proba = pipe.predict_proba(X_test_ohe)[: , 1]
except Exception:
    try:
        df_dec = pipe.decision_function(X_test_ohe)
        proba = 1 / (1 + np.exp(-df_dec))
    except Exception:
        proba = pipe.predict(X_test_ohe).astype(float)

yhat = (proba >= 0.5).astype(int)

accuracy = accuracy_score(y_test, yhat)
precision = precision_score(y_test, yhat, zero_division=0)
sensitivity = recall_score(y_test, yhat, zero_division=0)
f1 = f1_score(y_test, yhat, zero_division=0)
tn, fp, fn, tp = confusion_matrix(y_test, yhat).ravel()
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
fpr, tpr, _ = roc_curve(y_test, proba)
auc_roc = auc(fpr, tpr)
auprc = average_precision_score(y_test, proba)

metrics = {
    'accuracy': accuracy,
    'precision': precision,
    'sensitivity (recall)': sensitivity,
    'specificity': specificity,
    'f1': f1,
    'auc_roc': auc_roc,
    'auprc': auprc
}

print({k: round(v, 4) for k, v in metrics.items()})
```

```
from sklearn.pipeline import Pipeline
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_curve, auc, average_precision_score, confusion_matrix)
import numpy as np

if 'models' not in globals() or not isinstance(models, dict):
    models = {}

# Treinar o modelo NN_deep se ainda não estiver treinado
if 'NN_deep' not in models or models['NN_deep'] is None:
    models['NN_deep'] = Pipeline([
        ('clf', MLPClassifier(hidden_layer_sizes=(256, 128, 64, 32),
                             activation='relu',
                             solver='adam',
                             early_stopping=True,
                             validation_fraction=0.1,
```

```

        n_iter_no_change=20,
        max_iter=1000,
        random_state=42))
    ])

pipe = models['NN_deep']

# helper para obter uma amostra compatível com tipo de X_train_ohe
def _sample(X):
    return X.iloc[:1] if hasattr(X, "iloc") else X[:1]

# verifica se precisa treinar (maneira robusta)
need_fit = True
try:
    if hasattr(pipe, "predict_proba"):
        pipe.predict_proba(_sample(X_train_ohe))
    else:
        pipe.predict(_sample(X_train_ohe))
    need_fit = False
except Exception:
    need_fit = True

if need_fit:
    pipe.fit(X_train_ohe, y_train)

# obter probabilidades (com fallbacks)
try:
    proba = pipe.predict_proba(X_test_ohe)[: , 1]
except Exception:
    try:
        df_dec = pipe.decision_function(X_test_ohe)
        proba = 1 / (1 + np.exp(-df_dec))
    except Exception:
        proba = pipe.predict(X_test_ohe).astype(float)

yhat = (proba >= 0.5).astype(int)

accuracy = accuracy_score(y_test, yhat)
precision = precision_score(y_test, yhat, zero_division=0)
sensitivity = recall_score(y_test, yhat, zero_division=0)
f1 = f1_score(y_test, yhat, zero_division=0)
tn, fp, fn, tp = confusion_matrix(y_test, yhat).ravel()
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
fpr, tpr, _ = roc_curve(y_test, proba)
auc_roc = auc(fpr, tpr)
auprc = average_precision_score(y_test, proba)

metrics = {
    'accuracy': accuracy,
    'precision': precision,
    'sensitivity (recall)': sensitivity,
    'specificity': specificity,

```

```

    'f1': f1,
    'auc_roc': auc_roc,
    'auprc': auprc
}

```

```

print({k: round(v, 4) for k, v in metrics.items()})

```

Gráficos das redes

```

# Gráficos das redes
def plot_mlp(pipe, title=None, max_edges_to_show=None):
    # pipe: Pipeline contendo ('clf', MLPClassifier) já fit
    if 'clf' not in pipe.named_steps:
        raise ValueError("Pipeline não contém step 'clf'")
    clf = pipe.named_steps['clf']
    if not hasattr(clf, 'coefs_'):
        raise ValueError("O classificador não parece estar treinado (sem coefs_). Rode .fit antes.")
    coefs = clf.coefs_
    intercepts = clf.intercepts_
    layer_sizes = [coef.shape[0] for coef in coefs] + [coefs[-1].shape[1]]

    # posições dos nós
    n_layers = len(layer_sizes)
    v_spacing = 1.0
    h_spacing = 2.5
    positions = []
    for i, size in enumerate(layer_sizes):
        x = i * h_spacing
        y_top = (size - 1) * v_spacing / 2.0
        layer_pos = [(x, y_top - j * v_spacing) for j in range(size)]
        positions.append(layer_pos)

    fig, ax = plt.subplots(figsize=(max(6, n_layers*2), 6))
    ax.set_title(title or getattr(pipe, '__repr__', lambda: 'MLP')())
    ax.axis('off')

    # desenhar arestas
    for i, W in enumerate(coefs):
        src_pos = positions[i]
        tgt_pos = positions[i+1]
        flat_weights = np.abs(W).flatten()
        if max_edges_to_show is None:
            thr = 0.0
        else:
            # manter apenas os maiores pesos se desejado
            kth = np.sort(flat_weights)[-min(max_edges_to_show, flat_weights.size)]
            thr = max(0.0, kth)
        for s_idx, s in enumerate(src_pos):
            for t_idx, t in enumerate(tgt_pos):
                w = W[s_idx, t_idx]
                if np.abs(w) <= thr:

```

```

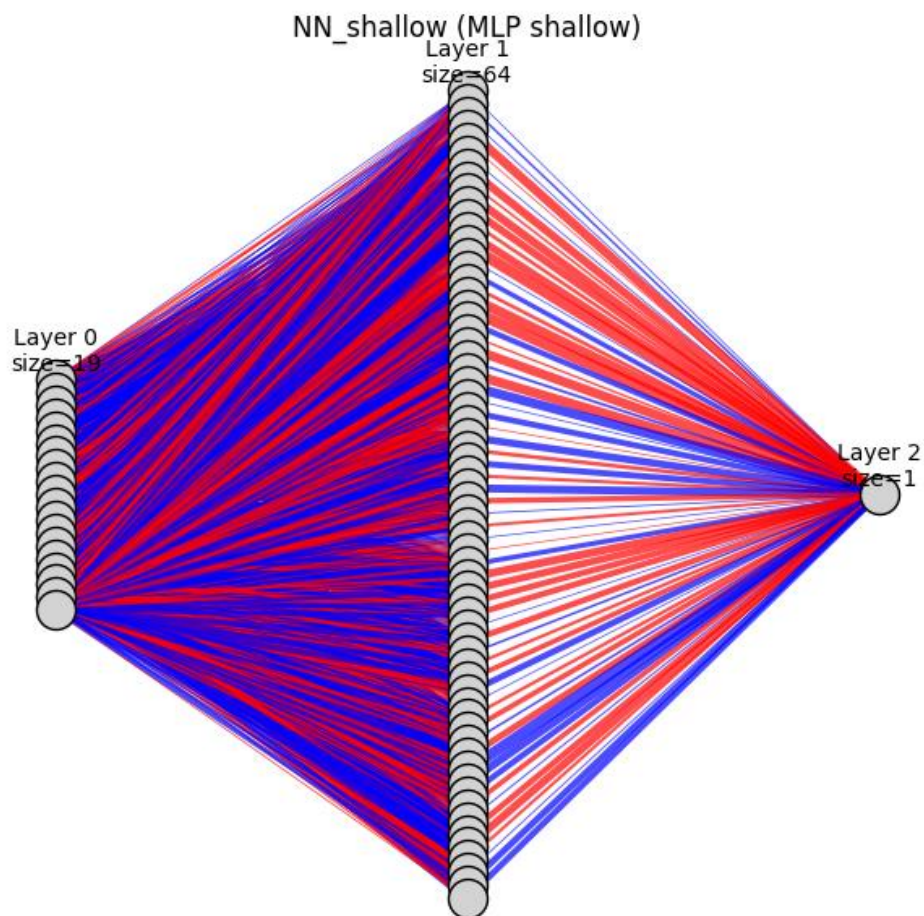
        continue
    lw = 0.5 + 4.5 * (np.abs(w) / np.max(np.abs(W))) # largura relativa
    color = 'red' if w < 0 else 'blue'
    ax.plot([s[0], t[0]], [s[1], t[1]], color=color, linewidth=lw, alpha=0.7)

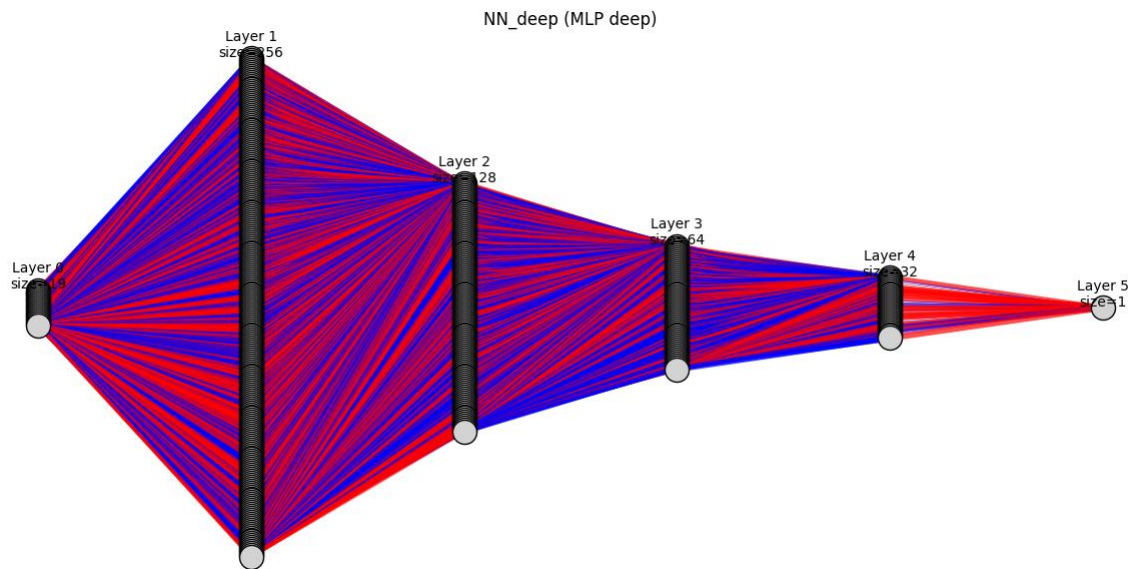
# desenhar nós
for layer in positions:
    xs = [p[0] for p in layer]
    ys = [p[1] for p in layer]
    ax.scatter(xs, ys, s=300, color='lightgray', edgecolors='k', zorder=3)
# labels das camadas
for i, layer in enumerate(positions):
    ax.text(layer[0][0], max(p[1] for p in layer)+0.6,
            f"Layer {i}\nsize={len(layer)}", ha='center')

plt.tight_layout()
plt.show()

# Exemplos de uso (ajuste nomes conforme seu dicionário models)
# Certifique-se que os modelos foram treinados (fit)
plot_mlp(models['NN_shallow'], title='NN_shallow (MLP shallow)')
# se tiver e estiver treinado:
plot_mlp(models['NN_deep'], title='NN_deep (MLP deep)')

```





Árvore de Classificação

```
# 1) Verificando chaves disponíveis
print("models keys:", list(models.keys()))

# 2) Se 'DecisionTree' não existir, (re)defina-a:
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline

if 'DecisionTree' not in models:
    models['DecisionTree'] = Pipeline([
        ('pre', preprocessor_for_linear),
        ('clf', DecisionTreeClassifier(max_depth=None, class_weight='balanced',
random_state=42))
    ])
    print("DecisionTree adicionada ao dicionário models.")

# 3) Treine e calcule métricas
pipe = models['DecisionTree']
# treina se necessário
try:
    _ = pipe.named_steps['clf'].tree_
    fitted = True
except Exception:
    fitted = False

if not fitted:
    pipe.fit(X_train, y_train)

# obter probabilidades (ou fallback)
import numpy as np
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
roc_curve, auc, average_precision_score, confusion_matrix)

try:
```

```

    proba = pipe.predict_proba(X_test)[:, 1]
except Exception:
    try:
        df_dec = pipe.decision_function(X_test)
        proba = 1 / (1 + np.exp(-df_dec))
    except Exception:
        proba = pipe.predict(X_test).astype(float)

yhat = (proba >= 0.5).astype(int)

accuracy = accuracy_score(y_test, yhat)
precision = precision_score(y_test, yhat, zero_division=0)
sensitivity = recall_score(y_test, yhat, zero_division=0)
f1 = f1_score(y_test, yhat, zero_division=0)
tn, fp, fn, tp = confusion_matrix(y_test, yhat).ravel()
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
fpr, tpr, _ = roc_curve(y_test, proba)
auc_roc = auc(fpr, tpr)
auprc = average_precision_score(y_test, proba)

metrics = {
    'accuracy': accuracy,
    'precision': precision,
    'sensitivity (recall)': sensitivity,
    'specificity': specificity,
    'f1': f1,
    'auc_roc': auc_roc,
    'auprc': auprc
}

print({k: round(v, 4) for k, v in metrics.items()})

```

Gráfico da Árvore de Classificação

```

from sklearn.tree import plot_tree, export_text
import matplotlib.pyplot as plt

# garantir que o pipeline exista
if 'DecisionTree' not in models:
    from sklearn.tree import DecisionTreeClassifier
    from sklearn.pipeline import Pipeline
    models['DecisionTree'] = Pipeline([
        ('pre', preprocessor_for_linear),
        ('clf', DecisionTreeClassifier(max_depth=None, class_weight='balanced',
random_state=42))
    ])

pipe = models['DecisionTree']

# treinar se não estiver ajustado
try:
    _ = pipe.named_steps['clf'].tree_

```

```
fitted = True
except Exception:
    fitted = False
```

```
if not fitted:
    pipe.fit(X_train, y_train)
```

```
# tentar obter nomes das features após o pré-processamento (OneHotEncoder)
feature_names = None
try:
```

```
    pre = pipe.named_steps['pre']
    ohe = pre.named_transformers_['cat']
    feature_names = ohe.get_feature_names_out(FEATURE_COLS)
```

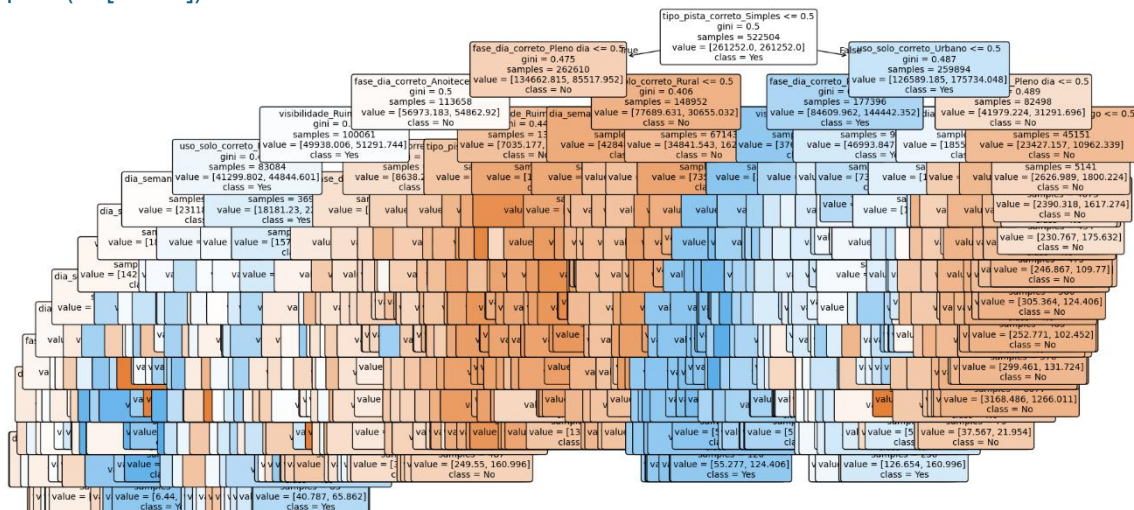
```
except Exception:
    try:
```

```
        # se preprocessor_for_linear já estiver ajustado no escopo
        feature_names = ohe_cols
```

```
    except Exception:
        feature_names = FEATURE_COLS
```

```
# plot da árvore (ajuste max_depth para visualizar menos/mais níveis)
clf = pipe.named_steps['clf']
plt.figure(figsize=(20, 10))
plot_tree(clf, feature_names=feature_names, class_names=['No','Yes'], filled=True,
rounded=True, fontsize=10)
plt.show()
```

```
# representar a árvore em texto (primeiros 5000 caracteres)
txt = export_text(clf, feature_names=list(feature_names) if feature_names is not None else
None, max_depth=4)
print(txt[:5000])
```



```
|--- tipo_pista_correto_Simples <= 0.5
| |--- fase_dia_correto_Plano dia <= 0.50
| | |--- fase_dia_correto_Anoitecer <= 0.50
| | | |--- visibilidade_Ruim <= 0.50
| | | | |--- uso_solo_correto_Rural <= 0.50
| | | | | |--- truncated branch of depth 10
```

```

| | | | |--- uso_solo_correto_Rural > 0.50
| | | | |--- truncated branch of depth 10
| | | |--- visibilidade_Ruim > 0.50
| | | | |--- dia_semana_correto_Sexta <= 0.50
| | | | |--- truncated branch of depth 9
| | | | |--- dia_semana_correto_Sexta > 0.50
| | | | |--- truncated branch of depth 4
| | |--- fase_dia_correto_Anoitecer > 0.50
| | | |--- visibilidade_Ruim <= 0.50
| | | | |--- tipo_pista_correto_Múltipla <= 0.50
| | | | |--- truncated branch of depth 9
| | | | |--- tipo_pista_correto_Múltipla > 0.50
| | | | |--- truncated branch of depth 8
| | | |--- visibilidade_Ruim > 0.50
| | | | |--- tipo_pista_correto_Múltipla <= 0.50
| | | | |--- truncated branch of depth 8
| | | | |--- tipo_pista_correto_Múltipla > 0.50
| | | | |--- truncated branch of depth 4
| |--- fase_dia_correto_Plano dia > 0.50
| | |--- uso_solo_correto_Rural <= 0.50
| | | |--- dia_semana_correto_Domingo <= 0.50
| | | | |--- dia_semana_correto_Sábado <= 0.50
| | | | |--- truncated branch of depth 8
| | | | |--- dia_semana_correto_Sábado > 0.50
| | | | |--- truncated branch of depth 4
| | | |--- dia_semana_correto_Domingo > 0.50
| | | | |--- visibilidade_Parcial <= 0.50
| | | | |--- truncated branch of depth 3
| | | | |--- visibilidade_Parcial > 0.50
| | | | |--- class: 0
| | |--- uso_solo_correto_Rural > 0.50
| | | |--- visibilidade_Ruim <= 0.50
| | | | |--- tipo_pista_correto_Dupla <= 0.50
| | | | |--- truncated branch of depth 8
| | | | |--- tipo_pista_correto_Dupla > 0.50
| | | | |--- truncated branch of depth 8
| | | |--- visibilidade_Ruim > 0.50
| | | | |--- dia_semana_correto_Quinta <= 0.50
| | | | |--- truncated branch of depth 7
| | | | |--- dia_semana_correto_Quinta > 0.50
| | | | |--- truncated branch of depth 2
|--- tipo_pista_correto_Simples > 0.50
| |--- uso_solo_correto_Urbano <= 0.50
| | |--- fase_dia_correto_Plano dia <= 0.50
| | | |--- visibilidade_Ruim <= 0.50
| | | | |--- fase_dia_correto_Anoitecer <= 0.50
| | | | |--- truncated branch of depth 9
| | | | |--- fase_dia_correto_Anoitecer > 0.50
| | | | |--- truncated branch of depth 8
| | | |--- visibilidade_Ruim > 0.50
| | | | |--- fase_dia_correto_Anoitecer <= 0.50
| | | | |--- truncated branch of depth 8

```



```

    ])

pipe = models['RandomForest']

# checar se já foi treinado; senão, treinar
fitted = False
try:
    # RandomForestClassifier tem atributo estimators_ após fit
    _ = pipe.named_steps['clf'].estimators_
    fitted = True
except Exception:
    fitted = False

if not fitted:
    pipe.fit(X_train, y_train)

# obter probabilidades (fallbacks se necessário)
try:
    proba = pipe.predict_proba(X_test)[:, 1]
except Exception:
    try:
        df_dec = pipe.decision_function(X_test)
        proba = 1 / (1 + np.exp(-df_dec))
    except Exception:
        proba = pipe.predict(X_test).astype(float)

yhat = (proba >= 0.5).astype(int)

# métricas
accuracy = accuracy_score(y_test, yhat)
precision = precision_score(y_test, yhat, zero_division=0)
sensitivity = recall_score(y_test, yhat, zero_division=0)
f1 = f1_score(y_test, yhat, zero_division=0)
tn, fp, fn, tp = confusion_matrix(y_test, yhat).ravel()
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
fpr, tpr, _ = roc_curve(y_test, proba)
auc_roc = auc(fpr, tpr)
auprc = average_precision_score(y_test, proba)

metrics = {
    'accuracy': accuracy,
    'precision': precision,
    'sensitivity (recall)': sensitivity,
    'specificity': specificity,
    'f1': f1,
    'auc_roc': auc_roc,
    'auprc': auprc
}

print({k: round(v, 4) for k, v in metrics.items()})

```

XGBoost

```
# assume X_train, y_train, X_test, y_test e models já existem
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_curve, auc, average_precision_score, confusion_matrix)
import numpy as np

# garantir que a entrada exista no dicionário
if 'XGBoost' not in models:
    import xgboost as xgb
    from sklearn.pipeline import Pipeline
    models['XGBoost'] = Pipeline([
        ('pre', preprocessor_for_linear),
        ('clf', xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss',
                                random_state=42))
    ])

pipe = models['XGBoost']

# checar se já foi treinado; senão, treinar
fitted = False
try:
    # XGBClassifier tem atributo classes_ após fit
    _ = pipe.named_steps['clf'].classes_
    fitted = True
except Exception:
    fitted = False

if not fitted:
    pipe.fit(X_train, y_train)

# obter probabilidades (fallbacks se necessário)
try:
    proba = pipe.predict_proba(X_test)[:, 1]
except Exception:
    try:
        df_dec = pipe.decision_function(X_test)
        proba = 1 / (1 + np.exp(-df_dec))
    except Exception:
        proba = pipe.predict(X_test).astype(float)

yhat = (proba >= 0.5).astype(int)

# métricas
accuracy = accuracy_score(y_test, yhat)
precision = precision_score(y_test, yhat, zero_division=0)
sensitivity = recall_score(y_test, yhat, zero_division=0)
f1 = f1_score(y_test, yhat, zero_division=0)
tn, fp, fn, tp = confusion_matrix(y_test, yhat).ravel()
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
fpr, tpr, _ = roc_curve(y_test, proba)
auc_roc = auc(fpr, tpr)
```

```
auprc = average_precision_score(y_test, proba)
```

```
metrics = {  
    'accuracy': accuracy,  
    'precision': precision,  
    'sensitivity (recall)': sensitivity,  
    'specificity': specificity,  
    'f1': f1,  
    'auc_roc': auc_roc,  
    'auprc': auprc  
}
```

```
print({k: round(v, 4) for k, v in metrics.items()})
```

```
print("keys:", list(models.keys()))  
for k, v in models.items():  
    print(k, "->", type(v))
```

SVM (*Support Vector Machine*) Linear - Máquina de Vetores de Suporte

```
from sklearn.pipeline import Pipeline  
from sklearn.svm import LinearSVC  
from sklearn.calibration import CalibratedClassifierCV
```

```
# Garantir que exista o modelo SVM linear no dicionário  
if 'SVM_linear' not in models:  
    models['SVM_linear'] = Pipeline([  
        ('pre', preprocessor_for_linear),  
        ('clf', CalibratedClassifierCV(LinearSVC(), cv=5)) # calibração para probabilidades  
    ])
```

```
pipe = models['SVM_linear']
```

```
# checar se já foi treinado; senão, treinar  
fitted = False  
try:
```

```
    _ = pipe.named_steps['clf'].classes_  
    fitted = True
```

```
except Exception:  
    fitted = False
```

```
if not fitted:  
    pipe.fit(X_train, y_train)
```

```
# obter probabilidades
```

```
try:
```

```
    proba = pipe.predict_proba(X_test)[:, 1]
```

```
except Exception:
```

```
    # caso não haja predict_proba (poucas vezes), usar decision_function calibrada
```

```
    df = pipe.decision_function(X_test)
```

```
    # Convertendo scores para probabilidades com sigmoid
```

```
    proba = 1 / (1 + np.exp(-df))
```

```

yhat = (proba >= 0.5).astype(int)

# métricas
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_curve, auc, average_precision_score, confusion_matrix)

accuracy = accuracy_score(y_test, yhat)
precision = precision_score(y_test, yhat, zero_division=0)
sensitivity = recall_score(y_test, yhat, zero_division=0)
f1 = f1_score(y_test, yhat, zero_division=0)
tn, fp, fn, tp = confusion_matrix(y_test, yhat).ravel()
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
fpr, tpr, _ = roc_curve(y_test, proba)
auc_roc = auc(fpr, tpr)
auprc = average_precision_score(y_test, proba)

metrics = {
    'accuracy': accuracy,
    'precision': precision,
    'sensitivity (recall)': sensitivity,
    'specificity': specificity,
    'f1': f1,
    'auc_roc': auc_roc,
    'auprc': auprc
}

print({k: round(v, 4) for k, v in metrics.items()})

```

AUC e AUPRC

```

import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc, precision_recall_curve,
average_precision_score
import numpy as np

# ajuste de tamanhos (modifique valores conforme desejar)
title_fs = 18
label_fs = 18
legend_fs = 16
tick_fs = 16

def get_proba_for_pipeline(pipe, X_raw, X_ohc=None):
    """
    Retorna array de probabilidades (classe 1) para um pipeline/estimator.
    Estratégia:
    1) tentar pipe.predict_proba(X_raw)
    2) tentar usar step 'pre' ou 'transformer' para transformar X_raw e usar clf.predict_proba
    3) tentar pipe.decision_function / fallback pipe.predict
    4) se X_ohc fornecido, tentar com ele
    """

```

```

# 1) tentativa direta
try:
    return pipe.predict_proba(X_raw)[: , 1]
except Exception:
    pass

# 2) se pipeline com named_steps e um pré processador
try:
    if hasattr(pipe, "named_steps"):
        # procura por step com nome comum 'pre' ou 'transformer' ou 'prep'
        for candidate in ('pre', 'preproc', 'preprocessor', 'transformer', 'prep'):
            if candidate in pipe.named_steps:
                pre = pipe.named_steps[candidate]
                clf = pipe.named_steps.get('clf') or pipe.steps[-1][1]
                X_trans = pre.transform(X_raw)
                try:
                    return clf.predict_proba(X_trans)[: , 1]
                except Exception:
                    # talvez clf espere shape diferente — continue
                    pass
            # se o último step é o classificador separado do pre
            last_name, last_est = pipe.steps[-1]
            if last_name != 'pre' and hasattr(last_est, "predict_proba"):
                # tenta transformar com tudo menos o último step
                from sklearn.pipeline import Pipeline
                pre_pipe = Pipeline(pipe.steps[:-1]) if len(pipe.steps) > 1 else None
                if pre_pipe is not None:
                    X_trans = pre_pipe.transform(X_raw)
                    return last_est.predict_proba(X_trans)[: , 1]
except Exception:
    pass

# 3) decision_function / predict em X_raw
try:
    df = pipe.decision_function(X_raw)
    return 1 / (1 + np.exp(-np.asarray(df)))
except Exception:
    pass
try:
    pred = pipe.predict(X_raw).astype(float)
    return pred
except Exception:
    pass

# 4) usar X_ohe se fornecido
if X_ohe is not None:
    try:
        return pipe.predict_proba(X_ohe)[: , 1]
    except Exception:
        try:
            df = pipe.decision_function(X_ohe)
            return 1 / (1 + np.exp(-np.asarray(df)))

```

```

except Exception:
    try:
        return pipe.predict(X_ohe).astype(float)
    except Exception:
        pass

# se tudo falhar, lança erro para ser capturado fora
raise RuntimeError("Could not obtain probabilities for pipeline")

# PLOT usando a função acima
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
axes[0].plot([0, 1], [0, 1], 'k--', alpha=0.4)
colors = plt.cm.tab10

i = 0
for name, pipe in models.items():
    try:
        # forneça X_test_ohe se você tiver (substitua o nome se for outro)
        X_ohe = globals().get('X_test_ohe', None)
        proba = get_proba_for_pipeline(pipe, X_test, X_ohe=X_ohe)
        proba = np.asarray(proba)

        print(f"{name}:    proba    shape={proba.shape},    min={proba.min():.4f},
max={proba.max():.4f}")

        if np.all(proba == proba[0]):
            print(f" -> {name} outputs constant score; skipping.")
            continue

        fpr, tpr, _ = roc_curve(y_test, proba)
        roc_auc = auc(fpr, tpr)
        axes[0].plot(fpr, tpr, label=f"{name} (AUC={roc_auc:.3f})", color=colors(i % 10),
linewidth=2)

        recall, precision, _ = precision_recall_curve(y_test, proba)
        ap = average_precision_score(y_test, proba)
        axes[1].plot(recall, precision, label=f"{name} (AUPRC={ap:.3f})", color=colors(i % 10),
linewidth=2)

        i += 1
    except Exception as e:
        print(f"Skipped {name}: {e}")
        continue

# formatação e fontes
axes[0].set_xlabel("False Positive Rate", fontsize=label_fs)
axes[0].set_ylabel("True Positive Rate", fontsize=label_fs)
axes[0].set_title("ROC Curves", fontsize=title_fs)
axes[0].legend(loc="lower right", fontsize=legend_fs)
axes[0].tick_params(axis='both', which='major', labelsize=tick_fs)
axes[0].grid(alpha=0.3)

```

```

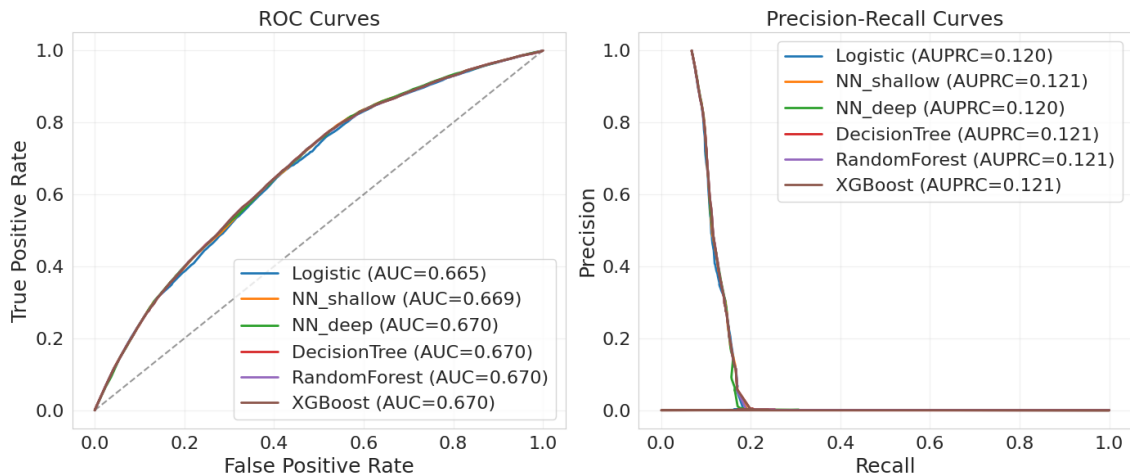
axes[1].set_xlabel("Recall", fontsize=label_fs)
axes[1].set_ylabel("Precision", fontsize=label_fs)
axes[1].set_title("Precision-Recall Curves", fontsize=title_fs)
axes[1].legend(loc="upper right", fontsize=legend_fs)
axes[1].tick_params(axis='both', which='major', labelsize=tick_fs)
axes[1].grid(alpha=0.3)

```

```

plt.tight_layout()
plt.show()

```



Discussão sobre os modelos

O método SVM foi aplicado com as funções linear, radial e polinomial, de modo que a linear foi a melhor. Os resultados foram aplicados no VS Code, já que o colab demandou muito tempo e a memória não suportou. Destaca-se que apenas a SVM não foi aplicada no código do presente MVP. Os resultados da qualidade dos modelos são apresentados na tabela abaixo.

De posse dos resultados, percebemos certo desbalanceamento, de modo que vários modelos (RN, RNP, XGBoost) aparentemente classificam tudo como negativo (alta acurácia 0,9317, precisão e sensibilidade = 0), o que gera AUC ~ 0,67, mas baixo desempenho para detectar positivos. Modelos que efetivamente detectam positivos, como Regressão Logística (RL), Árvore de Classificação (AC) e Random Forest (RF) — apresentam sensibilidade alta (0,67–0,74), porém baixa precisão (~0,10) e F1. Isso indica muitos falsos positivos.

SVM linear tem alta especificidade (0,86) mas baixa sensibilidade (0,20) — prefere negativos, logo perde positivos. Dessa forma, uma possível recomendação de melhor ajuste associado ao objetivo de detectar positivos com equilíbrio entre sensibilidade e precisão, seria escolher como base AC ou RF (AC tem a maior sensibilidade 0,7414 e AUC/AUPRC competitivos).

Porém, podemos fazer corrigir o desbalanceamento, aplicar *oversampling* (SMOTE/ADASYN) ou *undersampling* no treino. Após o *undersampling*, tivemos uma amostra com 111.562, de modo que 40% dos dados tiveram ocorrência de óbito e 60% não.

Tabela 1: Medidas de qualidade do ajuste para os modelos avaliados

Model	Acurácia	Precisão	Sensib.	Especif.	F1	AUPRC	AUC
RL	0,5754	0,1025	0,6726	0,5683	0,1780	0,1196	0,6649
AC	0,5262	0,0999	0,7414	0,5104	0,1762	0,1210	0,6698
RN	0,9317	0,0000	0,0000	1,0000	0,0000	0,1205	0,6695
RNP	0,9317	0,0000	0,0000	1,0000	0,0000	0,1213	0,6702
RF	0,5269	0,1000	0,7401	0,5113	0,1761	0,1209	0,6697
XGBoost	0,9317	0,0000	0,0000	1,0000	0,0000	0,1209	0,6698
SVM Linear	0,8282	0,0686	0,2013	0,8602	0,1023	0,0568	0,5227

UNDERSAMPLING

UNDERSAMPLING

```
import numpy as np
import pandas as pd
from collections import Counter
```

```
assert 'df07_26' in globals(), "df07_26 não está definido no ambiente."
```

```
# novos nomes para não sobrescrever nada existente
FEATURE_COLS_NEW = ['dia_semana_correto','fase_dia_correto',
                    'tipo_pista_correto','visibilidade','uso_solo_correto']
TARGET_COL_NEW = 'obito'
```

```
# 1) remover linhas com NaN nas colunas de interesse (features + target)
df_clean_new = df07_26[FEATURE_COLS_NEW, TARGET_COL_NEW].copy().dropna()
```

```
# 2) normalizar target para binário (1 = 'Sim', 0 = 'Não')
positive_labels = {'sim','Sim','SIM','1','True','true','TRUE','S','s','yes','Yes','YES'}
df_clean_new['y_new'] = df_clean_new[TARGET_COL_NEW].astype(str).apply(lambda x: 1
if x in positive_labels else 0)
```

```
# contar antes
cnt_before = Counter(df_clean_new['y_new'])
print("Before:", cnt_before)
```

```
# definir proporção desejada para a classe positiva (40%)
desired_pos_frac = 0.40
```

```
n_pos = cnt_before.get(1, 0)
n_neg = cnt_before.get(0, 0)
if n_pos == 0:
    raise ValueError("Não há exemplos positivos ('Sim') após limpeza; não é possível ajustar
proporção.")
```

```
# calcular tamanho total desejado e número de negativos desejados
```

```

total_desired = int(round(n_pos / desired_pos_frac))
n_neg_desired = total_desired - n_pos
if n_neg_desired < 0:
    # já há mais positivos que a fração desejada; aqui optamos por não oversamplear
    positivos
    # ajustamos para manter todos os positivos e não reduzir negativos (apenas aviso)
    n_neg_desired = n_neg
    total_desired = n_pos + n_neg_desired

print(f"Target after resampling (approx): positives={n_pos}, negatives={n_neg_desired},
total={total_desired}")

# se a quantidade desejada de negativos for maior que a disponível, apenas mantém todos
if n_neg_desired >= n_neg:
    print("Quantidade negativa desejada >= existente; não será realizado undersampling
(mantém dataset limpo).")
    df_resampled_new = df_clean_new.copy()
else:
    # sample aleatório sem reposição da classe negativa para obter n_neg_desired
    df_pos = df_clean_new[df_clean_new['y_new'] == 1]
    df_neg = df_clean_new[df_clean_new['y_new'] == 0]
    df_neg_sampled = df_neg.sample(n=n_neg_desired, random_state=42, replace=False)
    # recombina e embaralha
    df_resampled_new = pd.concat([df_pos, df_neg_sampled],
ignore_index=True).sample(frac=1, random_state=42).reset_index(drop=True)

# contagens após
cnt_after = Counter(df_resampled_new['y_new'])
print("After:", cnt_after)
print("Resulting dataframe variable: df_resampled_new (rows =
{}).format(len(df_resampled_new)))

# df_resampled_new contém os dados limpos e com proporção aproximada 40% 'Sim' /
60% 'Não'
# Você pode salvar ou usar as colunas FEATURE_COLS_NEW e 'y_new' para treinar os
modelos.

```

Reajuste dos modelos para os dados reduzidos após o *undersampling* (≈ 20 min)

```

# Usa o dataset resultante do seu passo anterior: df_resampled_new (já sem NaN e com
~40% 'Sim')
# Se o dataframe tiver outro nome, substitua df_resampled_new abaixo.
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

```

```

from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix)
import numpy as np
import pandas as pd

# --- Verificações / entrada ---
assert 'df_resampled_new' in globals(), "df_resampled_new não encontrado. Use o
dataframe criado no passo anterior."
print("Total rows in resampled df:", len(df_resampled_new))
print("Class counts (resampled):", Counter(df_resampled_new['y_new']))

# Optionally ensure we use exactly 111562 lines if requested:
TARGET_N = 111562
if len(df_resampled_new) != TARGET_N:
    print(f"Warning: df_resampled_new has {len(df_resampled_new)} rows (target
{TARGET_N}). "
        "If you want exactly TARGET_N, slice or resample accordingly.")
    # exemplo para ajustar (apenas se quiser forçar):
    # df_resampled_use = df_resampled_new.sample(n=TARGET_N,
random_state=42).reset_index(drop=True)
    df_resampled_use = df_resampled_new.copy()
else:
    df_resampled_use = df_resampled_new.copy()

# renomear variáveis para não sobrescrever anteriores
FEATURE_COLS_FINAL = ['dia_semana_correto', 'fase_dia_correto',
                      'tipo_pista_correto', 'visibilidade', 'uso_solo_correto']
TARGET_COL_FINAL = 'y_new'

X_all = df_resampled_use[FEATURE_COLS_FINAL]
y_all = df_resampled_use[TARGET_COL_FINAL].values

# --- Holdout 80/20 estratificado (aplicado SOMENTE ao dataset resampled) ---
X_train_rs, X_test_rs, y_train_rs, y_test_rs = train_test_split(
    X_all, y_all, test_size=0.20, stratify=y_all, random_state=42
)
print("Train shape:", X_train_rs.shape, "Test shape:", X_test_rs.shape)
print("Train counts:", Counter(y_train_rs), "Test counts:", Counter(y_test_rs))

# --- Preprocessor (OneHot) compatível com versões do sklearn ---
try:
    ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    ohe = OneHotEncoder(handle_unknown='ignore', sparse=False)

preproc_final = ColumnTransformer([
    ('cat', Pipeline([('imp', SimpleImputer(strategy='constant', fill_value='__MISSING__')),
                      ('ohe', ohe)]), FEATURE_COLS_FINAL)
], remainder='drop')

```

```

# Transformar treino/teste (OHE) e guardar arrays para modelos que precisam
X_train_ohe_rs = preproc_final.fit_transform(X_train_rs)
X_test_ohe_rs = preproc_final.transform(X_test_rs)

# --- Classificadores (sem novo undersampling; dataset já ajustado) ---
classifiers_final = {
    'Logistic': LogisticRegression(solver='lbfgs', max_iter=1000),
    'DecisionTree': DecisionTreeClassifier(random_state=42),
    'RandomForest': RandomForestClassifier(n_estimators=200, random_state=42),
    'XGBoost': XGBClassifier(use_label_encoder=False, eval_metric='logloss',
random_state=42),
    'SVM_Linear': SVC(kernel='linear', probability=True, random_state=42),
    'NN_shallow': MLPClassifier(hidden_layer_sizes=(64,), max_iter=500, random_state=42),
    'NN_deep': MLPClassifier(hidden_layer_sizes=(256,128,64), early_stopping=True,
max_iter=1000, random_state=42)
}

models_final = {}
results_final = {}

for name, clf in classifiers_final.items():
    # montar pipeline: preproc_final já aplicado manualmente para NN/SVM; para
    uniformidade, use pipeline with only clf
    # vamos treinar usando as OHE matrices (X_train_ohe_rs / X_test_ohe_rs)
    # Alguns estimadores (DecisionTree, RandomForest, XGBoost) aceitam arrays
    diretamente
    print("Training", name)
    clf.fit(X_train_ohe_rs, y_train_rs)
    models_final[name] = clf

    # obter probabilidades de forma robusta
    try:
        proba = clf.predict_proba(X_test_ohe_rs)[: , 1]
    except Exception:
        try:
            df_score = clf.decision_function(X_test_ohe_rs)
            proba = 1 / (1 + np.exp(-df_score))
        except Exception:
            proba = clf.predict(X_test_ohe_rs).astype(float)

    yhat = (proba >= 0.5).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test_rs, yhat).ravel()
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0

    metrics = {
        'accuracy': accuracy_score(y_test_rs, yhat),
        'precision': precision_score(y_test_rs, yhat, zero_division=0),
        'sensitivity': recall_score(y_test_rs, yhat, zero_division=0),
        'specificity': specificity,
        'f1': f1_score(y_test_rs, yhat, zero_division=0),
        'auc_roc': roc_auc_score(y_test_rs, proba),
        'auprc': average_precision_score(y_test_rs, proba)
    }

```

```

    }

    results_final[name] = metrics
    print(f'{name}: { {k: round(v,4) for k,v in metrics.items()} }')

# models_final contém os classificadores treinados (usando OHE)
# results_final contém as métricas no holdout 20%

import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc, precision_recall_curve,
average_precision_score
import numpy as np

# fonte/tamanhos
title_fs = 18
label_fs = 18
legend_fs = 16
tick_fs = 16

def get_proba_for_estimator(est, X_raw=None, X_ohe=None):
    """
    Retorna probabilidades para estimador/pipeline.
    - Se for pipeline com named_steps, tenta predict_proba(X_raw) etc.
    - Se for estimator que espera OHE arrays, usa X_ohe quando fornecido.
    """
    # 1) tentativa direta com X_raw
    if X_raw is not None:
        try:
            return est.predict_proba(X_raw)[:, 1]
        except Exception:
            pass
        try:
            df = est.decision_function(X_raw)
            return 1 / (1 + np.exp(-np.asarray(df)))
        except Exception:
            pass
        try:
            return est.predict(X_raw).astype(float)
        except Exception:
            pass

    # 2) tentar com X_ohe
    if X_ohe is not None:
        try:
            return est.predict_proba(X_ohe)[:, 1]
        except Exception:
            pass
        try:
            df = est.decision_function(X_ohe)
            return 1 / (1 + np.exp(-np.asarray(df)))
        except Exception:
            pass

```

```

try:
    return est.predict(X_ohe).astype(float)
except Exception:
    pass

# 3) se est for pipeline com named_steps, tentar transformar internamente
try:
    if hasattr(est, "named_steps"):
        try:
            return est.predict_proba(X_raw)[:, 1]
        except Exception:
            pass
except Exception:
    pass

raise RuntimeError("Could not obtain probabilities for estimator/pipeline")

# Usa models_final, X_test_ohe_rs e y_test_rs (criadas no seu script anterior)
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
axes[0].plot([0, 1], [0, 1], 'k--', alpha=0.4)
colors = plt.cm.tab10

i = 0
for name, est in models_final.items():
    try:
        proba = get_proba_for_estimator(est, X_raw=None, X_ohe=X_test_ohe_rs)
        proba = np.asarray(proba)

        print(f"{name}:    proba    shape={proba.shape},    min={proba.min():.4f},
max={proba.max():.4f}")

        if np.all(proba == proba[0]):
            print(f" -> {name} outputs constant score; skipping.")
            continue

        fpr, tpr, _ = roc_curve(y_test_rs, proba)
        roc_auc = auc(fpr, tpr)
        axes[0].plot(fpr, tpr, label=f"{name} (AUC={roc_auc:.3f})", color=colors(i % 10),
linewidth=2)

        recall, precision, _ = precision_recall_curve(y_test_rs, proba)
        ap = average_precision_score(y_test_rs, proba)
        axes[1].plot(recall, precision, label=f"{name} (AUPRC={ap:.3f})", color=colors(i % 10),
linewidth=2)

        i += 1
    except Exception as e:
        print(f"Skipped {name}: {e}")
        continue

# formatação e fontes
axes[0].set_xlabel("False Positive Rate", fontsize=label_fs)

```

```

axes[0].set_ylabel("True Positive Rate", fontsize=label_fs)
axes[0].set_title("ROC Curves", fontsize=title_fs)
axes[0].legend(loc="lower right", fontsize=legend_fs)
axes[0].tick_params(axis='both', which='major', labelsize=tick_fs)
axes[0].grid(alpha=0.3)

```

```

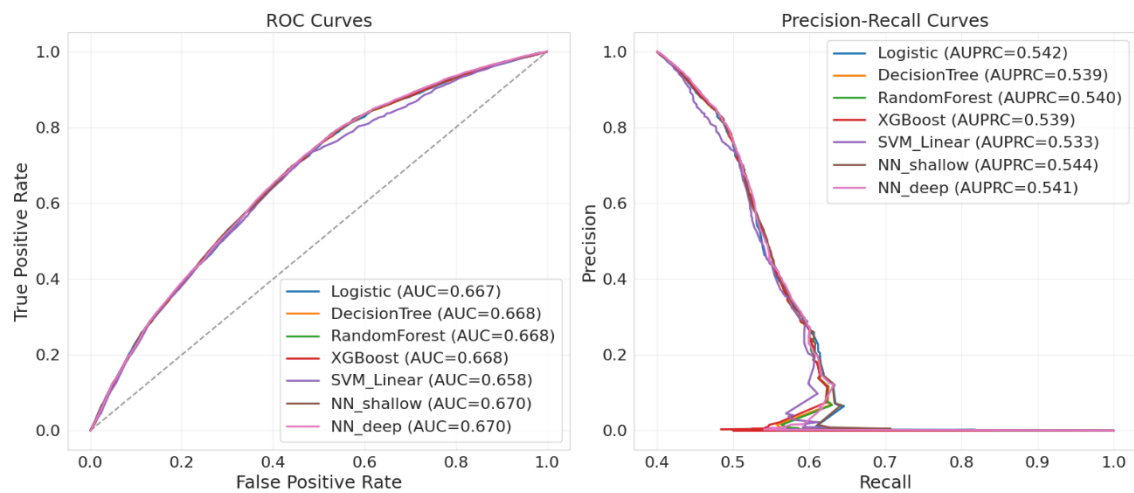
axes[1].set_xlabel("Recall", fontsize=label_fs)
axes[1].set_ylabel("Precision", fontsize=label_fs)
axes[1].set_title("Precision-Recall Curves", fontsize=title_fs)
axes[1].legend(loc="upper right", fontsize=legend_fs)
axes[1].tick_params(axis='both', which='major', labelsize=tick_fs)
axes[1].grid(alpha=0.3)

```

```

plt.tight_layout()
plt.show()

```



Análises dos modelos após o UNDERSAMPLING

Os resultados da qualidade dos modelos aplicados aos dados sob undersampling são apresentados na tabela 2.

Tabela 2: Medidas de qualidade para os modelos avaliados sob undersampling

Model	Acurácia	Precisão	Sensib.	Especif.	F1	AUPRC	AUC
RL	0,6371	0,5866	0,3139	0,8525	0,4090	0,5424	0,6665
AC	0,6351	0,5703	0,3552	0,8216	0,4378	0,5395	0,6675
RN	0,6363	0,5740	0,3516	0,8260	0,4361	0,5436	0,6699
RNP	0,6369	0,5763	0,3487	0,8291	0,4345	0,6701	0,6701
RF	0,6351	0,5706	0,3547	0,8220	0,4375	0,5401	0,6679
XGBoost	0,6351	0,5706	0,3545	0,8222	0,4373	0,5390	0,6677
SVM Linear	0,6363	0,5946	0,2848	0,8706	0,3852	0,5328	0,6584

O desempenho geral de todos os modelos tem acurácia ~0.63–0.64 e AUC ~0.666–0.670, indicando uma separação preditiva modesta. Quanto à relação entre precisão e sensibilidade, há trade-off claro. SVM e RL apresentam maior precisão (SVM 0.5946 e RL 0.5866) e alta especificidade (SVM 0.8706, RL 0.8525). Isto é, com esses modelos reduzimos falsos positivos, mas perdemos muitos positivos (sensibilidade baixa: SVM 0.2848, RL 0.3139).

Os modelos mais sensíveis são RN / RNP / AC / RF, pois têm sensibilidade (RN 0.3516, RNP 0.3487; AC 0.3552) e capturam mais positivos que SVM, com alguma queda de precisão. Já em relação às métricas compostas, o F1 é melhor em AC (0.4378) e muito próximas entre RF e XGBoost. AUPRC e AUC máximos são com RNP (AUPRC=0.6701, AUC=0.6701), indicando melhor desempenho em ranking/probabilidades.

Dessa forma, de acordo com objetivo geral (melhor equilíbrio e ranking), a escolha mais plausível seria a **RNP** (rede neural profunda), pois apresenta melhor AUC e AUPRC (melhor para ordenação de probabilidades e problemas desbalanceados) e sensibilidade e precisão intermediárias. Por outra ótica, se assumirmos prioridade de minimizar falsos positivos (priorizar precisão/especificidade), a opção escolher **SVM Linear** ou **Regressão Logística** e ajustar threshold para manter precisão desejada. Se prioridade for capturar o máximo de positivos (priorizar sensibilidade/recall), escolheríamos **AC** ou **RF**.

Após uma análise dos modelos, focaremos na em um ajuste mais detalhado da **rede neural profunda (RNP)**, que consiste em, sob os dados que realizamos o undersampling, fazer tuning dos hiperparâmetros, com threshold tuning e calibração.

Rede Neural Profunda aprimorada (> 80 min de execução)

Devido ao tempo de execução, o código foi colocado na forma de texto para não prejudicar a avaliação do docente. Em seguida, os resultados foram colocados na forma de tabelas ou gráficos. De qualquer, se houver interesse em executar o código, basta copiar o código, colar e executar. Porém, a demanda de tempo não é recomendada, visto que a demanda foi alta, mas houve um ganho significativo em termos de previsão, como mostrado a seguir. Conto com vossa compreensão, dada a importância na obtenção dos resultados. Mas como tenho ciência da quantidade de MVP's a serem avaliados, tentei poupá-lo do alto tempo de execução.

```
# Tuning + calibração + threshold tuning para Rede Neural profunda (MLP) usando os dados undersampled
```

```
# X_train_ohe_rs, y_train_rs, X_test_ohe_rs, y_test_rs
```

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.model_selection import StratifiedKFold, RandomizedSearchCV
```

```
from sklearn.calibration import CalibratedClassifierCV
```

```
from sklearn.metrics import precision_recall_curve, f1_score, average_precision_score, roc_auc_score
```

```
import numpy as np
```

```
import joblib
```

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
# 1) Definir estimador base (deep MLP)
```

```

base = MLPClassifier(random_state=42, max_iter=1000)

# 2) Espaço de hiperparâmetros (Randomized)
param_dist = {
    'hidden_layer_sizes': [(256,128,64), (128,64,32), (512,256,128), (256,128)],
    'activation': ['relu', 'tanh'],
    'alpha': [1e-5, 1e-4, 1e-3, 1e-2],      # L2 regularization
    'learning_rate_init': [1e-4, 1e-3, 1e-2],
    'solver': ['adam'],                     # usar 'adam' para redes profundas
    'early_stopping': [True],
    'n_iter_no_change': [10, 20]
}

cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=42)
rs = RandomizedSearchCV(base, param_dist, n_iter=20, scoring='average_precision',
                        cv=cv, random_state=42, n_jobs=-1, verbose=1)

# 3) Rodar busca
rs.fit(X_train_ohe_rs, y_train_rs)
print("Best params:", rs.best_params_)
print("Best AUPRC (CV):", rs.best_score_)

best_clf = rs.best_estimator_

# 4) Calibração (Platt / sigmoid) com cross-validation interno
calibrated = CalibratedClassifierCV(best_clf, cv=cv, method='sigmoid')
calibrated.fit(X_train_ohe_rs, y_train_rs)

# opcional: salvar modelo calibrado
# joblib.dump(calibrated, "mlp_deep_calibrated.joblib")

# 5) Obter probabilidades no set de teste e avaliar AUC/AUPRC
proba = calibrated.predict_proba(X_test_ohe_rs)[: , 1]
auprc = average_precision_score(y_test_rs, proba)
aucroc = roc_auc_score(y_test_rs, proba)
print(f"AUPRC (test): {auprc:.4f}, AUC-ROC (test): {aucroc:.4f}")

# 6) Threshold tuning — escolher threshold que maximiza F1 ou outro trade-off
precision, recall, thresholds = precision_recall_curve(y_test_rs, proba)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-12) # evita div por 0
best_idx = np.nanargmax(f1_scores)
best_threshold = thresholds[best_idx] if best_idx < len(thresholds) else 0.5
best_f1 = f1_scores[best_idx]
print(f"Best threshold by F1: {best_threshold:.4f}, F1: {best_f1:.4f}")

# 7) Aplicar threshold e mostrar métricas finais
yhat_thresh = (proba >= best_threshold).astype(int)
final_f1 = f1_score(y_test_rs, yhat_thresh)
final_auprc = average_precision_score(y_test_rs, proba)
final_aucroc = roc_auc_score(y_test_rs, proba)
print(f"Final metrics (threshold {best_threshold:.4f}): F1={final_f1:.4f},
AUPRC={final_auprc:.4f}, AUC={final_aucroc:.4f}")

```

```
# 8) Imprimir confusion matrix / precision/recall
from sklearn.metrics import confusion_matrix, precision_score, recall_score
tn, fp, fn, tp = confusion_matrix(y_test_rs, yhat_thresh).ravel()
print("confusion matrix: TN,FP,FN,TP =", (tn, fp, fn, tp))
print("precision:", precision_score(y_test_rs, yhat_thresh))
print("recall:", recall_score(y_test_rs, yhat_thresh))
```

Modelo final: 'calibrated' + use 'best_threshold' for decision rule

```
# Acurácia e especificidade
proba = calibrated.predict_proba(X_test_ohe_rs)[: , 1]
yhat_thresh = (proba >= best_threshold).astype(int)
```

```
acc = accuracy_score(y_test_rs, yhat_thresh)
tn, fp, fn, tp = confusion_matrix(y_test_rs, yhat_thresh).ravel()
specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
```

```
print(f"Accuracy: {acc:.4f}")
print(f"Specificity: {specificity:.4f}")
```

The screenshot shows a Jupyter Notebook window titled 'MVP - Machine Learning & Analytics.ipynb'. The code is as follows:

```
[48]: # 5) Obter probabilidades no set de teste e avaliar AUC/AUPRC
proba = calibrated.predict_proba(X_test_ohe_rs)[: , 1]
auprc = average_precision_score(y_test_rs, proba)
aucroc = roc_auc_score(y_test_rs, proba)
print(f"AUPRC (test): {auprc:.4f}, AUC-ROC (test): {aucroc:.4f}")

# 6) Threshold tuning - escolher threshold que maximiza F1 ou outro trade-off
precision, recall, thresholds = precision_recall_curve(y_test_rs, proba)
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-12) # evita div por 0
best_idx = np.nanargmax(f1_scores)
best_threshold = thresholds[best_idx] if best_idx < len(thresholds) else 0.5
best_f1 = f1_scores[best_idx]
print(f"Best threshold by F1: (best_threshold:.4f), F1: (best_f1:.4f)")

# 7) Aplicar threshold e mostrar métricas finais
yhat_thresh = (proba >= best_threshold).astype(int)
final_f1 = f1_score(y_test_rs, yhat_thresh)
final_auprc = average_precision_score(y_test_rs, proba)
final_aucroc = roc_auc_score(y_test_rs, proba)
print(f"Final metrics (threshold {best_threshold:.4f}): F1={final_f1:.4f}, AUPRC={final_auprc:.4f}, AUC={final_aucroc:.4f}")

# 8) Imprimir confusion matrix / precision/recall
from sklearn.metrics import confusion_matrix, precision_score, recall_score
tn, fp, fn, tp = confusion_matrix(y_test_rs, yhat_thresh).ravel()
print("confusion matrix: TN,FP,FN,TP =", (tn, fp, fn, tp))
print("precision:", precision_score(y_test_rs, yhat_thresh))
print("recall:", recall_score(y_test_rs, yhat_thresh))

# Modelo final: 'calibrated' + use 'best_threshold' for decision rule
```

The interface includes a top bar with 'Arquivo', 'Editar', 'Ver', 'Inserir', 'Ambiente de execução', 'Ferramentas', and 'Ajuda'. Below the code editor is a terminal window showing 'ariáveis' and 'Terminal'. The status bar at the bottom indicates 'Em execução (1h4m58s)' and 'Python 3'.

Código completo para:

- desenhar arquitetura (diagramas) das duas MLPs (antes e depois do tuning)

#

Entrada esperada no ambiente:

- best_clf (o RNP antes da calibração? ou o melhor da busca) -> aqui chamaremos rnp_tuned (calibrado = `calibrated`)

- rnp_before (a RNP anterior não-aprimorada). Se você não tiver uma variável com isso, troque por models_final['RNP'] ou similar.

- preproc_final, X_train_ohe_rs, X_test_ohe_rs, X_train_rs, X_test_rs, y_train_rs, y_test_rs (do seu fluxo anterior)

#

Saída:

- 2 figuras da arquitetura (cada MLP)

```

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.inspection import permutation_importance
from sklearn.utils import Bunch
import networkx as nx
import math
import warnings
warnings.filterwarnings("ignore")

# ----- 1) Preparar referências aos modelos -----
# Ajuste os nomes abaixo conforme suas variáveis do notebook:
# rnp_before: a RNP antiga (não-tuned) — ex: models_final.get('RNP') ou um estimador salvo
# rnp_tuned: RNP aprimorada. No seu fluxo anterior 'best_clf' é o melhor MLP e 'calibrated' é o calibrado.
rnp_tuned = globals().get('calibrated', None)    # calibrado (CalibratedClassifierCV em volta do best_clf)
rnp_raw_tuned = globals().get('best_clf', None)  # melhor MLP antes da calibração
rnp_before = globals().get('rnp_before', None)   # substitua se tiver o antigo

# fallback: se não existir variável 'rnp_before', tente models_final['NN_deep'] ou similar
if rnp_before is None:
    rnp_before = globals().get('models_final', {}).get('NN_deep', None)

# garantir pelo menos um estimador disponível
if rnp_raw_tuned is None and rnp_tuned is not None:
    # extrair estimador interno do calibrated se for CalibratedClassifierCV
    try:
        rnp_raw_tuned = rnp_tuned.base_estimator
    except Exception:
        rnp_raw_tuned = None

if rnp_raw_tuned is None and rnp_before is None:
    raise RuntimeError("Não encontrei os estimadores RNP. Defina rnp_before e/ou best_clf/calibrated no ambiente.")

# Usaremos X_test_ohe_rs (matriz OHE) e nomes das colunas (se disponíveis)
Xohe_train = globals().get('X_train_ohe_rs', None)
Xohe_test = globals().get('X_test_ohe_rs', None)
ohe_feature_names = globals().get('ohe_cols', None)

# Se o preprocessor estiver disponível, extrair nomes a partir do OHE transformer
if ohe_feature_names is None and 'preproc_final' in globals():
    try:
        ohe_feature_names = preproc_final.named_transformers_['cat'].named_steps['ohe'].get_feature_names_out(FEATURE_COLS_FINAL)
    except Exception:
        ohe_feature_names = None

```

```

# Se não houver Xohe disponíveis, transforme X_test_rs com preproc_final
if Xohe_test is None and 'preproc_final' in globals() and 'X_test_rs' in globals():
    Xohe_test = preproc_final.transform(X_test_rs)
    Xohe_train = preproc_final.transform(X_train_rs)
    try:
        ohe_feature_names =
preproc_final.named_transformers_['cat'].named_steps['ohe'].get_feature_names_out(FE
ATURE_COLS_FINAL)
    except Exception:
        ohe_feature_names = None

# converter matrizes para DataFrame para facilitar interpretação (se temos nomes)
if ohe_feature_names is not None and Xohe_test is not None:
    Xohe_test_df = pd.DataFrame(Xohe_test, columns=ohe_feature_names)
    Xohe_train_df = pd.DataFrame(Xohe_train, columns=ohe_feature_names) if Xohe_train is
not None else None
else:
    # criar nomes genéricos
    if Xohe_test is not None:
        ncol = Xohe_test.shape[1]
        cols = [f"x{i}" for i in range(ncol)]
        Xohe_test_df = pd.DataFrame(Xohe_test, columns=cols)
        Xohe_train_df = pd.DataFrame(Xohe_train, columns=cols) if Xohe_train is not None else
None
        ohe_feature_names = cols
    else:
        raise RuntimeError("Não consegui obter X_test_ohe_rs nem X_train_ohe_rs. Execute o
preprocessador antes.")

# ----- Função para plotar arquitetura MLP -----
def plot_mlp_architecture(mlp, title="MLP architecture", ax=None):
    """
    Desenha um diagrama simples das camadas e nós de um MLPClassifier.
    mlp: estimator (MLPClassifier) ou object with attribute hidden_layer_sizes
    """

    # obter camadas
    if hasattr(mlp, "hidden_layer_sizes"):
        hidden = mlp.hidden_layer_sizes
    elif hasattr(mlp, "coefs_"):
        hidden = tuple([w.shape[1] for w in mlp.coefs_[:-1]])
    else:
        hidden = getattr(mlp, "hidden_layer_sizes", (100,))
    if isinstance(hidden, int):
        hidden = (hidden,)
    # input size (usar número de features OHE)
    n_in = Xohe_test_df.shape[1]
    layers = [n_in] + list(hidden) + [1] # saída única (prob)
    # montar grafo
    G = nx.DiGraph()
    pos = {}
    y_gap = 1.0

```

```

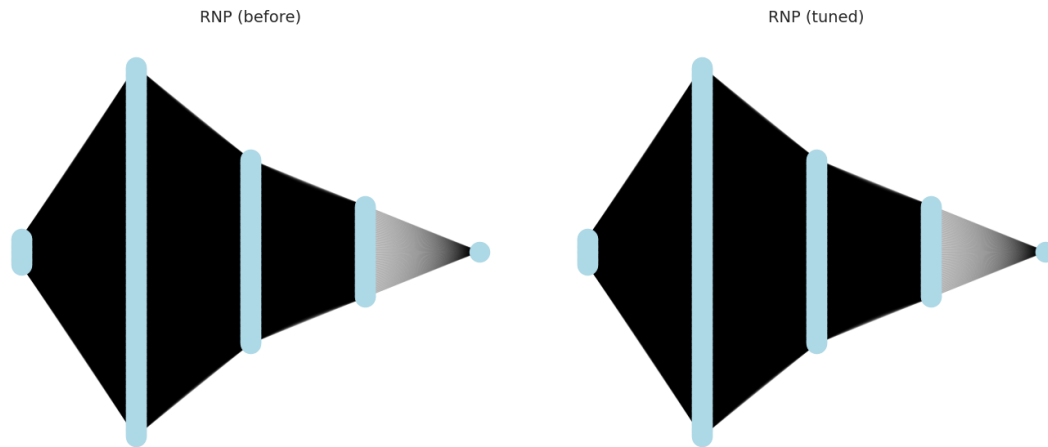
x_gap = 2.0
max_nodes = max(layers)
for li, n_nodes in enumerate(layers):
    x = li * x_gap
    y0 = (max_nodes - n_nodes) / 2.0
    for ni in range(n_nodes):
        node = f"L{li}_N{ni}"
        G.add_node(node)
        pos[node] = (x, y0 + ni * y_gap)
        # conectar a camada anterior
        if li > 0:
            for pi in range(layers[li-1]):
                G.add_edge(f"L{li-1}_N{pi}", node)
# plot
if ax is None:
    fig, ax = plt.subplots(figsize=(8, 6))
ax.set_title(title, fontsize=14)
nx.draw_networkx_nodes(G, pos, node_size=300, node_color='lightblue', ax=ax)
nx.draw_networkx_edges(G, pos, arrowstyle='-', arrows=False, alpha=0.3, ax=ax)
ax.axis('off')

# ----- 2) Plotar arquiteturas para os dois modelos -----
fig = plt.figure(figsize=(14,6))
ax1 = fig.add_subplot(1,2,1)
try:
    plot_mlp_architecture(rnp_before if rnp_before is not None else rnp_raw_tuned,
                          title="RNP (before)", ax=ax1)
except Exception as e:
    ax1.text(0.5,0.5,f"Could not plot before RNP: {e}", ha='center')

ax2 = fig.add_subplot(1,2,2)
try:
    # se rnp_raw_tuned existe, desenhar arquitetura dele; senão rnp_tuned.base_estimator
    plot_mlp_architecture(rnp_raw_tuned if rnp_raw_tuned is not None else rnp_before,
                          title="RNP (tuned)", ax=ax2)
except Exception as e:
    ax2.text(0.5,0.5,f"Could not plot tuned RNP: {e}", ha='center')

plt.tight_layout()
plt.show()

```



```
import matplotlib.pyplot as plt
import numpy as np

def plot_or_bar_colored(df_or, title="Odds ratios", top_n=20, ax=None):
    dfp = df_or.copy()
    # usar coef (log-odds) para ordenar por magnitude e decidir cor
    dfp['mag'] = np.abs(dfp['coef'])
    dfp = dfp.sort_values('mag', ascending=False).head(top_n).iloc[::-1]

    ors = dfp['or'].values
    features = dfp['feature'].values
    ci_low = dfp['ci_low'].values
    ci_high = dfp['ci_high'].values
    coefs = dfp['coef'].values

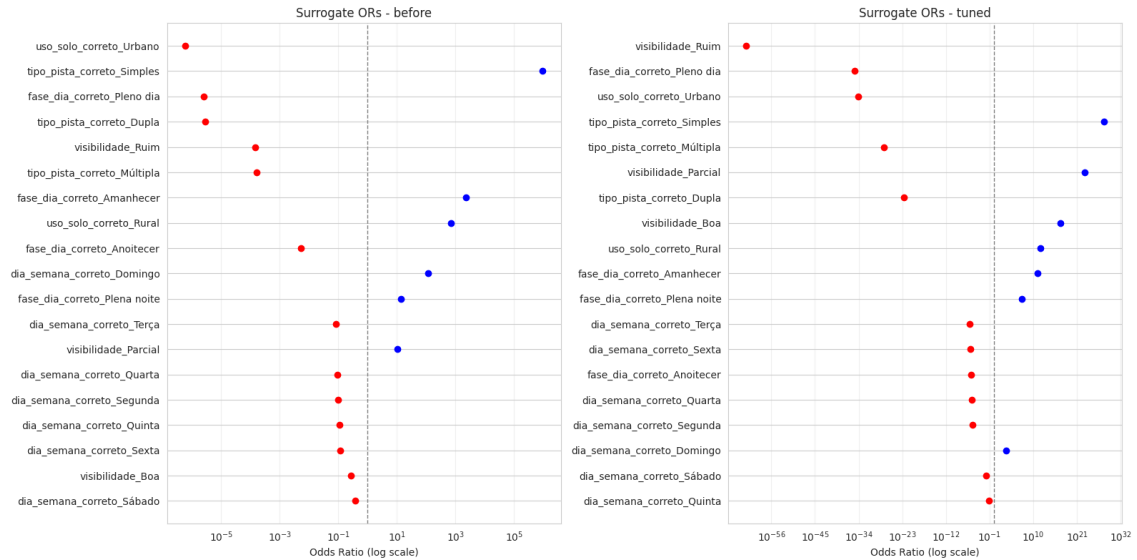
    # cores: azul se coef>0 (log-odds positivo -> aumenta prob), vermelho se coef<0
    colors = ['blue' if c > 0 else 'red' for c in coefs]

    if ax is None:
        fig, ax = plt.subplots(figsize=(8, max(6, 0.3 * len(features))))
    # plot horizontal errorbars
    for xi, feat, lo, hi, col in zip(ors, features, ci_low, ci_high, colors):
        ax.errorbar(x=xi, y=feat, xerr=[[xi - lo], [hi - xi]], fmt='o', color=col, ecolor=col, capsize=3)

    ax.set_xscale('log')
    ax.set_xlabel('Odds Ratio (log scale)')
    ax.set_title(title)
    ax.axvline(1.0, color='gray', linestyle='--', linewidth=1) # OR=1 reference
    ax.grid(True, axis='x', alpha=0.3)
    return ax

# Exemplo de uso (substituir surrogate_results pelos seus resultados já calculados):
fig, axes = plt.subplots(1, 2, figsize=(16, 8))
labs = list(surrogate_results.keys())
for i, lab in enumerate(labs[:2]):
    ax = plot_or_bar_colored(surrogate_results[lab], title=f"Surrogate ORs - {lab}", top_n=20,
    ax=axes[i])
plt.tight_layout()
```

plt.show()



import pandas as pd

localizar os dois estimadores

est_before = globals().get('rnp_before', None)

est_after = globals().get('best_clf', None)

se só existir calibrado, pegar base_estimator

if est_after is None and 'calibrated' in globals():

try:

est_after = calibrated.base_estimator

except Exception:

est_after = None

if est_before is None or est_after is None:

raise RuntimeError("Estimadores não encontrados. Certifique-se de que 'rnp_before' e 'best_clf' ou 'calibrated' existem no ambiente.")

obter dicionários de parâmetros

params_before = est_before.get_params()

params_after = est_after.get_params()

unificar chaves

all_keys = sorted(set(list(params_before.keys()) + list(params_after.keys())))

construir DataFrame comparativo

rows = []

for k in all_keys:

rows.append({

'param': k,

'before': params_before.get(k, None),

'after' : params_after.get(k, None)

})

df_params_compare = pd.DataFrame(rows).set_index('param')

```
# opcional: mostrar apenas parâmetros relevantes (ex.: hidden_layer_sizes, activation,
alpha, solver, learning_rate_init, max_iter, early_stopping, n_iter_no_change)
relevant =
['hidden_layer_sizes','activation','alpha','solver','learning_rate_init','max_iter','early_stopping',
n_iter_no_change','batch_size','learning_rate']
df_relevant = df_params_compare.loc[[p for p in relevant if p in df_params_compare.index]]
```

```
# exibir tabela completa e a tabela resumida
print("Full parameters comparison:")
display(df_params_compare) # em notebook fica mais apresentável
print("\nRelevant params:")
display(df_relevant)
```

	before	after
param		
hidden_layer_sizes	(256, 128, 64)	(256, 128, 64)
activation	relu	tanh
alpha	0.0001	0.001
solver	adam	adam
learning_rate_init	0.001	0.0001
max_iter	1000	1000
early_stopping	True	True
n_iter_no_change	10	20
batch_size	auto	auto
learning_rate	constant	constant

```
import numpy as np
import pandas as pd
```

```
# 1) carregar tabela or_logistic (já calculada no passo anterior) ou recalculá-la se necessário
# assumimos or_logistic tem colunas: feature, coef, OR, CI_lower, CI_upper
# se não existir, rode o bloco de logistic anterior; aqui assumimos disponível.
```

```
# 2) Calcular OR empírico para NN_deep por coluna OHE, comparando com baseline
(primeira categoria do mesmo variável)
nn = models_final['NN_deep']
X_test_nn = X_test_ohe_rs.copy()
```

```
# extrair categorias e índices por variável do OneHotEncoder
ohe = preproc_final.named_transformers_['cat'].named_steps['ohe']
cat_lists = ohe.categories_
indices_by_var = {}
start = 0
for col, cats in zip(FEATURE_COLS_FINAL, cat_lists):
    n = len(cats)
    indices_by_var[col] = list(range(start, start + n))
    start += n
```

```
# função para prever probs de forma robusta
def predict_proba_safe(model, X):
    try:
```

```

    return model.predict_proba(X)[: ,1]
except Exception:
    try:
        df = model.decision_function(X)
        return 1/(1+np.exp(-df))
    except Exception:
        return model.predict(X).astype(float)

```

calcular OR empírico por categoria e mapear para nome de feature OHE (col__category)

```
nn_rows = []
```

```
for col in FEATURE_COLS_FINAL:
```

```
    idxs = indices_by_var[col]
```

```
    cats = ohe.categories_[FEATURE_COLS_FINAL.index(col)]
```

```
    baseline_idx = idxs[0] # baseline = primeira categoria
```

```
    for j, cat in enumerate(cats):
```

```
        col_idx = idxs[j]
```

```
        # construir X1 (essa categoria ativa) e X0 (baseline ativa)
```

```
        X1 = X_test_nn.copy()
```

```
        X0 = X_test_nn.copy()
```

```
        X1[:, idxs] = 0
```

```
        X0[:, idxs] = 0
```

```
        X1[:, col_idx] = 1
```

```
        X0[:, baseline_idx] = 1
```

```
        p1 = predict_proba_safe(nn, X1)
```

```
        p0 = predict_proba_safe(nn, X0)
```

```
        odds1 = np.clip(p1, 1e-9, 1-1e-9) / (1 - np.clip(p1, 1e-9, 1-1e-9))
```

```
        odds0 = np.clip(p0, 1e-9, 1-1e-9) / (1 - np.clip(p0, 1e-9, 1-1e-9))
```

```
        mean_odds1 = odds1.mean()
```

```
        mean_odds0 = odds0.mean()
```

```
        or_emp = mean_odds1 / mean_odds0 if mean_odds0 > 0 else np.nan
```

```
        feat_name = f"{col}__{cat}"
```

```
        nn_rows.append({
```

```
            'feature': feat_name,
```

```
            'NN_empirical_OR': or_emp,
```

```
            'NN_mean_odds_cat': mean_odds1,
```

```
            'NN_mean_odds_baseline': mean_odds0,
```

```
            'variable': col,
```

```
            'category': cat
```

```
        })
```

```
or_nn_empirical = pd.DataFrame(nn_rows)
```

3) Preparar tabela da logística alinhada com nomes OHE

assumir or_logistic existente; se não existir, substitua leitura com recalculo.

padronizar nome de feature em or_logistic para coincidir com formato "col__cat"

```
or_log = or_logistic.copy()
```

some implementations may have 'const' row; remove const for merge

```
or_log = or_log[or_log['feature'] != 'const'].copy()
```

garantir que nomes batam: or_log.feature já foi construído como feature_names_ohe (col__cat)

```

# unir
merged = pd.merge(or_log[['feature','OR','CI_lower','CI_upper']],
                  or_nn_empirical[['feature','NN_empirical_OR','NN_mean_odds_cat','NN_mean_odds_baseline']],
                  on='feature', how='outer')

# ordenar por variável então categoria (extrair variable from feature)
merged['variable'] = merged['feature'].apply(lambda x: x.split('__')[0] if isinstance(x, str) and
'__' in x else "")
merged['category'] = merged['feature'].apply(lambda x: x.split('__')[1] if isinstance(x, str)
and '__' in x else "")
merged = merged.sort_values(['variable','category']).reset_index(drop=True)

print("Tabela combinada (Logistic OR vs NN_deep empirical OR):")
print(merged.head(200)) # mostrar primeiras linhas; ajuste se quiser ver todas

# salvar CSV
merged.to_csv('or_logistic_vs_nn_deep.csv', index=False)
print("Salvo em or_logistic_vs_nn_deep.csv")

# gerar HTML mais organizado da tabela merged
from IPython.display import display, HTML

# formatação numérica
merged_display = merged.copy()
for col in ['OR','CI_lower','CI_upper','NN_empirical_OR','NN_mean_odds_cat','NN_mean_odds_baseline']:
    if col in merged_display.columns:
        merged_display[col] = merged_display[col].map(lambda x: f"{x:.3f}" if pd.notnull(x) else
        "")

# CSS simples para estilo e destaque
css = """
<style>
table.custom-table { border-collapse: collapse; width: 100%; font-family: Arial, sans-serif;
}
table.custom-table th, table.custom-table td { border: 1px solid #ddd; padding: 6px 8px;
text-align: center; }
table.custom-table th { background-color: #f4f4f8; color: #333; font-weight: 600; }
tr:hover td { background-color: #fafafa; }
td.highlight-high { background-color: #ffe6e6; } /* OR empírico alto (NN) */
td.highlight-low { background-color: #e6f7ff; } /* OR empírico baixo (NN) */
.caption { font-family: Arial, sans-serif; margin-bottom: 8px; font-size: 14px; color: #333; }
</style>
"""

# regra simples de destaque: NN_empirical_OR > 2 -> high, < 0.5 -> low
def highlight_row(val, colname):
    try:
        v = float(val)
    except:

```

```

        return ""
    if colname == 'NN_empirical_OR':
        if v > 2:
            return 'highlight-high'
        if v < 0.5:
            return 'highlight-low'
    return ""

# construir HTML table com classes aplicadas
html_rows = []
# header
cols = merged_display.columns.tolist()
header = "<tr>" + "".join([f"<th>{c}</th>" for c in cols]) + "</tr>"
html_rows.append(header)
# body
for _, r in merged_display.iterrows():
    cells = []
    for c in cols:
        cell_val = r[c] if pd.notnull(r[c]) else ""
        cls = highlight_row(cell_val, c)
        cls_attr = f' class="{cls}"' if cls else ""
        cells.append(f"<td{cls_attr}>{cell_val}</td>")
    html_rows.append("<tr>" + "".join(cells) + "</tr>")

html_table = f'<div class="caption">Tabela: OR Logistic (95% CI) vs NN_deep empirical OR
— baseline = primeira categoria por variável</div>'
html_table += f'<table class="custom-table">{ "".join(html_rows) }</table>'

html_page = "<html><head><meta charset='utf-8'/>" + css + "</head><body>" + html_table
+ "</body></html>"

# salvar em arquivo
with open('or_logistic_vs_nn_deep.html', 'w', encoding='utf-8') as f:
    f.write(html_page)

# exibir inline se possível (notebook)
try:
    display(HTML(html_page))
except Exception:
    print("Arquivo salvo: or_logistic_vs_nn_deep.html")

```

feature	OR	CI_lower	CI_upper	NN_empirical_OR	NN_mean_odds_cat	NN_mean_odds_baseline	variable	category
dia_semana_correto__Domingo	1.122			1.000	0.826	0.826	dia_semana_correto	Domingo
dia_semana_correto__Quarta	0.901			0.809	0.668	0.826	dia_semana_correto	Quarta
dia_semana_correto__Quinta	0.932			0.839	0.693	0.826	dia_semana_correto	Quinta
dia_semana_correto__Segunda	0.953			0.865	0.715	0.826	dia_semana_correto	Segunda
dia_semana_correto__Sexta	0.935			0.870	0.718	0.826	dia_semana_correto	Sexta
dia_semana_correto__Sábado	1.038			0.951	0.785	0.826	dia_semana_correto	Sábado
dia_semana_correto__Terça	0.915			0.786	0.649	0.826	dia_semana_correto	Terça
fase_dia_correto__Amanhecer	1.296	0.000	inf	1.000	1.011	1.011	fase_dia_correto	Amanhecer
fase_dia_correto__Anoitecer	0.798	0.000	inf	0.686	0.694	1.011	fase_dia_correto	Anoitecer
fase_dia_correto__Plena noite	1.284	0.000	inf	0.962	0.973	1.011	fase_dia_correto	Plena noite
fase_dia_correto__Pleno dia	0.601	0.000	inf	0.507	0.513	1.011	fase_dia_correto	Pleno dia
tipo_pista_correto__Dupla	0.747	0.000	inf	1.000	0.463	0.463	tipo_pista_correto	Dupla
tipo_pista_correto__Múltipla	0.754	0.000	inf	0.907	0.420	0.463	tipo_pista_correto	Múltipla
tipo_pista_correto__Simples	1.418	0.000	inf	1.914	0.886	0.463	tipo_pista_correto	Simples
uso_solo_correto__Rural	1.185	0.000	inf	1.000	0.838	0.838	uso_solo_correto	Rural
uso_solo_correto__Urbano	0.674	0.000	inf	0.576	0.483	0.838	uso_solo_correto	Urbano
visibilidade__Boa	0.967	0.000	inf	1.000	0.752	0.752	visibilidade	Boa
visibilidade__Parcial	1.117	0.000	inf	1.123	0.845	0.752	visibilidade	Parcial
visibilidade__Ruim	0.738	0.000	inf	0.821	0.617	0.752	visibilidade	Ruim

```
# assumir or_logistic e or_nn_empirical já criados previamente
```

```
# criar cópias
```

```
or_log = or_logistic.copy()
```

```
or_log = or_log[or_log['feature'] != 'const'].copy()
```

```
nn_emp = or_nn_empirical.copy() # tem NN_mean_odds_cat e NN_mean_odds_baseline
```

```
# calcular medida normalizada 0-1
```

```
def nn_contrib_row(row):
```

```
    oc = row.get('NN_mean_odds_cat', np.nan)
```

```
    ob = row.get('NN_mean_odds_baseline', np.nan)
```

```
    if pd.isna(oc) or pd.isna(ob):
```

```
        return np.nan
```

```
    denom = oc + ob
```

```
    if denom == 0:
```

```
        return np.nan
```

```
    return float(oc / denom)
```

```
nn_emp['NN_contrib'] = nn_emp.apply(nn_contrib_row, axis=1)
```

```
# selecionar colunas desejadas
```

```
nn_sel = nn_emp[['feature','NN_contrib']].copy()
```

```
# unir com OR da regressão logística (manter OR e IC)
```

```
merged_simple = pd.merge(or_log[['feature','OR','CI_lower','CI_upper']], nn_sel,
on='feature', how='outer')
```

```
# ordenar e exibir
```

```
merged_simple['variable'] = merged_simple['feature'].apply(lambda x: x.split('__')[0] if
isinstance(x, str) and '__' in x else '')
```

```
merged_simple['category'] = merged_simple['feature'].apply(lambda x: x.split('__')[1] if
isinstance(x, str) and '__' in x else '')
```

```
merged_simple
```

```
=
```

```
merged_simple.sort_values(['variable','category']).reset_index(drop=True)
```

```
# formato de exibição (opcional)
```

```
print(merged_simple[['feature','OR','CI_lower','CI_upper','NN_contrib']].head(200))

# salvar
merged_simple.to_csv('or_and_nn_contrib.csv', index=False)
print("Salvo: or_and_nn_contrib.csv")
```

	feature	OR	CI_lower	CI_upper	NN_contrib
0	dia_semana_correto_Domingo	1.121862	NaN	NaN	0.500000
1	dia_semana_correto_Quarta	0.900620	NaN	NaN	0.447233
2	dia_semana_correto_Quinta	0.932434	NaN	NaN	0.456101
3	dia_semana_correto_Segunda	0.953392	NaN	NaN	0.463941
4	dia_semana_correto_Sexta	0.935489	NaN	NaN	0.465202
5	dia_semana_correto_Sábado	1.038280	NaN	NaN	0.487475
6	dia_semana_correto_Terça	0.914749	NaN	NaN	0.440139
7	fase_dia_correto_Amanhecer	1.296307	0.0	inf	0.500000
8	fase_dia_correto_Anoitecer	0.797567	0.0	inf	0.407014
9	fase_dia_correto_Plena noite	1.284213	0.0	inf	0.490428
10	fase_dia_correto_Pleno dia	0.601034	0.0	inf	0.336640
11	tipo_pista_correto_Dupla	0.746928	0.0	inf	0.500000
12	tipo_pista_correto_Múltipla	0.753635	0.0	inf	0.475734
13	tipo_pista_correto_Simples	1.417663	0.0	inf	0.656872
14	uso_solo_correto_Rural	1.184815	0.0	inf	0.500000
15	uso_solo_correto_Urbano	0.673517	0.0	inf	0.365404
16	visibilidade_Boa	0.967011	0.0	inf	0.500000
17	visibilidade_Parcial	1.117465	0.0	inf	0.528864
18	visibilidade_Ruim	0.738489	0.0	inf	0.450727

Discussão final e Hipóteses levantadas

Os resultados da qualidade do modelo de redes neurais profundas (RNP) aprimorado aplicado aos dados sob undersampling são apresentados na tabela 3. Comparando as duas redes, percebemos uma arquitetura muito similar, de modo que o tamanho das camadas ocultas é similar (256, 128, 64). A ativação **relu** ativa poucas unidades, facilita treino de redes profundas, enquanto a **tanh** é simétrica, pode saturar e tornar treinamento mais lento, que foi exatamente o que ocorreu, sendo até 2 vezes mais lenta.

A constante de regularização alpha saiu de 0.0001 para 0.001. O aumento da penalização, tende a reduzir overfitting, mas também pode causar underfitting se muito grande. A taxa de aprendizado foi de 0.001 para 0.0001 (passos menores), o que leva a um treino mais lento e pode melhorar a estabilidade, tendo o custo de requerer mais iterações para convergir. O número de iterações também aumentou de 10 para 20, evitando paradas prematuras, compatível com LR menor.

Tabela 3: Medidas de qualidade da RNP e da RNP aprimorada após o undersampling

Model	Acurácia	Precisão	Sensib.	Especif.	F1	AUPRC	AUC
RNP	0,5841	0,4248	0,8231	0,8291	0,6129	0,5487	0,6712
RNP	0,6369	0,5763	0,3487	0,8291	0,4345	0,6701	0,6701

Porém, apesar da alta demanda computacional, a versão inicial com a função de ativação relu, com alpha menor e lr maior obteve métricas de teste melhores porque teve treino mais rápido e alcançou representações mais fortes antes de a regularização/passos pequenos reduzirem a aprendizagem. Além disso, a mudança para tanh + alpha maior + lr menor pode ter levado a underfitting, que significa um modelo mais simples efetivo (pesos menores) e/ou não totalmente convergido dentro do máximo de iterações. De forma geral,

o ajuste tornou o modelo mais conservador (menos overfitting), mas possivelmente exagerou e reduziu a performance no conjunto de teste. Isto é, o tuned foi mais conservador (mais regularização, passos menores) e provavelmente subajustou. Dessa forma, o ajuste a ser considerado ainda foi a RNP inicial.

Dessa forma, podemos levar em consideração as hipóteses de interesse com base no ajuste selecionado, apesar de todos os resultados convergirem quanto à relação entre as variáveis explicativas e o desfecho. De acordo com as hipóteses iniciais levantadas, temos:

1. A chance de óbito em rodovias federais tem relação com o dia da semana: O dia da semana com maior fator de risco foi o domingo com 12,2% de chance a mais de óbito, enquanto o sábado oferece em torno de 4%. Os demais dias são fatores de proteção, de modo que a quarta foi o dia que oferece em torno de 10% de chance a mais de não ocorrer óbito;

2. A probabilidade de ocorrência de óbito em sinistros está relacionada ao tipo de pista: O tipo de pista simples aumenta em mais de 41% de chance mais de óbito, sendo um alto fator de risco, enquanto as pistas dupla e múltipla diminuem em aproximadamente 25% a chance de óbito;

3. A condição de visibilidade associada ao clima pode contribuir para ocorrência de óbito nos acidentes em rodovias federais: A visibilidade parcial aumenta em mais de 11% a chance de óbito, enquanto a má visibilidade reduz em mais de 26% a chance de óbito. Já a boa visibilidade praticamente não influencia a ocorrência de óbito;

4. A região do país tem relação influencia na chance de óbito no sinistro de trânsito em rodovias federais: A zona rural aumenta em 18% a chance de óbito, enquanto o trecho urbano reduz mais de 32% a chance de óbito. Além disso, pudemos ver que na análise de correspondência, e nas metodologias de aprendizado não supervisionado, que a predominância de acidentes está fortemente associado aos estados das regiões norte e nordeste.

Conclusão

A análise inferencial, realizada nos sinistros de trânsito nas rodovias federais de 2007 a 2026, revelou relações importantes entre as características. Das quatro hipóteses levantadas, todas foram validadas através de testes e métodos de aprendizado supervisionado e não supervisionado.

Todas avaliadas no estudo tiveram relação com a ocorrência de óbito. Fatores como zona rural, região norte/nordeste, dia da semana sexta/sábado/domingo e tipo de pista simples foram considerados fatores de risco, indicando um aumento na chance de óbito. Destaca-se que a relação com um maior risco de acidente fatal na região norte, seguida pela região nordeste, pode ser devido às condições estruturais das rodovias. Todas as variáveis avaliadas tiveram relação confirmadas estatisticamente pelos métodos empregados.

Há associação também entre a condição de visibilidade e a ocorrência de óbito. Segundo as análises, em especial as de aprendizado supervisionado, a visibilidade parcial tem maior incidência na ocorrência de acidentes fatais, sendo a condição boa e ruim relacionadas considerados fatores de proteção. Em resumo, a ocorrência de óbito no sinistro tem relação com a fase do dia, em que plena noite e amanhecer são fatores de risco; com o dia da semana, com predominância para domingo, sábado e sexta; com a

fase do dia, com a condição de visibilidade, de modo que a parcial é o fator de risco; e com o tipo de pista; em a pista simples é um fator de risco.

Além das relações diretas com as variáveis candidatas a fatores de risco para a ocorrência de óbito, as relações entre as variáveis foram confirmadas. Por exemplo, existe relação entre todas as variáveis entre si combinadas 2 a 2, que observado pelo teste qui-quadrado e pela medida V de Cramér, que apesar de não demonstrarem tanta força, mas têm relação. Tais resultados são relevantes para uma tomada de decisão mais segura para os condutores de modo que possam evitar acidentes fatais, bem como as instituições podem elaborar projetos públicos que conscientizem os condutores.

Quanto às **metodologias aplicadas**, foram aplicadas a regressão logística, a árvore de classificação, random forest, redes neurais, redes neurais profundas e SVM. Como os dados são desbalanceados, aplicamos as metodologias nos dados originais e nos dados após a aplicação do undersampling (quando reduzimos a elementos da categoria majoritária para equilibrar). Após o balanceamento dos dados, os resultados foram significativamente melhores e apontaram as **redes neurais profundas (RNP)** como superiores na classificação. Em seguida, tentou-se aprimorar a RNP, mas a demanda computacional aumentou o tempo e o processamento sem oferecer um ganho significativo. Portanto, a **RNP** foi considerado o melhor modelo para predição da ocorrência de acidentes nas rodovias federais.

Como **dificuldade** da pesquisa, tivemos primeiramente o **TEMPO DE EXECUÇÃO** e a questão do desbalanceamento entre as categorias, já que acidentes com óbito representam apenas 4,86% dos sinistros. Após a o balanceamento através do undersampling, ainda assim as medidas de qualidade não foram ótimas, visto que existem condições que favorecem o acidente fatal, não necessariamente são regra, pois existem condições que o óbito ocorre em condições inesperadas. Dessa forma, há uma dificuldade do modelo apresentar uma alta precisão. Como trabalho futuro, pode ser aplicado um modelo Bayesiano e uma análise discriminante para avaliar se tais abordagens podem apresentar alguma melhora na performance de classificação.

Referências

OMS (Organização Mundial da Saúde) Global status report on road safety 2018. Disponível em: <https://www.who.int/publications/i/item/9789241565684>

OMS (Organização Mundial da Saúde) Década de Ação pela Segurança no Trânsito 2021-2030. Disponível em: <https://www.paho.org/pt/noticias/28-10-2021-oms-lanca-decada-acao-pela-seguranca-no-transito-2021-2030>

PRF (POLÍCIA RODOVIÁRIA FEDERAL). Anuário 2021. Distrito federal, 2021. disponível em: https://www.gov.br/prf/pt-br/acao-a-informacao/dados-abertos/diest-arquivos/copy_of_anuario-2021_final.html

PRF (POLÍCIA RODOVIÁRIA FEDERAL). Acidentes. Disponível em: <https://www.gov.br/prf/pt-br/acao-a-informacao/dados-abertos/dados-abertos-acidentes>

AGRESTI, A. An introduction to categorical data analysis. 3rd ed. Hoboken, New Jersey: John Wiley Sons, 2019.

R Core Team. R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria, 2022. Disponível em: <https://www.R-project.org/>

ROSEMBERG, V. T. Estudo sobre a Incidência de Acidentes Fatais no Trânsito do Distrito Federal. Orientador: Prof. Leandro Tavares Correia. 2018.58 f. Trabalho de Conclusão de Curso– Universidade de Brasília– UnB Departamento de Estatística, Brasília- DF, 2018. Disponível em: https://bdm.unb.br/bitstream/10483/24356/1/2018_VitorTeofiloRosemberg_tcc.pdf

TRANSPORTE, C. N. do. Painel CNT de Acidentes Rodoviários-Principais dados-2021. CNT, 2021. Disponível em: <https://www.cnt.org.br/agencia-cnt/acidentes-e-mortes-nas-rodovias-federais-aumentam-em-2021>

MORAIS, Marcleiton Ribeiro et al. Letalidade do acidente de trânsito na modernista Palmas/TO: uma abordagem econométrica. Informe Gepec, v. 18, n. 1, p. 156-176, 2014.

MINGOTI, S. A Análise de Dados Através Métodos de Estatística Multivariada: Uma Abordagem Aplicada. 4. ed. Belo Horizonte: Editora UFMG, 2020.

ONSV (Observatório Nacional de Segurança Viária). Acidentes de trânsito são uma questão de saúde pública no Brasil Disponível em: <https://www.onsv.org.br/comunicacao/artigos/acidentes-de-transito-sao-uma-questao-de-saude-publica-no-brasil>